C++ 프로그래밍

코딩 테스트

- 문장별 근거 · 인쇄 점검

검증 범위

중복을 합친 1248개 항목의 문장·코드·수식과 원문 근거를 대조할 수 있습니다. 번호는 학습 노트·치트시트와 같습니다. 출처는 파일 코드와 원문 절·쪽수를 구분해 표시했습니다.

B의 쪽수는 인쇄 쪽수이며 PDF 면 번호에서 5를 뺀 값입니다. K는 PDF 면 번호, L은 강의 슬라이드 번호입니다. 근거 대조 문구는 원문 발췌 또는 원문을 줄인 확인 문구입니다. 인용된 관련 페이지의 문맥과 대조했으며, OCR 자료의 코드·수식과 주요 표는 원본 이미지로 확인했습니다.

인쇄 품질 점검

파일	점검 면	잘림·넘침	폰트	제목 고립	표 분리
학습 노트	1-177	이상 없음	이상 없음	이상 없음	분리 없음
확인·시험	1-25	이상 없음	이상 없음	이상 없음	분리 없음
근거·점검	1-159	이상 없음	이상 없음	이상 없음	분리 없음

최종 PDF의 모든 페이지를 이미지로 변환해 확인했습니다. 좌표·글꼴 포함·대체 글리프·제목·표 배치 검사 결과를 함께 대조했습니다.

01 개발환경·변수·기본 문법

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-001	강의 초반부는 메모리 레이아웃과 명령어에 따른 메모리의 변화를 이해하는 것을 주요 목표로 한다. 근거 대조 문구: 메모리레이아웃과명령어에따른메모리의변화이해	(L00 Course introduction, 슬라이드 5)
U01-002	배열, 명령문과 연산자, 조건문과 반복문을 다룬다. 근거 대조 문구: 배열 / 명령문과연산자 / 제어문	(L02 Today, 슬라이드 2)
U01-003	컴파일러는 소스 코드를 오브젝트 코드로 변환하는 도구이다. 근거 대조 문구: 컴파일러: 소스코드를오브젝트코드로변환하는도구	(L01 Terminologies, 슬라이드 5)
U01-004	링커는 오브젝트 코드를 실행파일(exe)로 변환하는 도구이다. 근거 대조 문구: 링커: 오브젝트코드를실행파일(exe)로변환하는도구	(L01 Terminologies, 슬라이드 5)
U01-005	IDE (Integrated Development Environment)는 텍스트 에디터, 컴파일러, 링커, 디버거를 묶은 개발환경이다. 근거 대조 문구: 텍스트에디터+ 컴파일러+ 링커+ (디버거)	(L01 Terminologies, 슬라이드 5)
U01-006	Compiler Warnings는 코드에 잠재적인 문제가 있을 것으로 예상될 때 발생한다. 근거 대조 문구: 코드에잠재적인문제가있을것으로예상될때	(L01 Compiler Warnings, 슬라이드 15)
U01-007	키워드는 언어 자체에서 예약된 단어이며, 식별자는 변수·함수·타입 등 개발자가 지정하는 부분이다. 근거 대조 문구: 언어자체에서예약된단어들 / 변수, 함수, 타입등개발자가지정하는부분	(L01 C++ Keyword, Identifier, Operator, 슬라이드 20)
U01-008	전처리기는 컴파일 이전에 처리되며 "#"으로 시작한다. 근거 대조 문구: 컴파일이전에처리됨 / “#” 으로시작	(L01 C++ preprocessor, 슬라이드 21)
U01-009	main() 함수는 프로그램의 진입점이다. 근거 대조 문구: 프로그램의진입점(=프로그램이실행되면가장먼저실행되는함수)	(L01 main() function, 슬라이드 25)
U01-010	cout과 <<는 표준 출력 스트림과 삽입 연산자이며, cin과 >>는 표준 입력 스트림과 추출 연산자이다. 근거 대조 문구: C++의표준출력스트림, 삽입연산자 / C++의표준입력스트림, 추출연산자	(L01 Basic I/O, 슬라이드 29)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-011	주석은 프로그래머에게 정보를 제공하기 위한 것이며 전처리 단계에서 무시(삭제)된다. 근거 대조 문구: 프로그래머가읽을수있는정보를제공 / 전처리단계에서무시(삭제)	(L01 C++ Comments, 슬라이드 24)
U01-012	이름공간(namespace)은 정의된 객체가 어디 소속인지 지정하며, 소속된 이름공간이 다르면 같은 이름도 다른 것으로 취급한다. 근거 대조 문구: 어떤정의된객체에대해어디소속인지지정 / 소속된이름공간이다르면다른것으로취급	(B 2장·이름공간(namespace), 인쇄 p.16)
U01-013	std는 C++ 표준 라이브러리의 모든 함수와 객체 등이 정의된 이름공간이다. 근거 대조 문구: std 는C++ 표준라이브러리의모든함수, 객체등이정의된이름공간	(B 2장·이름공간(namespace), 인쇄 p.16)
U01-014	이름 없는 이름공간에 정의된 것들은 해당 파일 안에서만 접근할 수 있다. 근거 대조 문구: 해당이름공간에정의된것들은해당파일안에서만접근할수있게됩니다	(B 2장·이름 없는 이름공간, 인쇄 p.20)
U01-015	책은 using namespace std; 대신 std::를 직접 붙여 소속을 명시하는 방식을 권장한다. 근거 대조 문구: std:: 를직접앞에붙여서std 의이름공간의함수이다라고명시	(B 2장·이름공간(namespace), 인쇄 p.19)
U01-016	메모리는 읽고 쓸 수 있는 바이트 공간의 집합이며, 강의에서는 메인 메모리(RAM)를 지칭한다. 근거 대조 문구: 메모리는읽고쓸수있는바이트공간의집합 / 메인메모리(RAM)를지칭	(L01 Memory, 슬라이드 36)
U01-017	변수는 메모리의 주소에 붙이는 이름이며, 변수를 만들면 타입에 따라 메모리 공간이 확보된다. 근거 대조 문구: 메모리의주소에붙이는이름! / 공간의크기(몇칸)는변수의타입에따라결정	(L01 Variable, 슬라이드 41)
U01-018	변수를 정의하면서 초기값을 설정하는 것을 초기화라고 한다. 근거 대조 문구: 변수를정의하면서초기값을설정하는것을초기화라고함	(L01 Definition and Initialization, 슬라이드 42)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-019	sizeof 연산자는 타입 또는 변수의 바이트 단위 크기를 리턴한다. 근거 대조 문구: 타입또는변수의바이트단위크기를리턴	(L01 Using the sizeof operator, 슬라이드 47)
U01-020	상수는 변수와 유사하지만 초기화 이후 변할 수 없는 값이다. 근거 대조 문구: 변수와유사하지만, 초기화이후변할수없는값	(L01 Constant, 슬라이드 49)
U01-021	배열은 동일한 타입인 요소들의 집합인 복합(compound) 데이터 타입이다. 근거 대조 문구: 복합(compound) 데이터타입 / 동일한타입인요소들의집합	(L02 Array, 슬라이드 5)
U01-022	배열은 길이가 고정되어 있고 연속된 메모리 주소에 저장된다. 근거 대조 문구: 고정된(정해진) 길이 / 연속된메모리주소에저장	(L02 Arrays, 슬라이드 6)
U01-023	배열 인덱스는 0부터 시작하여 마지막 인덱스는 size-1이며, out of bound를 체크하지 않는다. 근거 대조 문구: 인덱스는0부터시작하며, 마지막인덱스는size-1 / out of bound 체크하지않음	(L02 Arrays, 슬라이드 6)
U01-024	표현식은 코드의 가장 작은 구성요소이다. 근거 대조 문구: 코드의가장작은구성요소	(L02 Expressions, 슬라이드 14)
U01-025	명령문은 명령을 수행하는 코드 단위이며 세미콜론(";")으로 끝난다. 근거 대조 문구: 명령을수행하는코드단위 / 세미콜론(“;”) 으로끝나는문장	(L02 Statements, 슬라이드 15)
U01-026	대입 연산자는 오른쪽의 값을 계산하여 왼쪽에 대입하며, 왼쪽은 대입이 가능해야 한다. 근거 대조 문구: 오른쪽의값을계산하여왼쪽에대입 / 왼쪽은대입이가능해야함	(L02 Operations (1), 슬라이드 16)
U01-027	산술 연산자는 +, -, *, /, %(mod)이며 증감 연산자는 Prefix와 Postfix로 구분한다. 근거 대조 문구: +, -, *, /, %(mod) / Prefix / Postfix	(L02 Operations (2), 슬라이드 17)
U01-028	논리 연산자 !, &&, ||의 Short-circuit evaluation은 결과 파악이 가능하면 나머지 연산을 하지 않는다. 근거 대조 문구: !, &&, || / 결과파악이가능한경우나머지연산을하지않음	(L02 Operations (3), 슬라이드 18)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-029	if문은 표현식이 참일 경우에만 실행하는 명령문이다. 근거 대조 문구: 표현식이참일경우에만실행하는명령문	(L02 Conditional statement, 슬라이드 25)
U01-030	블록은 "{"로 시작하여 "}"로 끝나며, 블록 내에서 선언된 지역변수는 그 블록 내에서만 접근할 수 있다. 근거 대조 문구: 블록은“{“로시작해서“}”로끝남 / 블록내에서만접근가능	(L02 Conditional statement, 슬라이드 26)
U01-031	if-else문은 표현식의 참·거짓 여부에 따라 분기하고, switch문은 switch, case, default를 사용한 분기문이다. 근거 대조 문구: 표현문의참/거짓여부에따라명령문실행분기 / switch, case, default를사용한분기문	(L02 Conditional statement, 슬라이드 27,29)
U01-032	?: 연산자는 조건 표현식이 참이면 expr1의 값을, 거짓이면 expr2의 값을 리턴하는 삼항 연산자이다. 근거 대조 문구: 참이라면expr1의값을리턴 / 거짓이라면expr2의값을리턴 / 삼항연산자	(L02 Conditional statement, 슬라이드 30)
U01-033	강의는 반복문으로 for문, while문, do-while문을 구분한다. 근거 대조 문구: For문 / While, do-while문	(L02 (Summary) Controlling flow, 슬라이드 41)
U01-034	C++에서는 변수를 사용하기 직전 어느 위치에서든지 선언할 수 있고, 책의 for문 예제는 초기화 부분에서 int i를 선언한다. 근거 대조 문구: 변수를사용하기직전어느위치에서든지변수를선언 / for (int i = 1; i <= 10; i++)	(B 2장·C++ 와 C 언어의 공통 문법 구조, 인쇄 p.25)
U01-035	continue는 이후 문장을 실행하지 않고 다음 iteration으로 곧바로 넘어간다. 근거 대조 문구: Continue문이후의문장은실행되지않음 / 다음iteration으로곧바로넘어가기위해사용	(L02 Looping Statement, 슬라이드 39)
U01-036	break는 이후 문장을 실행하지 않고 루프 밖으로 바로 빠져나간다. 근거 대조 문구: Break문이후의문장은실행되지않음 / 루프밖으로바로빠져나가기위해사용	(L02 Looping Statement, 슬라이드 39)
U01-037	각 .cpp 소스 코드를 컴파일러가 .obj 오브젝트 코드로 변환한다. 근거 대조 문구: File1.cpp → File1.obj	(L01 C++ Build Process, 슬라이드 6)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-038	링커가 오브젝트 코드와 라이브러리를 연결하여 실행 프로그램(.exe)을 만든다. 근거 대조 문구: 링커 / C++ 표준라이브러리 / 실행프로그램(.exe)	(L01 C++ Build Process, 슬라이드 6)
U01-039	고정 크기의 배열을 정의한다. 근거 대조 문구: 정의와초기화- (고정크기선언	(L02 (Summary) Arrays, 슬라이드 11)
U01-040	리스트 초기화 "{}"를 사용하여 초기화한다. 근거 대조 문구: 리스트초기화“{}”	(L02 (Summary) Arrays, 슬라이드 11)
U01-041	인덱스로 개별 요소에 접근하여 사용한다. 근거 대조 문구: 인덱스로개별요소에접근하여사용	(L02 (Summary) Arrays, 슬라이드 11)

U01-042	첫 C++ 프로그램 - 원문 발췌 (원문 코드 발췌)
	#include <iostream>
	int main() {
	std::cout << "Hello, World!!" << std::endl;
	return 0;
	}
	근거 대조 문구: std::cout << "Hello, World!!" << std::endl;
		(B 2장·첫 C++ 프로그램 분석하기, 인쇄 p.15)

U01-043	iostream 헤더 파일은 C++의 표준 입출력에 필요한 것들을 포함한다. 근거 대조 문구: iostream 헤더파일은C++ 에서표준입출력에필요한것들을 포함	(B 2장·첫 C++ 프로그램 분석하기, 인쇄 p.15)
U01-044	byte와 bit: 1 byte = 8 bit = 2^8 = 256개의 숫자 표현 가능; 양수 범위 0~255 근거 대조 문구: 1 byte = 8 bit = 2^8 = 256개의 숫자 표현 가능	(L01 Bit, Byte, Hex, 슬라이드 35)
U01-045	Compiler Errors 근거 대조 문구: 프로그래밍언어의규칙을위반하는경우	(L01 Compiler Errors, 슬라이드 14)
U01-046	프로그래밍 언어의 규칙을 위반하는 경우이다. 근거 대조 문구: 프로그래밍언어의규칙을위반하는경우	(L01 Compiler Errors, 슬라이드 14)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-047	Linker Errors 근거 대조 문구: obj 파일의링크과정에서오류가있을경우	(L01 Linker Errors, 슬라이드 16)
U01-048	obj 파일의 링크 과정에서 오류가 있는 경우이다. 근거 대조 문구: obj 파일의링크과정에서오류가있을경우	(L01 Linker Errors, 슬라이드 16)
U01-049	Runtime Errors 근거 대조 문구: 프로그램의실행도중발생하는오류	(L01 Runtime Errors, 슬라이드 16)
U01-050	프로그램의 실행 도중 발생하는 오류이다. 근거 대조 문구: 프로그램의실행도중발생하는오류	(L01 Runtime Errors, 슬라이드 16)
U01-051	Logical Errors 근거 대조 문구: 프로그램의동작에관한논리적오류	(L01 Logical Errors, 슬라이드 17)
U01-052	프로그램의 동작에 관한 논리적 오류이다. 근거 대조 문구: 프로그램의동작에관한논리적오류	(L01 Logical Errors, 슬라이드 17)
U01-053	배열은 동일한 타입 요소들의 집합이며, 고정 크기로 정의하고 인덱스를 통해 사용한다. 근거 대조 문구: 동일한타입요소들의집합, 고정된길이 / 인덱스로개별요소에접근	(L02 (Summary) Arrays, 슬라이드 11)
U01-054	표현식은 가장 작은 단위이며 명령문은 세미콜론으로 끝을 표기하는 코드의 구성 단위이다. 근거 대조 문구: 표현식(가장작은단위 / 명령문(코드의구성단위, 세미콜론으로끝	(L02 (Summary) Statements and Operators, 슬라이드 21)
U01-055	제어 흐름에는 조건문과 반복문이 있으며, continue와 break를 사용한다. 근거 대조 문구: 조건문 / 반복문 / Continue, break	(L02 (Summary) Controlling flow, 슬라이드 41)
U01-056	컴파일러는 소스 코드를 오브젝트 코드로, 링커는 오브젝트 코드를 실행파일(exe)로 변환한다. 근거 대조 문구: 컴파일러: 소스코드를오브젝트코드로 / 링커: 오브젝트코드를실행파일	(L01 Terminologies, 슬라이드 5)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U01-057	첫 인덱스는 0이고 마지막 인덱스는 size-1이다. 근거 대조 문구: 인덱스는0부터시작하며, 마지막인덱스는size-1	(L02 Arrays, 슬라이드 6)
U01-058	continue는 다음 iteration으로 넘어가고, break는 루프 밖으로 빠져나간다. 근거 대조 문구: 다음iteration으로곧바로넘어가기 / 루프밖으로바로빠져나가기	(L02 Looping Statement, 슬라이드 39)
U01-059	continue 근거 대조 문구: continue	(L02 Looping Statement, 슬라이드 39)
U01-060	break 근거 대조 문구: break	(L02 Looping Statement, 슬라이드 39)

02 함수·포인터·참조·메모리

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-001	함수의 정의·프로토타입·매개변수·return·기본 인수·오버로딩·호출 동작·inline·재귀를 다룬다. 근거 대조 문구: 정의 / 프로토타입 / 매개변수 / return문 / 기본인수 / 오버로딩 / 함수호출 / inline / 재귀함수	(L03 Today, 슬라이드 3)
U02-002	포인터의 정의·주소 접근·역참조·동적 할당과 참조자 및 전달 방식의 차이를 다룬다. 근거 대조 문구: 포인터의정의 / 주소로의접근 / 역참조 / 동적메모리할당 / 참조자란? / 포인터vs 참조자	(L04 (Summary) Pointers·References, 슬라이드 4,41)
U02-003	함수 정의에 필요한 요소는 이름, 매개변수 리스트, 리턴 타입, 본문(body)이다. 근거 대조 문구: 이름 / 매개변수리스트 / 리턴타입 / 본문(body)	(L03 Function Definition, 슬라이드 10)
U02-004	함수 프로토타입은 함수를 사용하기 전에 입력 매개변수와 반환형을 미리 알려주는 명령문이다. 근거 대조 문구: 함수를사용하기이전에입력매개변수와반환형을미리알려주는명령문	(L03 Function Prototypes, 슬라이드 20)
U02-005	return문은 함수의 결과값을 전달하며 함수를 즉각적으로 종료한다. 근거 대조 문구: return 문을통해서함수의결과값을전달 / 함수는즉각적으로종료	(L03 Function Return, 슬라이드 28)
U02-006	반환값이 없는 경우 반환형을 void로 명시한다. 근거 대조 문구: 반환값이없는경우void로명시	(L03 Function Definition, 슬라이드 13)
U02-007	호출에서 전달하는 값은 인수(argument), 함수 정의의 전달 항목은 인자 또는 매개변수(parameter)이며 개수·순서·타입이 일치해야 한다. 근거 대조 문구: 인수(argument) / 인자또는매개변수(parameter) / 개수, 순서와타입이일치	(L03 Function Parameters, 슬라이드 23)
U02-008	pass-by-value에서는 데이터 값이 복사되어 전달되므로 함수 안의 사본을 바꾸어도 원본은 바뀌지 않는다. 근거 대조 문구: 데이터의값이복사되어전달 / 사본을바꾼다고원본이바뀌지않음	(L03 Pass-by-value, 슬라이드 25)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-009	기본 인수는 인수가 주어지지 않았을 때 사용할 기본값으로, 프로토타입 또는 정의부 중 한쪽에만 선언하며 여러 개면 오른쪽부터 선언한다. 근거 대조 문구: 인수가주어지지않을시기본값 / 둘다선언해서는안됨 / 오른쪽부터선언	(L03 Default Argument Values, 슬라이드 29)
U02-010	함수 오버로딩은 서로 다른 매개변수 리스트를 갖는 동일한 이름의 함수를 정의하는 것이다. 근거 대조 문구: 서로다른매개변수리스트를갖는동일한이름의함수를정의	(L03 Function Overloading, 슬라이드 32)
U02-011	반환 타입만 다른 함수 오버로딩은 불가능하다. 근거 대조 문구: 반환타입만다른오버로딩은불가능	(L03 Function Overloading, 슬라이드 34)
U02-012	함수의 매개변수와 지역변수는 함수 실행 중에만 존재한다. 근거 대조 문구: 함수의(복사된) 인자및지역변수들은함수의실행중에만존재	(L03 Local/Global Scope, 슬라이드 36)
U02-013	static 지역변수는 지역 밖에 정의한 것처럼 활용할 수 있지만 scope 밖에서 접근할 수는 없다. 근거 대조 문구: 지역밖에정의한것처럼활용 / scope 밖에서접근할수는없음	(L03 Local/Global Scope, 슬라이드 37)
U02-014	강의의 inline 설명은 컴파일 단계에서 함수 호출을 함수 내 명령문으로 대체하며 바이너리 파일 용량이 커질 수 있다고 정리한다. 근거 대조 문구: 컴파일단계에서함수내의명령문으로함수호출이대체 / 바이너리파일의용량이커질수있음	(L03 inline function, 슬라이드 61)
U02-015	재귀함수는 스스로를 호출하는 함수이며, 재귀 호출을 끝내는 base case가 반드시 실행되어야 한다. 근거 대조 문구: 스스로를호출하는함수 / 재귀호출을끝내는base case가반드시실행	(L03 Recursive Function, 슬라이드 63)
U02-016	포인터는 주소값을 저장하는 변수이다. 근거 대조 문구: 포인터변수는주소값을저장	(L04 Accessing Pointer Address, 슬라이드 10)
U02-017	포인터는 타입 뒤에 *를 붙여 정의하며, nullptr은 아무것도 가리키지 않는 상태를 의미한다. 근거 대조 문구: 기존변수타입뒤에“*” / 아무것도가리키지않는상태	(L04 Definition of Pointers, 슬라이드 9)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-018	주소 연산자 &는 적용한 피연산자의 주소값을 반환한다. 근거 대조 문구: 연산자가적용되는피연산자의주소값이반환	(L04 Accessing Pointer Address, 슬라이드 10)
U02-019	역참조는 * 연산자로 포인터의 주소에 저장된 데이터에 접근하는 것이다. 근거 대조 문구: 포인터의주소에저장된데이터에접근 / * 연산자를사용	(L04 Dereferencing a Pointer, 슬라이드 15)
U02-020	배열 이름은 첫 요소의 주소를 가리키며, 배열과 포인터는 정의 후 주소 변경 가능 여부와 sizeof 반환값에 차이가 있다. 근거 대조 문구: 배열의이름은배열의첫요소의주소 / 배열은주소값을정의이후변경불가. Sizeof() 반환값이다름	(L04 Arrays and Pointers, 슬라이드 26)
U02-021	참조자는 변수의 별명이며 선언과 동시에 초기화되어야 하고, 한 번 초기화되면 다른 변수의 참조자가 될 수 없다. 근거 대조 문구: 변수의별명 / 선언과동시에초기화되어야함 / 다른변수의참조자가될수없음	(L04 Reference, 슬라이드 42)
U02-022	상수 참조자로 선언하면 리터럴도 참조할 수 있다. 근거 대조 문구: 상수참조자로선언한다면리터럴도참조할수있습니다	(B 3장·상수에 대한 참조자, 인쇄 p.38)
U02-023	레퍼런스의 배열은 불가능하지만 배열의 레퍼런스는 가능하며, 배열 크기에 맞는 별명이 되어야 한다. 근거 대조 문구: arrays of references are illegal / 배열들의레퍼런스가불가능한것은아닙니다 / 반드시크기가3 인int 배열의별명	(B 3장·레퍼런스의 배열과 배열의 레퍼런스, 인쇄 p.38-40)
U02-024	댕글링 레퍼런스(Dangling reference)는 레퍼런스는 있지만 원래 참조하던 것이 사라진 레퍼런스이다. 근거 대조 문구: 레퍼런스는있는데원래참조하던것이사라진레퍼런스	(B 3장·지역변수의 레퍼런스를 리턴?, 인쇄 p.42)
U02-025	상수 레퍼런스로 함수의 리턴값을 받으면 그 값의 수명은 레퍼런스가 사라질 때까지 연장된다. 근거 대조 문구: 상수레퍼런스로리턴값을받게되면 / 레퍼런스가사라질때까지	(B 3장·참조자가 아닌 값을 리턴하는 함수를 참조자로 받기, 인쇄 p.45)
U02-026	동적 메모리 할당은 런타임에 힙 메모리를 할당하는 것이다. 근거 대조 문구: 런타임에“힙메모리”를할당	(L04 Dynamic Memory Allocation, 슬라이드 18)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-027	힙 메모리는 스스로 해제되지 않으므로 사용 후 해제하지 않으면 메모리 누수가 발생한다. 근거 대조 문구: 힙메모리는스택과는달리스스로해제되지않음 / 메모리누수발생	(L04 Dynamic Memory Allocation, 슬라이드 18)
U02-028	new와 delete가 짝을 이루고, new []와 delete []가 짝을 이룬다. 근거 대조 문구: new - delete 가짝 / new [] 와delete [] 가짝	(B 4장·new 로 배열 할당하기, 인쇄 p.51)
U02-029	함수 내부에서 동적으로 할당한 메모리의 주소는 반환할 수 있지만 지역변수에 대한 포인터를 반환해서는 안 된다. 근거 대조 문구: 동적으로할당된메모리의주소를반환→OK / 지역변수에대한포인터반환→안됨	(L04 Returning a Pointer, 슬라이드 36-37)
U02-030	pointers to const는 데이터가 const이고, 포인터는 다른 데이터를 가리킬 수 있다. 근거 대조 문구: 데이터가const / 포인터는다른데이터를가리킬수있음	(L04 Const and Pointers, 슬라이드 29)
U02-031	const pointers는 포인터가 const이고 데이터는 변할 수 있으며, const pointers to const는 둘 다 const이다. 근거 대조 문구: 포인터가const / 데이터는변할수있음 / 둘다const	(L04 Const and Pointers, 슬라이드 30-31)
U02-032	포인터를 전달하면 주소값을 통해 함수 범위 밖의 메모리에 접근할 수 있다. 근거 대조 문구: 주소값을명시하는포인터를활용하여범위밖메모리에접근	(L03 Pass-by-address, 슬라이드 55)
U02-033	const 참조자를 사용하는 전달은 원본을 수정하지 않고 복사 오버헤드가 클 때, 매개변수가 null이 되지 않음이 보장될 때 사용한다. 근거 대조 문구: 실제매개변수를수정하지않는경우 / 복사의오버헤드가클때 / null이되지않는것이보장	(L04 Pointers vs References, 슬라이드 56)
U02-034	동물 관리 예제는 Animal의 이름·나이·체력·배부른 정도·깨끗한 정도를 저장하고, play와 show_stat으로 상태 변경·출력을 수행한다. 근거 대조 문구: 이름 / 나이 / 체력 / 배부른정도 / 깨끗한정도 / play / show_stat	(B 4장·돌아온 마이펫, 인쇄 p.51)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-035	책은 new Animal 뒤에 초기화 함수를 따로 호출해야 하는 문제와, Bird·Fish에 공통 부분을 반복 작성하는 문제를 객체지향 도입에 연결한다. 근거 대조 문구: 반드시create_animal 함수를호출 / Animal 과동일한부분은가져다쓰고 / 객체지향프로그래밍의세계	(B 4장·돌아온 마이펫, 인쇄 p.55-56)
U02-036	함수를 호출하면 호출 영역을 구분하는 구분선이 생긴다. 근거 대조 문구: 함수를호출하면? →구분선이생김	(L03 (Summary) Functions, 슬라이드 64)
U02-037	그 영역에는 매개변수를 포함한 지역변수 공간이 들어간다. 근거 대조 문구: 지역변수공간(매개변수포함)	(L03 (Summary) Functions, 슬라이드 64)
U02-038	함수가 끝나면 그 공간은 자동으로 사라진다. 근거 대조 문구: 함수가끝나면? →자동으로사라짐	(L03 (Summary) Functions, 슬라이드 64)
U02-039	new로 공간을 할당하고 반환된 주소값을 포인터에 저장한다. 근거 대조 문구: int 크기의공간을할당하여그주소값을p 에	(B 4장·new 와 delete 의 사용, 인쇄 p.47)
U02-040	포인터를 역참조하여 값을 저장하고 출력한다. 근거 대조 문구: *p = 10; / 값을집어넣었고이를출력	(B 4장·new 와 delete 의 사용, 인쇄 p.47)
U02-041	마지막에 delete로 할당된 공간을 해제한다. 근거 대조 문구: 마지막으로할당된공간을해제하기위해서delete	(B 4장·new 와 delete 의 사용, 인쇄 p.47)

U02-042	참조자 매개변수 - 원문 발췌 (원문 코드 발췌)
	int change_val(int &p) {
	p = 3;
	
	return 0;
	}
	근거 대조 문구: int change_val(int &p) { p = 3; return 0; }
		(B 3장·참조자를 함수 인자로 받기, 인쇄 p.35)

U02-043	함수 안에서 p = 3;을 수행하는 것은 main 함수의 number에 number = 3;을 수행하는 것과 같다. 근거 대조 문구: main 함수의number 에number = 3; 을하는것과정확히같은작업	(B 3장·참조자를 함수 인자로 받기, 인쇄 p.35)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U02-044	new와 delete - 원문 발췌 (원문 코드 발췌)
	#include <iostream>
	int main() {
	int* p = new int;
	*p = 10;
	std::cout << *p << std::endl;
	delete p;
	return 0;
	}
	근거 대조 문구: int* p = new int; / delete p;
		(B 4장·new 와 delete 의 사용, 인쇄 p.46)

U02-045	new는 메모리를 할당하고 delete는 메모리를 해제한다. 근거 대조 문구: new 는 / 메모리를할당하고delete 는 / 메모리를해제	(B 4장·C++ 의 세계로, 인쇄 p.46)
U02-046	Pass-by-value 근거 대조 문구: 함수가실제매개변수(원본)를수정하지않는경우 / 크기가작은기본자료형	(L04 Pointers vs References, 슬라이드 51)
U02-047	원본을 수정하지 않을 때, 크기가 작은 기본 자료형을 전달한다. 근거 대조 문구: 함수가실제매개변수(원본)를수정하지않는경우 / 크기가작은기본자료형	(L04 Pointers vs References, 슬라이드 51)
U02-048	Pass-by-address 근거 대조 문구: 실제매개변수(원본)를수정해야하는경우 / nullptr이되어도상관없을때	(L04 Pointers vs References, 슬라이드 52)
U02-049	원본 수정이 필요하고, 포인터가 nullptr이어도 상관없을 때 사용한다. 근거 대조 문구: 실제매개변수(원본)를수정해야하는경우 / nullptr이되어도상관없을때	(L04 Pointers vs References, 슬라이드 52)
U02-050	Pass-by-reference 근거 대조 문구: 실제매개변수를수정하는경우 / null이되지않는것이보장될때	(L04 Pointers vs References, 슬라이드 55)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U02-051	원본을 수정하고, 매개변수가 null이 되지 않음이 보장될 때 사용한다. 근거 대조 문구: 실제매개변수를수정하는경우 / null이되지않는것이보장될 때	(L04 Pointers vs References, 슬라이드 55)
U02-052	함수는 매개변수를 복사하여 전달하고, 포인터와 참조자를 사용하여 다른 메모리 영역의 변수를 변경할 수 있다. 근거 대조 문구: 인자는복사하여전달 / 다른메모리영역의변수를변환하고싶을때사용	(L03 (Summary) Functions, 슬라이드 64)
U02-053	포인터는 *, 주소 접근은 &, 역참조는 *를 사용하며 동적 메모리는 new와 delete로 다룬다. 근거 대조 문구: 포인터는변수, * 사용 / 주소로의접근(& / 역참조(* / 동적메모리할당(new, delete	(L04 (Summary) Pointers, 슬라이드 39)
U02-054	참조자는 변수의 별명이며, 전달 방식에는 값·포인터·참조자 및 const를 사용하는 구분이 있다. 근거 대조 문구: 변수의별명 / Pass-by-value / Pointer / Reference / Const reference	(L04 (Summary) References, 슬라이드 57)
U02-055	반환 타입만 다른 오버로딩은 불가능하다. 근거 대조 문구: 반환타입만다른오버로딩은불가능	(L03 Function Overloading, 슬라이드 34)
U02-056	new []로 할당한 배열은 delete []로 해제한다. 근거 대조 문구: new [] 와delete [] 가짝	(B 4장·new 로 배열 할당하기, 인쇄 p.51)

03 클래스·생성·복사·const

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-001	절차적 프로그래밍과 객체지향 프로그래밍, 클래스·객체·생성자·소멸자·생성자 오버로딩을 다룬다. 근거 대조 문구: 객체지향프로그래밍 / 클래스와객체 / 생성자와소멸자 / 생성자오버로딩	(L05A Lecture Plan, 슬라이드 2,4)
U03-002	문자열 클래스 프로젝트는 배운 클래스의 중요한 내용을 복습하고 점검하기 위한 것이다. 근거 대조 문구: 클래스의중요한내용들을복습하고점검	(B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.138)
U03-003	절차적 프로그래밍은 데이터와 작업이 분리되고, 객체지향 프로그래밍은 데이터와 기능을 캡슐화한다. 근거 대조 문구: 데이터와작업이분리 / 데이터+ 기능캡슐화	(L05A (Summary) Object Oriented Programming, 슬라이드 10)
U03-004	캡슐화는 클래스가 데이터와 데이터를 기반으로 하는 기능을 모두 포함하는 것이다. 근거 대조 문구: 클래스는(데이터) + (데이터를기반으로기능)을모두포함	(L05A Object Oriented Programming, 슬라이드 8)
U03-005	정보은닉에서는 내부 구현을 감추고 외부로 노출된 인터페이스를 사용하게 한다. 근거 대조 문구: 사용자는내구부현에대해알필요도없고 / 외부로노출된인터페이스만활용가능	(L05A Object Oriented Programming, 슬라이드 8)
U03-006	강의는 객체지향 프로그래밍의 한계로 성능 손실 가능성과 설계에 필요한 추가 수고를 제시한다. 근거 대조 문구: 성능손실가능성, 디자인에더많은수고필요	(L05A (Summary) Object Oriented Programming, 슬라이드 10)
U03-007	클래스는 객체가 생성되기 위한 틀이며 객체의 데이터와 기능을 정의하는 사용자 정의 자료형이다. 근거 대조 문구: 객체(object)가생성되기위한틀 / 사용자정의“자료형”	(L05A Classes and Objects, 슬라이드 14)
U03-008	객체는 클래스로부터 생성된 실체이다. 근거 대조 문구: 클래스로부터생성된실체	(L05A Classes and Objects, 슬라이드 15)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-009	public은 객체에서 접근할 수 있고 private은 클래스 멤버들만 접근할 수 있다. 근거 대조 문구: public / 객체에서접근가능 / private / 클래스의멤버들만접근가능	(L05A Class Member Access Modifier, 슬라이드 24)
U03-010	멤버함수는 멤버변수에 직접 접근할 수 있으며, 클래스 안 또는 ClassName::MethodName 형식으로 클래스 밖에 구현할 수 있다. 근거 대조 문구: 멤버변수에접근이가능 / 클래스선언내에구현가능 / ClassName::MethodName	(L05A Implementing Member Methods, 슬라이드 29)
U03-011	명세와 구현을 분리할 때 클래스 선언은 .h에, 구현은 .cpp에 작성하고 Include guard로 헤더의 중복 포함을 방지한다. 근거 대조 문구: 클래스의선언은.h / 구현은.cpp / 중복적인헤더파일의포함을방지	(L05A Implementing Member Methods, 슬라이드 29,32)
U03-012	class의 기본 접근권한은 private이고 struct의 기본 접근권한은 public이다. 근거 대조 문구: 클래스: / private이기본값 / 구조체: / public이기본값	(L05A Struct vs Class, 슬라이드 36)
U03-013	생성자는 객체가 생성될 때 자동 호출되는 특수한 멤버함수로, 클래스와 이름이 같고 반환형이 없다. 근거 대조 문구: 객체가생성될때자동으로호출 / 클래스와동일한이름 / 반환형은존재하지않음	(L05A Constructors, 슬라이드 41)
U03-014	소멸자는 객체가 소멸할 때 자동 호출되며 클래스 이름 앞에 ~를 붙이고 반환형·매개변수가 없다. 근거 대조 문구: 객체가소멸할때자동으로호출 / 이름앞에“~” / 반환형및파라메터는존재하지않음	(L05A Destructor, 슬라이드 44)
U03-015	생성자는 매개변수가 다른 고유한 형태로 오버로딩할 수 있으나 소멸자는 오버로딩할 수 없다. 근거 대조 문구: 각각의생성자는고유해야함(매개변수가달라야함) / 오버로딩불가능	(L05A Overloading Constructor·Destructor, 슬라이드 44,53)
U03-016	기본 생성자는 인자가 없는 생성자이며, 생성자를 직접 구현하지 않으면 컴파일러가 만들어 준다. 근거 대조 문구: 인자가없는생성자 / 클래스의생성자를직접구현하지않으면	(L05A Default Constructor, 슬라이드 47)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-017	인자가 있는 생성자만 구현한 경우 기본 생성자는 자동 생성되지 않는다. 근거 대조 문구: 인자가있는생성자만구현한경우 / 기본생성자가자동생성되지않음	(L05A Declaring a Class, 슬라이드 49)
U03-018	생성자 초기화 리스트는 생성자 본체에서 값을 대입하는 대신 생성과 동시에 값을 지정한다. 근거 대조 문구: 생성과동시에값이지정됨	(L05A Constructor Initialization List, 슬라이드 56)
U03-019	클래스 안의 레퍼런스 변수나 상수는 생성자의 초기화 리스트로 초기화해야 한다. 근거 대조 문구: 레퍼런스변수나상수를 / 무조건초기화리스트를사용해서초기화	(B 5장·const, const, const!·생성자의 초기화 리스트, 인쇄 p.113)
U03-020	생성자 위임은 다른 생성자를 멤버 초기화 리스트 위치에서 호출하여 코드 반복을 줄이는 방법이다. 근거 대조 문구: 코드반복을줄일 / 다른생성자를멤버초기화리스트위치에서호출	(L05A Delegating Constructors, 슬라이드 57)
U03-021	생성자도 함수이므로 기본 매개변수를 사용할 수 있다. 근거 대조 문구: 생성자또한함수이므로, 기본매개변수사용가능	(L05A Default Constructor Parameters, 슬라이드 60)
U03-022	복사 생성자는 기존 객체를 기반으로 새로운 객체가 생성될 때 어떻게 복사할지를 정의한다. 근거 대조 문구: 기존객체를기반으로새로운객체가생성 / 객체가“어떻게” 복사될지정의	(L05B Copy Constructor, 슬라이드 6)
U03-023	강의는 값으로 매개변수 전달, 값으로 결과 반환, 기존 객체를 기반으로 새 객체 생성의 경우에 복사 생성이 일어난다고 설명한다. 근거 대조 문구: pass by value / value의형태로결과를반환 / 기존객체를기반으로새로운객체	(L05B Copy Constructor, 슬라이드 6)
U03-024	직접 구현하지 않으면 기본 복사 생성자가 자동 생성되며 멤버변수의 값을 복사하여 대입한다. 근거 대조 문구: 구현하지않으면 / 자동으로만들어져 / 멤버변수들의값을복사하여대입	(L05B Copy Constructor, 슬라이드 10)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-025	얕은 복사는 포인터가 가리키는 데이터가 아니라 포인터 주소값 자체를 복사한다. 근거 대조 문구: 데이터가아닌포인터주소값자체가복사됨	(L05B Shallow vs Deep Copy, 슬라이드 14)
U03-026	깊은 복사는 새로운 힙 공간을 할당한 뒤 동일한 데이터를 복사하여 복사 생성하는 방식이다. 근거 대조 문구: 새로운힙공간을할당한뒤동일한데이터복사	(L05B Shallow vs Deep Copy, 슬라이드 20)
U03-027	강의의 얕은 복사 예제는 같은 메모리를 두 번 해제하려 하면서 오류가 발생한다. 근거 대조 문구: 해제된공간을다시해제하려시도!→에러	(L05B Shallow vs Deep Copy, 슬라이드 18)
U03-028	this 포인터는 멤버함수를 호출한 객체의 주소값이다. 근거 대조 문구: 멤버함수를호출한객체의주소값	(L05B this Pointer, 슬라이드 26)
U03-029	값이 바뀌지 않음을 보장하는 멤버함수는 선언 뒤에 const를 붙여 const 객체에서 호출할 수 있게 한다. 근거 대조 문구: 멤버함수선언뒤에const를붙인다 / 값이바뀌지않는것이보장된함수를선별	(L05B const Object, 슬라이드 31)
U03-030	static 클래스 멤버변수와 멤버함수는 객체가 아니라 클래스에 속한다. 근거 대조 문구: 객체가아닌클래스에속하는변수 / 객체가아닌클래스에속하는함수	(L05B Static Class Members, 슬라이드 32)
U03-031	friend는 특정 함수나 클래스에 private 멤버 접근을 허용하며, 관계는 비대칭이고 전이되지 않는다. 근거 대조 문구: Private 멤버에대해접근을허용 / 비대칭 / 전이되지않음	(L05B Friends of a Class, 슬라이드 37)
U03-032	MyString은 문자열 데이터를 가리키는 포인터와 문자열 길이를 보관하여 문자열 크기 변경과 반복적인 길이 계산을 다룬다. 근거 대조 문구: 문자열데이터가저장된공간을가리키는포인터 / 문자열데이터의길이 / 길이를한번구해놓고	(B 5장·내가 만들어보는 문자열 클래스·문자열 클래스를 만들자, 인쇄 p.139)
U03-033	MyString은 string_length를 별도로 보관하므로 실제 문자에 해당하는 내용만 string_content에 저장한다. 근거 대조 문구: 실제문자에만해당하는내용만을string_content 에보관	(B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.140)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-034	memory_capacity는 현재 할당된 메모리 공간의 크기이며, reserve는 미리 공간을 예약하고 capacity는 할당된 크기를 알려준다. 근거 대조 문구: 현재할당된메모리공간의크기 / 미리예약해놓는reserve / capacity	(B 5장·내가 만들어보는 문자열 클래스·assign 함수, 인쇄 p.145-146)
U03-035	assign은 기존 문자열을 지정한 문자열로 바꾸며, insert는 지정 위치 앞에 문자열을 삽입하고 erase는 지정 위치에서 문자를 지운다. 근거 대조 문구: 원래있었던문자열이지워지고abc / 입력위치'앞' / num 은지우는문자의수	(B 5장·내가 만들어보는 문자열 클래스·assign·insert·erase, 인쇄 p.143,151,164)
U03-036	find는 find_from에서 시작하여 처음 발견한 문자열의 위치를 반환하며, 포함되지 않으면 -1을 반환한다. 근거 대조 문구: find_from 에서부터 / 가장첫번째str 의위치를리턴 / -1 을리턴	(B 5장·내가 만들어보는 문자열 클래스·find 함수, 인쇄 p.165)
U03-037	compare의 크기 비교는 문자열을 사전식으로 배열했을 때 어느 문자열이 뒤에 오는지를 판단하는 것이다. 근거 대조 문구: 사전식으로배열해서어떤문자열이더뒤에오는지판단	(B 5장·내가 만들어보는 문자열 클래스·compare, 인쇄 p.171)
U03-038	생성자를 explicit으로 선언하면 그 생성자를 이용한 암시적 변환을 막을 수 있다. 근거 대조 문구: 이생성자를이용한암시적변환을수행하지못하게막을수있습니다	(B 5장·클래스의 explicit과 mutable 키워드·explicit, 인쇄 p.183)
U03-039	mutable로 선언한 멤버변수는 const 멤버함수 안에서도 값을 바꿀 수 있다. 근거 대조 문구: 멤버변수를mutable 로선언하였다면const 함수에서도이들값을바꿀수있습니다	(B 5장·클래스의 explicit과 mutable 키워드·mutable, 인쇄 p.183)
U03-040	책의 Server 예제는 const인 GetUserInfo에서 캐쉬를 갱신하기 위해 Cache 멤버를 mutable로 선언한다. 근거 대조 문구: GetUserInfo 가const / mutable Cache cache;	(B 5장·클래스의 explicit과 mutable 키워드·mutable, 인쇄 p.186-187)
U03-041	원본의 문자열 길이를 복사하고 그 길이를 할당 용량으로 설정한다. 근거 대조 문구: string_length = str.string_length; / memory_capacity = str.string_length;	(B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-042	문자열 길이만큼 새 char 배열을 할당한다. 근거 대조 문구: string_content = new char[string_length];	(B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)
U03-043	원본의 문자를 새 배열에 차례로 복사한다. 근거 대조 문구: string_content[i] = str.string_content[i];	(B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)
U03-044	기존 주소를 보관하고 새 메모리를 할당한다. 근거 대조 문구: prev_string_content = string_content / new char[memory_capacity]	(B 5장·내가 만들어보는 문자열 클래스·insert, 인쇄 p.152)
U03-045	삽입 위치 앞부분, 삽입할 문자열, 기존 문자열의 나머지 뒷부분 순서로 복사한다. 근거 대조 문구: insert 되는부분직전까지 / 새롭에insert 되는문자열 / 원문자열의나머지뒷부분	(B 5장·내가 만들어보는 문자열 클래스·insert, 인쇄 p.152)
U03-046	기존 메모리를 해제하고 문자열 길이를 갱신한 뒤 *this를 반환한다. 근거 대조 문구: delete[] prev_string_content; / string_length = string_length + str.string_length; / return *this;	(B 5장·내가 만들어보는 문자열 클래스·insert, 인쇄 p.152)

U03-047	MyString 복사 생성자 - 원문 발췌 (원문 코드 발췌)
	MyString::MyString(const MyString& str) {
	string_length = str.string_length;
	memory_capacity = str.string_length;
	string_content = new char[string_length];
	
	for (int i = 0; i != string_length; i++) {
	string_content[i] = str.string_content[i];
	}
	}
	근거 대조 문구: MyString::MyString(const MyString& str) / string_content = new char[string_length];
		(B 5장·내가 만들어보는 문자열 클래스, 인쇄 p.148)

U03-048	얕은 복사 근거 대조 문구: 얕은복사의문제점	(L05B Shallow vs Deep Copy, 슬라이드 14)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-049	깊은 복사 근거 대조 문구: 깊은복사	(L05B Shallow vs Deep Copy, 슬라이드 20)
U03-050	클래스는 객체의 틀이며, 접근제한자로 멤버 접근을 제한하고 .h 명세와 .cpp 구현을 분리한다. 근거 대조 문구: 객체의틀 / 접근제한자 / .h파일에명세, .cpp 파일에구현	(L05A (Summary) Class and Object, 슬라이드 38)
U03-051	생성자 초기화 리스트는 생성과 동시에 값을 지정하고, 생성자 위임과 기본 매개변수를 사용할 수 있다. 근거 대조 문구: 생성과동시에값의지정 / 생성자위임 / 기본매개변수	(L05A (Summary) Constructor Overloading, 슬라이드 62)
U03-052	기본 복사 생성자는 멤버변수를 그대로 대입하며, 깊은 복사는 새 힙 공간을 할당하여 복사 생성한다. 근거 대조 문구: 멤버변수를그대로대입(얕은복사) / 새로운힙공간을할당하여복사생성	(L05B (Summary) Copy Constructor, 슬라이드 23)
U03-053	this는 현재 객체의 주소값이고, const·static·friend는 멤버의 변경 가능성·클래스 소속·접근 허용에 관계된다. 근거 대조 문구: 현재객체의주소값 / const / 객체가아닌클래스에속하는 / Private에접근을허용	(L05B (Summary) this, const, static, friend, 슬라이드 40)
U03-054	클래스는 객체 생성의 틀이자 사용자 정의 자료형이고, 객체는 클래스로부터 생성된 실체이다. 근거 대조 문구: 객체가생성되기위한틀 / 사용자정의 / 클래스로부터생성된실체	(L05A Classes and Objects, 슬라이드 14-15)
U03-055	생성자의 초기화 리스트를 사용하여 초기화해야 한다. 근거 대조 문구: 무조건초기화리스트를사용해서초기화	(B 5장·const, const, const!·생성자의 초기화 리스트, 인쇄 p.113)
U03-056	얕은 복사는 주소값을 복사하고, 깊은 복사는 새 힙 공간에 데이터를 복사한다. 근거 대조 문구: 포인터주소값자체가복사 / 새로운힙공간을할당한뒤동일한데이터복사	(L05B Shallow vs Deep Copy, 슬라이드 14,20)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U03-057	explicit은 해당 생성자를 이용한 암시적 변환을 막고, mutable은 const 함수 안에서 해당 멤버변수를 바꿀 수 있게 한다. 근거 대조 문구: 암시적변환을수행하지못하게 / const 함수에서도이들값을바꿀수	(B 5장·클래스의 explicit과 mutable 키워드, 인쇄 p.183)

04 상속·다형성

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-001	상속·유도 클래스·protected·생성자와 소멸자·기본 클래스 생성자와 멤버함수의 관계를 다룬다. 근거 대조 문구: 상속의정의 / 유도클래스 / protected / 생성자와소멸자 / 기본클래스의생성자 / 멤버함수	(L06 Inheritance, 슬라이드 8)
U04-002	다형성·동적 바인딩·가상함수·override/final·순수가상함수·추상클래스·인터페이스를 다룬다. 근거 대조 문구: 다형성과동적바인딩 / 가상함수 / override/final / 추상클래스 / 인터페이스	(L07 Polymorphism, 슬라이드 4)
U04-003	상속은 기본 클래스를 기반으로 새 클래스를 만드는 기능이다. 근거 대조 문구: 기본클래스를기반으로새클래스만드는기능	(L06 (Summary) Inheritance, 슬라이드 53)
U04-004	기본 클래스(base class/parent class/super class)는 상속의 대상이 되는 클래스이다. 근거 대조 문구: 상속의대상이되는클래스	(L06 Terminology, 슬라이드 16)
U04-005	유도 클래스(derived class/child class/sub class)는 기본 클래스로부터 생성되며 데이터와 행동을 상속받는 클래스이다. 근거 대조 문구: 기본클래스로부터생성 / 데이터와행동을기본클래스로부터상속	(L06 Terminology, 슬라이드 16)
U04-006	Manager는 Employee라는 is-a 관계이고, EmployeeList가 Employee를 포함하는 관계는 has-a이다. 근거 대조 문구: Manager is a Employee / EmployeeList 는Employee 들과has - a 관계	(B 7장·가상 함수와 다형성·is-a와 has-a, 인쇄 p.298,300)
U04-007	protected 멤버는 기본·유도 클래스 내부에서 접근 가능하지만 그 객체를 통한 외부 접근은 불가능하며, private은 유도 클래스에서 직접 접근할 수 없다. 근거 대조 문구: 기본클래스에서접근가능 / 유도클래스에서접근가능 / 객체로부터는접근불가능 / Private은유도클래스에서접근이불가능	(L06 Protected Member, 슬라이드 23)
U04-008	유도 클래스 객체 생성 시 기본 클래스 생성자가 먼저 호출되고 유도 클래스 생성자가 뒤에 호출된다. 근거 대조 문구: 먼저기본클래스의생성자 / 그이후유도클래스의생성자	(L06 Constructors and Destructors, 슬라이드 27)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-009	소멸자는 유도 클래스, 기본 클래스 순서로 호출되어 생성자와 반대 순서이다. 근거 대조 문구: 먼저유도클래스의소멸자 / 그이후기본클래스의소멸자	(L06 Constructors and Destructors, 슬라이드 31)
U04-010	유도 클래스의 초기화 리스트에서 원하는 기본 클래스 생성자를 호출하며, 명시하지 않으면 기본 클래스의 인자 없는 생성자가 호출된다. 근거 대조 문구: 초기화리스트를활용 / 기본생성자(인자가없는생성자)가호출	(L06 Passing Arguments to Base Class Constructor, 슬라이드 38)
U04-011	사용자가 유도 클래스의 복사 생성자를 구현한 경우 기본 클래스의 복사 생성자를 명시적으로 호출해야 하며, 명시하지 않으면 기본 클래스의 인자 없는 생성자가 호출된다. 근거 대조 문구: 명시하지않으면기본클래스의인자를받지않는생성자 / 복사/이동생성자를명시적으로호출	(L06 Copy Constructor, 슬라이드 45)
U04-012	정적 바인딩은 컴파일 시 타입을 기준으로 호출 함수를 결정한다. 근거 대조 문구: 정적바인딩은컴파일시타입을기준으로호출함수를결정	(L07 Polymorphism, 슬라이드 13)
U04-013	동적 바인딩은 런타임에 실제 메모리에 저장된 타입을 기준으로 호출 함수를 결정한다. 근거 대조 문구: 런타임시실제메모리에저장된타입을기준으로호출함수를결정	(L07 Polymorphism, 슬라이드 13)
U04-014	다형성(polymorphism)은 하나의 메소드를 호출했음에도 여러 가지 다른 작업을 하는 것이다. 근거 대조 문구: 하나의메소드를호출했음에도불구하고여러가지다른작업들을하는것	(B 7장·가상 함수와 다형성, 인쇄 p.321)
U04-015	C++의 런타임 다형성 구현에는 상속, 기본 클래스 포인터 또는 참조자, 가상함수가 필요하다. 근거 대조 문구: 상속 / 기본클래스포인터또는참조자 / 가상함수	(L07 Polymorphism, 슬라이드 6)
U04-016	재정의로 가려진 기본 클래스의 멤버함수는 Base::Member() 형식으로 호출할 수 있다. 근거 대조 문구: Base::Member()와같은형식으로호출가능	(L06 Using Member Function of Base Class, 슬라이드 47)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-017	오버라이드할 기본 클래스의 함수 앞에 virtual을 붙이면 가상함수가 된다. 근거 대조 문구: 기본클래스의함수 / virtual로선언 / 가상함수	(L07 Virtual Function, 슬라이드 15)
U04-018	유도 클래스를 기본 클래스 포인터로 해제하는 강의 예제는 기본 소멸자를 가상으로 선언하면 유도·기본 소멸자가 모두 호출된다. 근거 대조 문구: Entity의소멸자를가상으로선언 / Player의소멸자와Entity의소멸자가모두호출	(L07 Virtual Destructor, 슬라이드 18)
U04-019	override 지정자는 오버라이딩 의도를 표시하고, 함수 원형이 맞지 않는 실수를 컴파일 단계에서 찾도록 한다. 근거 대조 문구: 오버라이딩시실수를방지 / 함수원형이일치하지않는다는컴파일에러	(L07 override Specifier, 슬라이드 25,28)
U04-020	클래스의 final은 추가 상속을 막고, 멤버함수의 final은 유도 클래스에서 가상함수를 오버라이드하지 못하게 한다. 근거 대조 문구: 클래스를더이상상속하지못하도록 / 가상함수를오버라이드하지못하도록	(L07 final Specifier, 슬라이드 29)
U04-021	추상 클래스(Abstract Class)는 객체를 생성할 수 없으며 상속 계층에서 기본 클래스로 사용되는 클래스이다. 근거 대조 문구: 객체를생성할수없는클래스 / 기본클래스로사용	(L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33)
U04-022	가상 멤버함수의 선언 뒤에 =0을 붙이면 순수가상함수가 되며, 이를 하나 이상 갖는 클래스는 추상 클래스이다. 근거 대조 문구: 하나이상의“순수가상함수” / 선언뒤에“=0”	(L07 Pure Virtual Functions and Abstract Classes, 슬라이드 34)
U04-023	순수가상함수를 오버라이드하지 않는 유도 클래스도 추상 클래스로 간주된다. 근거 대조 문구: 오버라이드하지않는경우, 유도클래스도추상클래스	(L07 Pure Virtual Functions and Abstract Classes, 슬라이드 35)
U04-024	추상 클래스 자체의 객체를 만들 수 없어도 그 포인터·참조자로 오버라이드된 함수를 동적 바인딩할 수 있다. 근거 대조 문구: 추상(기본) 클래스의포인터/참조자를사용해 / 동적바인딩	(L07 Pure Virtual Functions and Abstract Classes, 슬라이드 38)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-025	강의는 순수가상함수만 가진 추상 클래스를 인터페이스 클래스라고 부른다. 근거 대조 문구: 순수가상함수"만"을가진추상클래스를인터페이스클래스	(L07 Abstract Classes and Interface, 슬라이드 40)
U04-026	가상함수 테이블(vtable)은 함수 이름과 실제로 대응하는 함수를 테이블로 저장한다. 근거 대조 문구: 함수의이름 / 실제로어떤함수 / 테이블로저장	(B 7장·상속에 관련된 잡다한 내용들·가상 함수의 구현 원리, 인쇄 p.327)
U04-027	강의의 구현 설명에서는 객체가 호출 정보를 가진 vtable을 가리키는 포인터(vptr)를 포함한다. 근거 대조 문구: 가상함수테이블(vtable)을가리키는포인터(vptr)를객체에포함	(L07 vtable, 슬라이드 44)
U04-028	책은 작동이 보장되지 않는 강제 다운캐스팅을 권장하지 않으며, dynamic_cast를 상속 관계의 포인터 사이 캐스팅 수단으로 소개한다. 근거 대조 문구: 다운캐스팅은작동이보장되지않는한 / dynamic_cast / 상속관계에있는두포인터들간	(B 7장·가상 함수와 다형성·dyanmic_cast, 인쇄 p.307)
U04-029	다중 상속(multiple inheritance)은 한 클래스가 여러 클래스를 상속받는 것이다. 근거 대조 문구: 한클래스가다른여러개의클래스들을상속 / 다중상속	(B 7장·상속에 관련된 잡다한 내용들·다중 상속, 인쇄 p.332)
U04-030	다중 상속에서 기반 클래스 생성자의 호출 순서는 상속하는 순서에 따른다. 근거 대조 문구: 오직상속하는순서에만좌우	(B 7장·상속에 관련된 잡다한 내용들·다중 상속, 인쇄 p.334)
U04-031	두 기반 클래스에 같은 이름의 멤버가 있으면 파생 객체에서 그 이름으로 접근할 때 모호성 오류가 발생할 수 있다. 근거 대조 문구: B 의a 인지, A 의a 인지구분할수없다는오류	(B 7장·상속에 관련된 잡다한 내용들·다중 상속 시 주의할 점, 인쇄 p.335)
U04-032	다이아몬드 상속은 상속되는 두 클래스가 공통 기반 클래스를 포함하여 그 내용이 중복되는 형태이다. 근거 대조 문구: 공통의베이스클래스를포함 / Human 의모든내용이중복	(B 7장·상속에 관련된 잡다한 내용들·다중 상속 시 주의할 점, 인쇄 p.336)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-033	공통 기반 Human을 virtual로 상속하면 Me에 Human을 한 번만 포함하도록 할 수 있으며 Me 생성자에서 Human 생성자도 호출해야 한다. 근거 대조 문구: Human 을virtual 로상속 / Human 을한번만포함 / Human 의생성자또한호출	(B 7장·상속에 관련된 잡다한 내용들·가상 상속, 인쇄 p.337)
U04-034	사원 관리 예제는 Employee와 Manager를 Employee*로 관리하고 calculate_pay와 print_info를 가상함수로 만들어 실제 종류에 맞게 호출한다. 근거 대조 문구: calculate_pay 함수와print_info 함수앞에virtual / Employee* / Manager 면Manager 의함수	(B 7장·가상 함수와 다형성·Employee 문제, 인쇄 p.318-319,321)
U04-035	차량 예제의 브리지 패턴은 하나의 범주를 Engine* 같은 멤버 포인터로 두고, 중첩된 일반화는 한 계층 아래 다른 계층의 파생 클래스를 둔다. 근거 대조 문구: 한가지카테고리를아예멤버포인터 / 한가지계층을먼저골라서 / 다른계층에해당하는파생클래스	(B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.338)
U04-036	같은 차량 예제의 다중 상속 방식은 환경과 동력원에 해당하는 클래스를 함께 상속받는다. 근거 대조 문구: 동력원과환경에해당하는클래스를상속 / GasPoweredEngine 과LandVehicle	(B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.338-339)
U04-037	책은 브리지 패턴·중첩된 일반화·다중 상속 중 절대적으로 우월한 방식은 없으며 상황에 맞게 고른다고 정리한다. 근거 대조 문구: 절대적으로우월한방식은없 / 상황에맞게최선의방식을	(B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.339)
U04-038	생성 시 기본 클래스 생성자를 먼저 호출한다. 근거 대조 문구: 먼저기본클래스의생성자가호출	(L06 Constructors and Destructors, 슬라이드 27)
U04-039	이후 유도 클래스 생성자를 호출한다. 근거 대조 문구: 그이후유도클래스의생성자	(L06 Constructors and Destructors, 슬라이드 27)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-040	소멸 시에는 유도 클래스 소멸자를 먼저 호출한다. 근거 대조 문구: 먼저유도클래스의소멸자가호출	(L06 Constructors and Destructors, 슬라이드 31)
U04-041	이후 기본 클래스 소멸자를 호출한다. 근거 대조 문구: 그이후기본클래스의소멸자	(L06 Constructors and Destructors, 슬라이드 31)
U04-042	포인터의 기반 클래스에서 함수의 정의를 찾는다. 근거 대조 문구: func1() 의정의를Parent 클래스에서찾아	(B 7장·가상 함수의 구현 원리, 인쇄 p.328)
U04-043	가상함수이면 직접 실행하는 대신 가상함수 테이블에서 대응 함수를 선택한다. 근거 대조 문구: 직접실행하는게아니라, 가상함수테이블에서	(B 7장·가상 함수의 구현 원리, 인쇄 p.328)
U04-044	실행 시 실제 객체의 가상함수 테이블을 참조하여 오버라이드된 함수를 호출한다. 근거 대조 문구: Child 객체의가상함수테이블 / Child::func1() 을호출	(B 7장·가상 함수의 구현 원리, 인쇄 p.328-329)

U04-045	순수가상함수와 가상 소멸자 - 원문 발췌 (원문 코드 발췌)
	class Animal {
	public:
	Animal() {}
	virtual ~Animal() {}
	virtual void speak() = 0;
	};
	근거 대조 문구: virtual ~Animal() {} / virtual void speak() = 0;
		(B 7장·순수 가상 함수와 추상 클래스, 인쇄 p.329)

U04-046	순수가상함수의 오버라이딩 - 원문 발췌 (원문 코드 발췌)
	class Dog : public Animal {
	public:
	Dog() : Animal() {}
	void speak() override { std::cout << "왈왈" << std::endl; }
	};
	근거 대조 문구: class Dog : public Animal / void speak() override
		(B 7장·순수 가상 함수와 추상 클래스, 인쇄 p.329)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-047	차량 예제: 브리지 패턴의 클래스 수: N + M; N: 차량이 작동하는 환경의 수, M: 동력원의 종류. 근거 대조 문구: 총N + M 개의클래스만생성	(B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.337-338)
U04-048	차량 예제: 중첩된 일반화·다중 상속의 조합: N × M; 각 환경과 동력원의 모든 조합에 해당하는 파생 클래스의 최대 수. 근거 대조 문구: 최대N × M 가지의파생클래스 / 파생클래스들을최대N × M 개	(B 7장·다중 상속은 언제 사용해야 할까?, 인쇄 p.338)
U04-049	정적 바인딩 근거 대조 문구: 정적바인딩	(L07 Polymorphism, 슬라이드 13)
U04-050	컴파일 시 근거 대조 문구: 컴파일시	(L07 Polymorphism, 슬라이드 13)
U04-051	타입을 기준으로 호출 함수를 결정한다. 근거 대조 문구: 타입을기준으로호출함수를결정	(L07 Polymorphism, 슬라이드 13)
U04-052	동적 바인딩 근거 대조 문구: 동적바인딩	(L07 Polymorphism, 슬라이드 13)
U04-053	런타임 근거 대조 문구: 런타임시	(L07 Polymorphism, 슬라이드 13)
U04-054	실제 메모리에 저장된 타입을 기준으로 결정한다. 근거 대조 문구: 실제메모리에저장된타입을기준	(L07 Polymorphism, 슬라이드 13)
U04-055	상속은 기본 클래스를 기반으로 새 클래스를 만들고, 유도 클래스의 초기화 리스트로 기본 클래스를 어떻게 생성할지 명시하는 기능이다. 근거 대조 문구: 기본클래스를기반으로새클래스 / 초기화리스트를사용해명시	(L06 (Summary) Inheritance, 슬라이드 53)
U04-056	다중 상속은 이름 충돌과 기반 클래스 중복에 주의하며, 설계 방식은 상황에 맞게 선택한다. 근거 대조 문구: 구분할수없다는오류 / 모든내용이중복 / 상황에맞게최선의방식	(B 7장·상속에 관련된 잡다한 내용들, 인쇄 p.335-336,339)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U04-057	생성은 기본-유도, 소멸은 유도-기본 순서이다. 근거 대조 문구: 기본생성-유도생성/ 유도소멸-기본소멸순서	(L06 (Summary) Inheritance, 슬라이드 53)
U04-058	정적 바인딩은 컴파일 시, 동적 바인딩은 런타임에 결정한다. 근거 대조 문구: 정적바인딩은컴파일시 / 동적바인딩 / 런타임시	(L07 Polymorphism, 슬라이드 13)
U04-059	그 유도 클래스도 추상 클래스로 간주되므로 객체를 생성할 수 없다. 근거 대조 문구: 객체를생성할수없는클래스 / 유도클래스도추상클래스	(L07 Pure Virtual Functions and Abstract Classes, 슬라이드 33,35)
U04-060	HandsomeHuman과 SmartHuman이 Human을 virtual로 상속하도록 하여 Me에 Human을 한 번만 포함시킨다. 근거 대조 문구: public virtual Human / Human 을한번만포함	(B 7장·상속에 관련된 잡다한 내용들·가상 상속, 인쇄 p.337)

05 연산자 오버로딩·N차원 배열 프로젝트

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-001	멤버함수·전역함수·스트림·대입·첨자 연산자의 오버로딩을 다룬다. 근거 대조 문구: 멤버함수 / 전역함수 / 스트림삽입및추출 / 대입 / 첨자	(L08 Operator Overloading, 슬라이드 4)
U05-002	N차원 배열 프로젝트는 연산자 오버로딩을 종합하여 []로 원소에 접근하는 배열 클래스를 구현한다. 근거 대조 문구: N 차원배열을제공하는클래스 / [] 를이용해서원소에접근	(B 6장·N 차원 배열 만들기 프로젝트, 인쇄 p.239-240)
U05-003	연산자 오버로딩은 +, =, * 등의 연산자를 사용자 정의 타입에 대해 사용할 수 있게 하는 기능이다. 근거 대조 문구: 연산자를유저정의타입(ex. 클래스)에대해사용할수있도록	(L08 Operator Overloading, 슬라이드 6)
U05-004	연산자 오버로딩은 연산 우선순위나 단항·이항·삼항 구분을 바꾸지 못하며, 기본 타입의 연산자나 새 연산자를 정의할 수 없다. 근거 대조 문구: 우선순위를바꿀수는없음 / 교체는불가능 / 기본타입 / 새로운연산자의정의불가	(L08 Operator Overloading, 슬라이드 11)
U05-005	[ ], ( ), ->, = 등은 멤버함수로만 오버로딩할 수 있다. 근거 대조 문구: [ ], ( ), ->, = / 멤버함수로만오버로딩가능	(L08 Operator Overloading, 슬라이드 11)
U05-006	operator+와 operator=를 정의해도 operator+=는 자동 생성되지 않으므로 별도로 구현해야 한다. 근거 대조 문구: 자동으로a = a + b; 로바꾸어주지않습니다 / operator+= 를따로	(B 6장·연산자 오버로딩, 인쇄 p.201)
U05-007	멤버함수 방식의 이항 연산자는 왼쪽 객체의 operator 함수에 오른쪽 객체를 인자로 넘긴다. 근거 대조 문구: 왼쪽에있는객체(p1)의멤버함수 / 오른쪽객체(p2)를인자로	(L08 Binary operator, as Member Function, 슬라이드 14)
U05-008	전역함수 방식의 이항 연산자는 왼쪽 피연산자도 매개변수로 전달한다. 근거 대조 문구: Lhs도매개변수로써전달	(L08 Binary operator, as Global Function, 슬라이드 22)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-009	멤버함수만으로는 p1*3이 가능해도 3*p1을 처리할 수 없는 경우가 있어 전역함수를 추가 구현한다. 근거 대조 문구: p1*3은함수호출가능, 3*p1은함수호출불가 / 전역함수를추가구현	(L08 Limitation of Member Function·Binary operator, as Global Function, 슬라이드 20,27)
U05-010	friend로 선언한 전역 연산자 함수는 클래스의 private 멤버에 접근할 수 있다. 근거 대조 문구: xpos와ypos에쉽게접근하기위해friend선언	(L08 Binary operator, as Global Function, 슬라이드 23)
U05-011	클래스 내부에 friend 함수의 코드가 있어도 그 함수가 멤버함수인 것은 아니다. 근거 대조 문구: 무조건멤버함수가아닙니다 / operator+는전역함수	(L08 Binary operator, as Global Function, 슬라이드 24-25)
U05-012	MyString의 operator==는 compare의 결과를 논리 부정하여 같을 때 true를 반환하도록 한다. 근거 대조 문구: return !compare(str); / 같으면true 를리턴	(B 6장·MyString 의 == 연산자 오버로딩, 인쇄 p.189)
U05-013	책의 Complex 사칙연산 함수는 원래 객체를 바꾸지 않고 계산 결과의 값을 반환한다. 근거 대조 문구: Complex& 가아닌Complex 를리턴 / real += c.real / 이러한문제를막기	(B 6장·복소수 클래스 만들기, 인쇄 p.196)
U05-014	자동 생성된 대입 연산자는 얕은 복사를 수행하므로 깊은 복사가 필요한 경우 직접 정의해야 한다. 근거 대조 문구: 자동생성된대입연산자는얕은복사 / 깊은복사를통한대입연산자직접정의	(L08 Assignment Operator Overloading, 슬라이드 37)
U05-015	대입 연산자는 자기 대입을 검사하고, 포인터 멤버의 데이터가 배열이면 해제 후 재할당을 고려한다. 근거 대조 문구: Self-assignmentchecking이필요 / 해제(delete [])후재할당	(L08 Assignment Operator Overloading, 슬라이드 39)
U05-016	유도 클래스의 대입 연산자는 기본 클래스의 대입 연산자를 호출한 다음 유도 클래스 속성을 대입한다. 근거 대조 문구: 기본클래스에대한대입연산자를호출하고난뒤 / 유도클래스의속성	(L08 Copy/Move Constructor & Overloaded Operator =, 슬라이드 40)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-017	<<는 cout 객체(ostream 클래스)의 오버로딩된 연산자이다. 근거 대조 문구: <<은cout 객체(ostream 클래스)의오버로딩된연산자	(L08 OStream, 슬라이드 30)
U05-018	스트림 삽입 연산자는 전역함수로 선언하고 ostream의 참조자를 반환하여 chain insertion을 가능하게 한다. 근거 대조 문구: ostream의참조자를반환 / chain insertion / 전역함수로선언	(L08 Stream Insertion and Extraction Overloading, 슬라이드 31)
U05-019	스트림 추출 연산자의 오른쪽 매개변수는 const가 아니다. 근거 대조 문구: 우측매개변수(rhs)는const가아님	(L08 Stream Insertion and Extraction Operators Overloading, 슬라이드 32)
U05-020	첨자 연산자는 멤버함수로 오버로딩하여 경계 검사 등의 기능을 확장할 수 있다. 근거 대조 문구: 멤버함수로오버로딩 / 경계검사등기능확장	(L08 Subscript([ ]) operator overloading, 슬라이드 43)
U05-021	첨자 연산자로 배열 내부 값을 수정하려면 참조형을 반환하고, const 객체를 위한 접근도 제공한다. 근거 대조 문구: 참조형으로반환해야만 / const PointArr 사용에있어서데이터에접근	(L08 Subscript([ ]) operator overloading, 슬라이드 45-46)
U05-022	Wrapper 클래스는 기본 자료형을 클래스로 포장하여 각 자료형을 객체로 사용하는 클래스이다. 근거 대조 문구: 기본자료형들(int, float 등등) 을클래스로포장	(B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.233)
U05-023	타입 변환 연산자는 operator 뒤에 변환할 타입을 쓰며 생성자처럼 반환 타입을 따로 쓰지 않는다. 근거 대조 문구: operator (변환하고자하는타입) () / 함수의리턴타입을써주시면안됩니다	(B 6장·int Wrapper 클래스 - 타입 변환 연산자, 인쇄 p.234)
U05-024	전위 증감은 매개변수 없는 operator++()/operator--()로, 후위 증감은 의미 없는 int 매개변수를 가진 형태로 구분한다. 근거 대조 문구: operator++(); / operator--(); / 전위와후위를구별하기위해int 인자	(B 6장·전위/후위 증감 연산자, 인쇄 p.236)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-025	전위 증감 연산자는 값을 바꾼 자기 자신을 반환한다. 근거 대조 문구: 전위증감연산의경우값이바뀐자기자신을리턴	(B 6장·전위/후위 증감 연산자, 인쇄 p.236)
U05-026	후위 증감 연산자는 값이 바뀌기 이전의 객체를 반환한다. 근거 대조 문구: 후위증감의경우값이바뀌기이전의객체를리턴	(B 6장·전위/후위 증감 연산자, 인쇄 p.236)
U05-027	static_cast는 언어가 지원하는 일반적인 타입 변환이고, const_cast는 객체의 상수성(const)을 없애는 타입 변환이다. 근거 대조 문구: static_cast : / 일반적인타입변환 / const_cast : / 상수성(const) 를없애는	(B 6장·C++ 스타일의 캐스팅, 인쇄 p.241)
U05-028	책은 dynamic_cast를 파생 클래스 사이의 다운캐스팅으로 소개하고, reinterpret_cast는 서로 관련 없는 포인터 사이의 캐스팅 등에 사용한다고 설명한다. 근거 대조 문구: dynamic_cast : 파생클래스사이에서의다운캐스팅 / reinterpret_cast / 서로관련이없는포인터	(B 6장·C++ 스타일의 캐스팅, 인쇄 p.241)
U05-029	프로젝트의 Array는 차원 수 dim, 각 차원의 크기 size, 시작 Address 포인터 top을 보관한다. 근거 대조 문구: 몇차원 / dim / size 배열 / Address * / top	(B 6장·N 차원 배열 만들기, 인쇄 p.246)
U05-030	Address는 level과 void* next를 가지며, 마지막 레벨에서는 int 배열을, 그 앞 레벨에서는 다음 Address 배열을 가리킨다. 근거 대조 문구: int level; / void* next; / int / Address 배열	(B 6장·N 차원 배열 만들기, 인쇄 p.243-244)
U05-031	initialize_address는 마지막 레벨이면 int 배열을 만들고, 그 전이면 다음 Address 배열을 만들어 각 원소를 재귀적으로 초기화한다. 근거 대조 문구: 현재처리하고있는Address 배열의레벨이(dim - 1) / 다음레벨 / initialize_address	(B 6장·N 차원 배열 만들기, 인쇄 p.247-248)
U05-032	생성은 위에서 아래로 메모리를 확장하고, 소멸은 아래에서 위로 메모리를 해제한다. 근거 대조 문구: 생성자는'위에서아래로' / 소멸자는'아래에서위로'	(B 6장·N 차원 배열 만들기, 인쇄 p.249)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-033	연속된 []의 중간 단계는 Int Wrapper 객체로 표현하고, 실제 데이터에 도달하면 int처럼 읽거나 대입하도록 한다. 근거 대조 문구: Wrapper 클래스객체를리턴 / 중간단계의정보 / 타입변환연산자	(B 6장·N 차원 배열 만들기·operator[] 문제, 인쇄 p.251-252,255)
U05-034	반복자(iterator)는 배열의 모든 원소를 순차적으로 접근할 수 있게 하며, ++로 다음 원소를 가리킨다. 근거 대조 문구: 모든원소들을순차적으로접근 / itr ++ 을하면그다음원소	(B 6장·N 차원 배열 만들기 프로젝트·반복자, 인쇄 p.260)
U05-035	Iterator는 location 배열로 현재 원소의 좌표를 보관한다. 근거 대조 문구: 현재Iterator 가어떤원소를가리키고있는지 / int * location 에배열로보관	(B 6장·N 차원 배열 만들기·반복자, 인쇄 p.260)
U05-036	begin은 첫 원소, end는 마지막 원소 바로 다음을 뜻하므로 end의 값을 참조하려 하면 오류가 발생한다. 근거 대조 문구: 시작(begin)은맨첫번째원소 / 마지막(end)은맨마지막원소 바로다음 / 마지막의값을참조하려고하면오류	(B 6장·N 차원 배열 만들기·반복자, 인쇄 p.261)
U05-037	증가 전 자기 자신으로 임시 객체를 복사 생성한다. 근거 대조 문구: Test temp(*this);	(B 6장·전위/후위 증감 연산자, 인쇄 p.238)
U05-038	현재 객체의 값을 증가시킨다. 근거 대조 문구: x++;	(B 6장·전위/후위 증감 연산자, 인쇄 p.238)
U05-039	증가 전 상태를 보관한 임시 객체를 반환한다. 근거 대조 문구: return temp;	(B 6장·전위/후위 증감 연산자, 인쇄 p.238)
U05-040	현재 Address가 마지막 레벨이면 해당 크기의 int 배열을 할당하고 재귀를 끝낸다. 근거 대조 문구: current->level == dim - 1 / new int[size[current->level]] / return;	(B 6장·N 차원 배열 만들기, 인쇄 p.247)
U05-041	마지막 레벨이 아니면 다음 Address 배열을 할당한다. 근거 대조 문구: current->next = new Address[size[current->level]];	(B 6장·N 차원 배열 만들기, 인쇄 p.247)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-042	새 각 원소의 level을 다음 단계로 정하고 initialize_address를 재귀 호출한다. 근거 대조 문구: current->level + 1 / initialize_address	(B 6장·N 차원 배열 만들기, 인쇄 p.248)

U05-043	대입 연산자 - 원문 발췌 (원문 코드 발췌)
	Complex& Complex::operator=(const Complex& c)
	
	{
	real = c.real;
	img = c.img;
	return *this;
	}
	근거 대조 문구: Complex& Complex::operator=(const Complex& c) / return *this;
		(B 6장·대입 연산자 함수, 인쇄 p.197)

U05-044	후위 증가 연산자 - 원문 발췌 (원문 코드 발췌)
	Test operator++(int) {
	Test temp(*this);
	x++;
	std::cout << "후위 증감 연산자" << std::endl;
	return temp;
	}
	근거 대조 문구: Test operator++(int) / Test temp(*this); / return temp;
		(B 6장·전위/후위 증감 연산자, 인쇄 p.238)

U05-045	N차원 배열을 하나의 1차원 배열로 할당하는 크기: x1 * x2 * … * xn; 원문에서 사용자가 원하는 배열을 arr[x1][x2]…[xn]으로 두었을 때의 크기. 근거 대조 문구: x1 * x2 * … * xn 크기의일차원배열을할당	(B 6장·N 차원 배열 만들기, 인쇄 p.242)
U05-046	멤버함수 근거 대조 문구: 멤버함수	(L08 Binary operator, as Member Function, 슬라이드 14)
U05-047	전역함수 근거 대조 문구: 전역함수	(L08 Binary operator, as Global Function, 슬라이드 22)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U05-048	책은 동등한 두 객체의 이항 연산자는 외부함수로, 한 객체만 바뀌는 이항 연산자와 단항 연산자는 멤버함수로 구현하는 것을 권한다. 근거 대조 문구: 동등한객체 / 외부함수 / 한객체만값이바뀐다 / 단항연산자는멤버함수	(B 6장·잡다한 연산자들의 오버로딩·정리, 인쇄 p.238-239)
U05-049	스트림 연산자는 연속 삽입을 위해 참조자를 반환하며, 깊은 복사가 필요한 대입 연산자는 직접 오버로딩한다. 근거 대조 문구: Chain insertion을위해참조자반환 / 깊은복사가필요한경우 / 직접오버로딩	(L08 (Summary)Operator Overloading, 슬라이드 48)
U05-050	첨자 연산자는 일반적으로 const 멤버와 참조 반환 함수를 각각 오버로딩한다. 근거 대조 문구: const 멤버와참조반환함수두개를오버로딩	(L08 (Summary)Operator Overloading, 슬라이드 48)
U05-051	연산 우선순위를 바꿀 수 없고 새로운 연산자를 정의할 수도 없다. 근거 대조 문구: 우선순위를바꿀수는없음 / 새로운연산자의정의불가	(L08 Operator Overloading, 슬라이드 11)
U05-052	전위는 operator++(), 후위는 의미 없는 int 매개변수를 둔 operator++(int) 형태로 구분한다. 근거 대조 문구: operator++(); / operator++(int); / 전위와후위를구별	(B 6장·전위/후위 증감 연산자, 인쇄 p.236)
U05-053	cout << p1 << p2 같은 chain insertion을 가능하게 하기 위해서이다. 근거 대조 문구: 참조자를반환하지않으면cout << p1 << p2 와같은연산불가	(L08 Stream Insertion and Extraction Overloading, 슬라이드 31)
U05-054	반복자는 모든 원소를 순차 접근하기 위해 추가하며, end는 마지막 원소 바로 다음을 뜻한다. 근거 대조 문구: 모든원소들을순차적으로접근 / 맨마지막원소바로다음	(B 6장·N 차원 배열 만들기·반복자, 인쇄 p.260-261)
U05-055	전위: operator++() 근거 대조 문구: operator++();	(B 6장·전위/후위 증감 연산자, 인쇄 p.236)
U05-056	후위: operator++(int) 근거 대조 문구: operator++(int);	(B 6장·전위/후위 증감 연산자, 인쇄 p.236)

06 표준 입출력·파일·엑셀 프로젝트

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-001	C++ 입출력 라이브러리의 클래스와 표준 스트림·파일 스트림을 다룬다. 근거 대조 문구: 파일스트림과의입출력을다루어	(B C++ 표준 입출력 라이브러리 / C++ 파일 입출력, 인쇄 p.340,356)
U06-002	배운 내용을 바탕으로 콘솔용 Excel을 만들고, 셀과 표를 확장한다. 근거 대조 문구: 콘솔로Excel 을만드는것	(B 엑셀 만들기 프로젝트, 인쇄 p.374,394)
U06-003	ios_base는 스트림의 입출력 형식 관련 데이터를 처리하고, ios는 스트림 버퍼를 초기화하며 입출력 작업의 상태를 처리한다. 근거 대조 문구: 스트림의입출력형식관련데이터를처리합니다.	(B C++ 입출력 라이브러리, 인쇄 p.341)
U06-004	스트림 버퍼는 데이터를 내보내거나 받아들이기 전에 임시로 저장하는 곳이다. 근거 대조 문구: 데이터를내보내거나받아들이기전에임시로저장하는곳	(B C++ 입출력 라이브러리, 인쇄 p.341)
U06-005	istream은 실제로 입력을 수행하며, operator>>가 정의되어 있고 cin은 istream 클래스의 객체 중 하나이다. 근거 대조 문구: istream 은실제로입력을수행하는클래스	(B istream 클래스, 인쇄 p.342)
U06-006	operator>>로 문자열을 입력하면 공백문자에 따라 분리해서 입력받는다. 근거 대조 문구: 공백문자에따라서각각을분리해서입력	(B istream 클래스, 인쇄 p.343-344)
U06-007	타입에 맞지 않는 값을 넣어 오류가 발생하면 failbit가 켜지고, 입력값을 받지 않고 리턴한다. 근거 대조 문구: 타입에맞지않는값을넣어서오류가발생하는경우에는failbit	(B istream 클래스, 인쇄 p.346)
U06-008	fail 함수는 failbit가 true이거나 badbit가 true이면 true를 리턴한다. 근거 대조 문구: failbit 가true 거나badbit 가true 면true	(B istream 클래스, 인쇄 p.348)
U06-009	clear()에 인자를 주지 않으면 플래그를 goodbit으로 초기화하고, ignore는 최대 첫 번째 인자만큼, 두 번째 인자가 나올 때까지 그 문자를 포함하여 버퍼에서 무시한다. 근거 대조 문구: 플래그를goodbit 으로초기화	(B istream 클래스, 인쇄 p.348)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-010	인자 두 개를 취하는 setf는 두 번째 인자의 내용을 초기화하고 첫 번째 인자의 형식 플래그를 적용한다. 근거 대조 문구: 두번째인자(위에서basefield) 의내용을초기화	(B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.350)
U06-011	조작자는 스트림을 조작하여 입력 혹은 출력 방식을 바꾸는 함수이다. 근거 대조 문구: 입력혹은출력방식을바꿔주는함수를조작자	(B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.351)
U06-012	std::endl은 개행문자를 출력하고 버퍼를 모두 내보내는 flush를 수행한다. 근거 대조 문구: 개행문자를출력하는것말고도, 버퍼를모두내보내는	(B 형식 플래그(format flag) 와 조작자(Manipulator), 인쇄 p.352)
U06-013	streambuf는 입력 버퍼와 출력 버퍼를 각각 get area와 put area라고 부르며, 버퍼의 시작·다음으로 읽을 문자·끝을 가리키는 포인터를 둔다. 근거 대조 문구: 각각get area 와put area	(B 스트림 버퍼에 대해, 인쇄 p.353)
U06-014	rdbuf는 입력을 수행하던 streambuf 객체의 포인터를 반환하고, snextc는 위치지정자를 한 칸 전진시킨 후 해당 문자를 엿본다. 근거 대조 문구: 스트림위치지정자를한칸전진시킨후	(B 스트림 버퍼에 대해, 인쇄 p.354)
U06-015	ifstream과 ofstream은 각각 istream과 ostream을 상속받으며 fstream 라이브러리에 포함된다. 근거 대조 문구: 각각을상속받은것이ifstream 과ofstream	(B C++ 파일 입출력 / fstream, 인쇄 p.356)
U06-016	ifstream 생성자에 파일 경로를 전달해 파일을 열고, is_open으로 파일이 열렸는지 확인한다. 근거 대조 문구: 파일이열렸는지의유무를리턴	(B C++ 파일 입출력 / fstream, 인쇄 p.357-358)
U06-017	ifstream 소멸자는 자동으로 close를 수행하며, 같은 객체를 다른 파일에 연결할 때는 close 후 open을 사용한다. 근거 대조 문구: 기존파일스트림과의연결을종료하고, 새로운파일과연결	(B C++ 파일 입출력 / fstream, 인쇄 p.358-359)
U06-018	binary 옵션은 이진 그대로의 값을 받는 설정이며, read의 두 번째 인자로 읽을 바이트 수를 전달한다. 근거 대조 문구: 두번째인자로반드시몇바이트를읽을지	(B C++ 파일 입출력 / fstream, 인쇄 p.360-361)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-019	seekg는 파일 위치지정자를 옮기며, tellg는 시작지점으로부터 위치지정자의 위치를 반환한다. 근거 대조 문구: tellg 함수는위치지정자의위치	(B 파일 전체 읽기, 인쇄 p.362)
U06-020	ifstream의 getline은 개행문자나 지정한 문자가 나올 때까지 최대 지정한 크기-1만큼 읽고, 버퍼가 먼저 차면 failbit를 켠다. 근거 대조 문구: 최대지정한크기- 1 만큼읽게	(B 파일 전체를 한 줄씩 읽기, 인쇄 p.363-364)
U06-021	getline(in, s)은 istream 객체와 string 객체를 받아 버퍼 크기를 지정하지 않고 개행문자 혹은 파일 끝까지 입력받는다. 근거 대조 문구: 굳이버퍼의크기를지정하지않아도	(B 파일 전체를 한 줄씩 읽기, 인쇄 p.365)
U06-022	ofstream은 파일이 없으면 생성하고, 특별한 설정 없이 기존 파일을 열면 내용을 지우고 새 내용으로 덮어쓴다. 근거 대조 문구: 해당파일내용이다지워지고새로운내용으로덮어	(B 파일에 쓰기, 인쇄 p.366)
U06-023	ios::app은 기존 파일의 뒤에 새 내용을 붙여 쓰고, ios::ate는 파일을 열 때 위치지정자를 파일 끝에 둔다. 근거 대조 문구: 파일을열때위치지정자가파일끝	(B 파일에 쓰기, 인쇄 p.367)
U06-024	std::istringstream은 문자열을 입력 스트림처럼 사용하고, std::ostringstream에 출력한 값은 str 함수로 불러온다. 근거 대조 문구: str 함수로현재문자열스트림에쓰여있는값	(B 문자열 스트림(std::stringstream), 인쇄 p.370-371)
U06-025	프로젝트의 Vector는 가변 길이 배열로 임의 위치의 원소에 접근하며, Stack은 항상 최상단의 데이터에만 접근한다. 근거 대조 문구: 항상최상단에있는데이터에만접근	(B 엑셀 만들기 프로젝트, 인쇄 p.374)
U06-026	Cell은 한 칸을 뜻하고, Table은 필요할 때 동적으로 생성한 Cell 객체의 포인터를 2차원 배열로 보관한다. 근거 대조 문구: 객체에대한포인터를2차원배열	(B 엑셀 만들기 프로젝트, 인쇄 p.383-385)
U06-027	Table의 print_table은 순수 가상함수이며, TxtTable·CSVTable·HtmlTable은 각 형식의 문자열로 표를 출력한다. 근거 대조 문구: virtual string print_table() = 0;	(B 엑셀 만들기 프로젝트, 인쇄 p.387,390-391)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-028	확장한 Cell은 stringify와 to_numeric을 순수 가상함수로 두며, StringCell·NumberCell·DateCell은 문자열·정수·시간 정보를 보관한다. 근거 대조 문구: 문자열, 정수, 시간정보를보관	(B 엑셀 만들기 프로젝트 2부 / Cell 클래스 확장, 인쇄 p.394-395)
U06-029	ExprCell 예제는 숫자를 한 자리 정수로, 셀 이름을 두 글자로 제한한다. 근거 대조 문구: 숫자는1 자리정수이고, 셀이름역시2자로제한	(B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.402)
U06-030	중위 표기법은 피연산자 사이에 연산자를 넣으며, 후위 표기법은 연산자를 피연산자 뒤에 두고 괄호 없이 표현한다. 근거 대조 문구: 연산자들이피연산자뒤쪽에위치	(B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)
U06-031	Excel 클래스는 사용자 입력을 받아 테이블을 생성·관리하며, sets·setn·setd·sete·out·exit 명령을 처리한다. 근거 대조 문구: 사용자의입력을받아서실제테이블을생성하고이를관리	(B 엑셀 만들기 프로젝트 2부 / 엑셀 프로그램, 인쇄 p.405-407)
U06-032	seekg로 위치지정자를 파일 끝으로 옮기고 tellg로 크기를 읽는다. 근거 대조 문구: 위치지정자를파일끝으로옮긴다.	(B 파일 전체를 한 번에 읽기, 인쇄 p.361-362)
U06-033	문자열을 그 크기로 resize한 뒤 위치지정자를 파일 맨 앞으로 옮긴다. 근거 대조 문구: 그크기의문자열을할당한다.	(B 파일 전체를 한 번에 읽기, 인쇄 p.361-362)
U06-034	read로 파일 전체 내용을 문자열에 저장한다. 근거 대조 문구: 파일전체에내용을문자열에저장	(B 파일 전체를 한 번에 읽기, 인쇄 p.363)
U06-035	피연산자를 만나면 스택에 push한다. 근거 대조 문구: 피연산자를만나면스택에push	(B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)
U06-036	연산자를 만나면 두 개를 pop하여 연산하고 결과를 다시 push한다. 근거 대조 문구: 스택에서두개를pop 한뒤	(B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)
U06-037	모든 계산이 끝나면 스택의 최종 결과값을 pop하여 리턴한다. 근거 대조 문구: 최종결과값을pop	(B 엑셀 만들기 프로젝트 2부 / to_numeric, 인쇄 p.401)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-038	피연산자는 exp_vec에 넣고, 여는 괄호는 스택에 push한다. 근거 대조 문구: 피연산자(셀이름이나숫자) 일경우그냥exp_vec	(B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.401)
U06-039	닫는 괄호를 만나면 여는 괄호가 나올 때까지 pop되는 연산자를 exp_vec에 넣는다. 근거 대조 문구: 여는괄호가pop 될때까지	(B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.401)
U06-040	연산자는 자신보다 우선순위가 낮은 연산자가 최상단에 오거나 스택이 빌 때까지 pop하여 exp_vec에 넣고, 자신을 push한다. 근거 대조 문구: 자기보다우선순위가낮은연산자	(B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.401)
U06-041	원문 구현은 수식 전체를 괄호로 감싸 마지막 괄호를 처리할 때 남은 연산자를 pop한다. 근거 대조 문구: 수식전체를( ) 로한번감싸는것	(B 엑셀 만들기 프로젝트 2부 / parse_expression 함수, 인쇄 p.404)

U06-042	파일 전체 읽기: 원문 발췌 (원문 코드 발췌)
	in.seekg(0, std::ios::end);
	int size = in.tellg();
	s.resize(size);
	in.seekg(0, std::ios::beg);
	in.read(&s[0], size);
	근거 대조 문구: in.seekg(0, std::ios::end);
	int size = in.tellg();
	s.resize(size);
	in.seekg(0, std::ios::beg);
	in.read(&s[0], size);
		(B 파일 전체를 한 번에 읽기, 인쇄 p.361)

U06-043	원문의 중위·후위 표기 예: 3 + 4 * 5 + 4 * (7 - 2) → 3 4 5 * + 4 7 2 - * +; 원문은 두 표기가 정확히 동일한 수식이라고 설명한다. 근거 대조 문구: 3 + 4 * 5 + 4 * (7 - 2)	(B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)
U06-044	goodbit 근거 대조 문구: goodbit	(B istream 클래스, 인쇄 p.345)
U06-045	스트림에 입출력 작업이 가능할 때 근거 대조 문구: goodbit	(B istream 클래스, 인쇄 p.345)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U06-046	badbit 근거 대조 문구: badbit	(B istream 클래스, 인쇄 p.345)
U06-047	스트림에 복구 불가능한 오류 발생 시 근거 대조 문구: badbit	(B istream 클래스, 인쇄 p.345)
U06-048	failbit 근거 대조 문구: failbit	(B istream 클래스, 인쇄 p.345)
U06-049	스트림에 복구 가능한 오류 발생 시 근거 대조 문구: failbit	(B istream 클래스, 인쇄 p.345)
U06-050	eofbit 근거 대조 문구: eofbit	(B istream 클래스, 인쇄 p.345)
U06-051	입력 작업 시에 EOF 도달 시 근거 대조 문구: eofbit	(B istream 클래스, 인쇄 p.345)
U06-052	두 피연산자를 pop하여 연산하고 결과를 다시 push한다. 근거 대조 문구: 그결과를다시스택에push	(B 엑셀 만들기 프로젝트 2부 / 중위 표기법과 후위 표기법, 인쇄 p.398)

07 템플릿·가변 인자·메타프로그래밍

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-001	템플릿으로 타입에 관계없이 코드를 작성하고 함수 템플릿과 클래스 템플릿을 다룬다. 근거 대조 문구: 함수템플릿	(L09 Generic Programming과 템플릿, 슬라이드 3)
U07-002	가변 길이 템플릿으로 임의 개수의 인자를 받고 TMP로 컴파일타임 연산을 다룬다. 근거 대조 문구: 임의의개수의인자를받는방법	(B 가변 길이 템플릿 / 템플릿 메타프로그래밍, 인쇄 p.445,463)
U07-003	Generic Programming은 타입에 관계없이 동작하는 일반적인 코드를 작성하는 방법이다. 근거 대조 문구: 타입에관계없이동작하는	(L09 Generic Programming, 슬라이드 5)
U07-004	매크로(#define)는 코드를 단순 대체하며, 매크로문을 괄호로 감싸고 세미콜론을 포함하지 않는 것이 기본 약속이다. 근거 대조 문구: 코드의단순대체	(L09 Generic Programming using Macro, 슬라이드 6,11)
U07-005	템플릿은 원하는 타입을 넣으면 컴파일러가 해당 타입으로 코드를 생성하는 틀이다. 근거 대조 문구: 사용자(프로그래머)가원하는타입을넣어	(B C++ 템플릿(template), 인쇄 p.413,416)
U07-006	template <typename T>의 T는 타입 이름을 받는 템플릿 인자이며, template <class T>도 같은 의미이다. 근거 대조 문구: 정확히typename T 와동일	(B C++ 템플릿(template), 인쇄 p.415-416)
U07-007	클래스 템플릿에 인자를 전달하여 실제 코드를 생성하는 것을 클래스 템플릿 인스턴스화(class template instantiation)라고 한다. 근거 대조 문구: 실제코드를생성하는것을클래스템플릿인스턴스화	(B C++ 템플릿(template), 인쇄 p.418)
U07-008	템플릿은 인스턴스화되지 않은 상태에서는 컴파일 시 실제 코드로 변환되지 않는다. 근거 대조 문구: 인스턴스화되지않고덩그러니있다면	(B C++ 템플릿(template), 인쇄 p.418)
U07-009	템플릿 특수화(template specialization)는 일부 경우를 따로 처리하는 것이며, 전달할 템플릿 인자가 없어도 template<>를 남겨야 한다. 근거 대조 문구: 일부경우에대해서따로처리	(B 템플릿 특수화(template specialization), 인쇄 p.419)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-010	원문의 Vector<bool>은 여러 bool을 정수의 비트에 묶어서 저장하도록 따로 특수화한다. 근거 대조 문구: 이렇게한데묶어서저장	(B 템플릿 특수화(template specialization), 인쇄 p.419-420)
U07-011	함수 템플릿 max(a, b)는 a와 b의 타입을 보고 컴파일러가 템플릿 인자를 추론하여 인스턴스화한다. 근거 대조 문구: a 와b 의타입을보고알아서	(B 함수 템플릿(Function template), 인쇄 p.428)
U07-012	원문의 bubble_sort가 작동하려면 컨테이너에 size·swap·operator[]가 정의되어 있어야 한다. 근거 대조 문구: size 함수와, swap 함수, 그리고[ ] 연산자	(B 함수 템플릿(Function template), 인쇄 p.429)
U07-013	템플릿이 사용하는 멤버가 전달한 타입에 없으면 런타임이 아니라 컴파일할 때 오류가 발생한다. 근거 대조 문구: 실행되었을때가아니라컴파일할때	(B 함수 템플릿(Function template), 인쇄 p.432)
U07-014	함수 객체(Function Object), 또는 Functor는 함수는 아니지만 함수처럼 사용하는 객체이며 원문에서는 operator()를 정의한다. 근거 대조 문구: 함수는아니지만함수인척을하는객체	(B 함수 객체(Function Object - Functor)의 도입, 인쇄 p.436)
U07-015	템플릿 인자는 타입 외의 값도 받을 수 있고, 원문의 add_num은 타입 T와 int num을 받는다. 근거 대조 문구: 타입만받을수있는것이아닙니다.	(B 타입이 아닌 템플릿 인자(non-type template arguments), 인쇄 p.438)
U07-016	std::array는 원소 타입과 크기를 템플릿 인자로 명시하며, 원문 예제에서는 런타임에 동적으로 크기를 할당하지 않는다. 근거 대조 문구: 런타임에서동적으로크기가할당되는것이아니라	(B 타입이 아닌 템플릿 인자(non-type template arguments), 인쇄 p.440)
U07-017	템플릿 인자에도 = 뒤에 디폴트 값을 지정할 수 있으며 타입 역시 디폴트로 지정할 수 있다. 근거 대조 문구: 타입역시디폴트로지정이가능	(B 디폴트 템플릿 인자, 인쇄 p.442-443)
U07-018	템플릿 파라미터 팩과 함수 파라미터 팩은 각각 0개 이상의 템플릿 인자와 함수 인자를 나타낸다. 근거 대조 문구: 0 개이상의함수인자를나타냅니다.	(B 가변 길이 템플릿, 인쇄 p.446)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-019	원문의 print는 첫 인자를 출력한 뒤 print(args...)로 나머지 인자를 재귀적으로 전달한다. 근거 대조 문구: 나머지인자들이쭈르륵	(B 가변 길이 템플릿, 인쇄 p.446-447)
U07-020	재귀 호출을 끝내는 print를 뒤에 정의하면 원문 예제는 컴파일 오류가 발생하므로 템플릿 함수의 작성 순서에 유의한다. 근거 대조 문구: 템플릿함수를작성할때그순서에유의	(B 순서를 바꾼다면?, 인쇄 p.448-449)
U07-021	StrCat의 두 번째 구현은 전체 문자열 길이를 먼저 구해 reserve로 공간을 할당한 뒤 append로 문자열을 붙인다. 근거 대조 문구: reserve 함수를통해서필요한만큼	(B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.451-452)
U07-022	파라미터 팩에 sizeof...을 사용하면 전체 인자의 개수를 리턴한다. 근거 대조 문구: 전체인자의개수를리턴	(B sizeof..., 인쇄 p.453)
U07-023	Fold 식은 파라미터 팩의 연산을 표현하며, Fold 식을 쓸 때 반드시 괄호 ()로 감싸야 한다. 근거 대조 문구: Fold 식을쓸때꼭() 로감싸	(B Fold Expression, 인쇄 p.455-456)
U07-024	템플릿 메타프로그래밍(Template Meta Programming, TMP)은 타입을 가지고 컴파일타임에 생성되는 코드로 프로그래밍하는 것이다. 근거 대조 문구: 타입을가지고컴파일타임에생성되는코드	(B 템플릿 메타프로그래밍 (Template Meta Programming - TMP), 인쇄 p.463-464)
U07-025	Factorial은 재귀적인 템플릿 구조를 사용하고 Factorial<1>의 특수화로 재귀 구조를 끝낸다. 근거 대조 문구: 재귀적구조가끝날수있게	(B 템플릿 메타프로그래밍 1부, 인쇄 p.464-465)
U07-026	GCD는 유클리드 호제법을 템플릿으로 표현하고, Ratio는 분자와 분모를 템플릿 인자로 받아 유리수를 나타낸다. 근거 대조 문구: 분자와분모를템플릿인자로받고	(B 템플릿 메타프로그래밍 1부, 인쇄 p.466-467)
U07-027	using은 typedef 대신 타입 이름을 정의하는 데 사용할 수 있다. 근거 대조 문구: typedef 대신에좀더직관적인using	(B 템플릿 메타프로그래밍 1부, 인쇄 p.468)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-028	의존 타입(dependent type)은 템플릿 인자에 따라 달라질 수 있는 타입이며, 원문은 의존하는 식별자가 타입임을 알리기 위해 typename을 붙인다. 근거 대조 문구: 템플릿인자에따라서어떠한타입이달라질	(B 템플릿 메타프로그래밍 2부 / 지난 강의 생각해보기 문제, 인쇄 p.477-478)
U07-029	Dim은 단위 정보를 타입으로 나타내고 quantity는 실제 데이터 q와 차원 정보 dim_type을 둔다. 근거 대조 문구: dim_type 에차원정보	(B 단위(Unit) 라이브러리, 인쇄 p.481-483)
U07-030	quantity의 +와 -는 같은 단위가 필요하고, *와 /는 다른 차원을 받아 새 차원을 만든다. 근거 대조 문구: 굳이Dim 이일치하지않아도	(B 단위(Unit) 라이브러리, 인쇄 p.483,486)
U07-031	auto를 사용하면 컴파일러가 초기화하는 식에서 해당 타입을 컴파일 시 추론한다. 근거 대조 문구: 컴파일시에컴파일러에의해추론	(B 타입을 알아서 추측해라! - auto 키워드, 인쇄 p.488-489)
U07-032	타입명을 T로 대체한다. 근거 대조 문구: 타입명을T로대체	(L09 Generic Programming using Template, 슬라이드 16)
U07-033	template <typename T> 또는 template <class T>로 T가 템플릿 인수임을 명시한다. 근거 대조 문구: T가템플릿인수라는것을명시	(L09 Generic Programming using Template, 슬라이드 16)
U07-034	GetStringSize로 합쳐질 문자열의 총길이를 계산한다. 근거 대조 문구: 총길이를계산	(B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.452)
U07-035	reserve로 concat_str에 필요한 공간을 미리 할당한다. 근거 대조 문구: 미리공간을할당	(B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.452)
U07-036	AppendToString이 append 후 나머지 인자들을 재귀적으로 전달하고 인자가 없는 함수로 종료한다. 근거 대조 문구: 재귀호출을끝내기위해선	(B 임의의 개수의 문자열을 합치는 함수, 인쇄 p.452)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U07-037	가변 길이 print: 원문 발췌 (원문 코드 발췌)
	template <typename T>
	void print(T arg) {
	std::cout << arg << std::endl;
	}
	
	template <typename T, typename... Types>
	void print(T arg, Types... args) {
	std::cout << arg << ", ";
	print(args...);
	}
	근거 대조 문구: template <typename T>
	void print(T arg) {
	std::cout << arg << std::endl;
	}
	
	template <typename T, typename... Types>
	void print(T arg, Types... args) {
	std::cout << arg << ", ";
	print(args...);
	}
		(B 가변 길이 템플릿, 인쇄 p.445)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U07-038	Factorial 템플릿과 특수화: 원문 발췌 (원문 코드 발췌)
	template <int N>
	struct Factorial {
	static const int result = N * Factorial<N - 1>::result;
	};
	
	template <>
	struct Factorial<1> {
	static const int result = 1;
	};
	근거 대조 문구: template <int N>
	struct Factorial {
	static const int result = N * Factorial<N - 1>::result;
	};
	
	template <>
	struct Factorial<1> {
	static const int result = 1;
	};
		(B 템플릿 메타프로그래밍 1부, 인쇄 p.464)

U07-039	단항 좌측 Fold: 원문 전개: (... + nums) → ((((1 + 4) + 2) + 3) + 10); 원문의 인자 1, 4, 2, 3, 10을 사용한 전개이다. 근거 대조 문구: return ((((1 + 4) + 2) + 3) + 10);	(B Fold Expression, 인쇄 p.455)
U07-040	매크로(#define) 근거 대조 문구: 매크로(#define)	(L09 Generic Programming using Macro, 슬라이드 6)
U07-041	코드의 단순 대체 근거 대조 문구: 코드의단순대체	(L09 Generic Programming using Macro, 슬라이드 6)
U07-042	템플릿 근거 대조 문구: C++ 템플릿	(L09 Generic Programming using Template, 슬라이드 15)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U07-043	컴파일러가 설계도를 기반으로 함수·클래스를 생성하며, 사용하지 않는 경우 생성하지 않는다. 근거 대조 문구: 사용하지않는경우생성하지않음	(L09 Generic Programming using Template, 슬라이드 15)
U07-044	파라미터 팩은 여러 인자를 나타내고 sizeof...은 그 인자의 개수를 반환한다. 근거 대조 문구: 전체인자의개수를리턴	(B 가변 길이 템플릿 / sizeof..., 인쇄 p.446,453)

U07-045	Fold 합산: 원문 발췌 (원문 코드 발췌)
	template <typename... Ints>
	int sum_all(Ints... nums) {
	return (... + nums);
	}
	근거 대조 문구: template <typename... Ints>
	int sum_all(Ints... nums) {
	return (... + nums);
	}
		(B Fold Expression, 인쇄 p.455)

08 STL 컨테이너·알고리즘·문자열

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-001	STL의 컨테이너·반복자·알고리즘을 다룬다. 근거 대조 문구: 반복자(iterator)	(B C++ 표준 템플릿 라이브러리, 인쇄 p.494)
U08-002	표준 문자열 라이브러리 string과 string_view를 살펴본다. 근거 대조 문구: 표준문자열라이브러리	(B C++ 문자열의 모든 것(string과 string_view), 인쇄 p.591)
U08-003	STL은 임의 타입의 객체를 보관하는 컨테이너, 원소에 접근하는 반복자, 반복자로 작업을 수행하는 알고리즘으로 구성된다. 근거 대조 문구: 컨테이너(container)	(B C++ 표준 템플릿 라이브러리(Standard Template Library - STL), 인쇄 p.494)
U08-004	시퀀스 컨테이너는 객체를 순차적으로 보관하고, 연관 컨테이너는 키를 바탕으로 대응되는 값을 찾는다. 근거 대조 문구: 시퀀스컨테이너(sequence container)	(B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.495)
U08-005	vector는 가변 길이 배열이며 원소가 메모리상 연속적으로 저장된다. 근거 대조 문구: 메모리상에서실제로순차적으로저장	(B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.495)
U08-006	vector의 임의 위치 접근은 O(1), 맨 뒤 추가·제거는 amortized O(1), 임의 위치 추가·제거는 O(n)으로 정리한다. 근거 대조 문구: amortized O(1)	(B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.500)
U08-007	vector가 할당한 공간을 다 채우면 큰 공간을 새로 할당하고 기존 원소들을 복사한다. 근거 대조 문구: 새로운큰공간을다시할당하고	(B C++ STL 컨테이너 - 벡터(std::vector), 인쇄 p.498)
U08-008	반복자(iterator)는 컨테이너 원소에 접근할 수 있는 포인터와 같은 객체이다. 근거 대조 문구: 원소에접근할수있는포인터와같은객체	(B 반복자(iterator), 인쇄 p.500)
U08-009	begin()은 첫 원소를, end()는 마지막 원소 한 칸 뒤를 가리키며, begin()==end()이면 빈 컨테이너를 나타낸다. 근거 대조 문구: 마지막원소한칸뒤	(B 반복자(iterator), 인쇄 p.500)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-010	반복자의 * 연산자는 가리키는 원소의 레퍼런스를 반환하고, vector의 insert는 지정 반복자 앞에 추가하며 erase는 그 원소를 지운다. 근거 대조 문구: 그반복자앞에원소를추가	(B 반복자(iterator), 인쇄 p.502,504)
U08-011	원문은 vector에서 erase·insert 후 기존 반복자를 계속 사용하는 문제를 다룬다. 근거 대조 문구: itr 은유효한반복자가아니게	(B 반복자(iterator), 인쇄 p.504-506)
U08-012	const_iterator는 가리키는 원소의 값을 바꿀 수 없으며 cbegin·cend로 얻고, 역반복자는 뒤에서 앞으로 이동한다. 근거 대조 문구: 가리키고있는원소의값을바꿀수없습니다.	(B 반복자(iterator), 인쇄 p.507-509)
U08-013	범위 기반 for에서 값으로 받으면 원소가 복사되며, const auto&로 받으면 원소를 상수 레퍼런스로 접근한다. 근거 대조 문구: 상수레퍼런스로접근	(B 범위 기반 for 문 (range based for loop), 인쇄 p.511,513)
U08-014	list는 양방향 연결 구조로 []·at를 제공하지 않으며, 원하는 위치를 가리킨 상태에서 원소 추가·제거를 O(1)에 수행한다. 근거 대조 문구: 링크값만바꿔주면	(B 리스트(list), 인쇄 p.513-514)
U08-015	list의 반복자는 한 칸씩 양방향 이동하는 BidirectionalIterator이고, vector의 반복자는 임의 위치에 접근하는 RandomAccessIterator이다. 근거 대조 문구: 한칸씩밖에이동할수없습니다.	(B 리스트(list), 인쇄 p.515)
U08-016	deque는 양 끝의 추가·제거와 임의 위치 접근을 O(1)에 수행하며, 임의 위치 추가·제거는 O(n)이다. 근거 대조 문구: 맨앞에원소를추가/제거하는작업까지도	(B 덱(deque - double ended queue), 인쇄 p.517)
U08-017	deque의 원소는 일정 크기의 블록에 나뉘어 있고, 각 블록의 위치를 보관하기 위한 메모리가 필요하다. 근거 대조 문구: 일정크기로잘려서각각의블록	(B 덱(deque - double ended queue), 인쇄 p.517-518)
U08-018	set은 정렬 상태를 유지하며 중복된 원소를 넣지 않고, find는 원소가 있으면 반복자, 없으면 end()를 반환한다. 근거 대조 문구: 이미같은원소가있다면이를insert 하지	(B 셋(set), 인쇄 p.525,528-529)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-019	set에서 두 원소가 같다는 것은 양쪽 방향의 operator< 결과가 모두 false라는 뜻이다. 근거 대조 문구: 모두false 라는뜻	(B 여러분이 만든 클래스 객체를 셋에 넣고 싶을 때, 인쇄 p.532)
U08-020	set의 비교는 operator< 또는 두 번째 템플릿 인자로 전달하는 비교 함수 객체로 지정할 수 있다. 근거 대조 문구: set 에두번째인자로넘겨주게	(B 여러분이 만든 클래스 객체를 셋에 넣고 싶을 때, 인쇄 p.530,535-536)
U08-021	map은 키와 대응하는 값을 보관하며, 반복자의 first는 키이고 second는 값이다. 근거 대조 문구: itr->second 를하면해당원소의값	(B 맵(map), 인쇄 p.536,538)
U08-022	map의 []는 없는 키를 참조하면 값의 디폴트 생성자를 호출하여 원소를 추가하고, 기존 키이면 대응 값을 바꿀 수 있다. 근거 대조 문구: 자동으로값의디폴트생성자를호출	(B 맵(map), 인쇄 p.538,540,542)
U08-023	multiset과 multimap은 중복을 허용하고, multimap은 []를 제공하지 않으며 equal_range로 해당 키의 원소 범위를 얻는다. 근거 대조 문구: 중복된원소를허락	(B 멀티셋(multiset)과 멀티맵(multimap), 인쇄 p.542,544-545)
U08-024	unordered_set·unordered_map은 원소가 정렬되지 않으며, 해시 함수를 사용해 원소를 저장할 위치를 정한다. 근거 대조 문구: 원소들이정렬되어있지않습니다.	(B 정렬되지 않은 셋과 맵(unordered_set, unordered_map), 인쇄 p.546,548)
U08-025	해시 함수는 임의 크기의 데이터를 고정 크기의 데이터에 대응시키고, 서로 다른 원소가 같은 해시값을 가지면 해시 충돌이라 한다. 근거 대조 문구: 이를해시충돌(hash collision)	(B 해시 함수(Hash function), 인쇄 p.548)
U08-026	unordered_set·unordered_map의 삽입·탐색은 평균 O(1), 최악 O(N)이고, rehash는 모든 원소를 다시 insert하여 O(N)이 걸린다. 근거 대조 문구: 평균적으로O(1)	(B 해시 함수(Hash function), 인쇄 p.549)
U08-027	사용자 클래스의 unordered 컨테이너에는 해시 함수와 충돌 시 비교할 operator==가 필요하다. 근거 대조 문구: operator== 는필요한데	(B 내가 만든 클래스를 unordered_set/unordered_map의 원소로 넣고 싶을 때, 인쇄 p.551)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-028	알고리즘은 보통 시작 반복자와 끝 바로 뒤 반복자를 받으며, 서술자(Predicate)에는 보통 bool을 반환하는 함수 객체를 전달한다. 근거 대조 문구: 서술자(Predicate)	(B C++ 표준 알고리즘 라이브러리, 인쇄 p.556)
U08-029	sort는 기본적으로 오름차순으로 정렬하며 RandomAccessIterator를 요구하고, 비교 방법을 추가 인자로 받을 수 있다. 근거 대조 문구: 반드시임의접근반복자	(B 정렬(sort, stable_sort, partial_sort), 인쇄 p.558)
U08-030	stable_sort는 크기가 같은 원소들의 기존 순서를 보존하며, partial_sort는 전체 원소 중 앞의 지정 구간을 작은 원소 순으로 정렬한다. 근거 대조 문구: 그순서를반드시보존	(B 정렬(sort, stable_sort, partial_sort), 인쇄 p.556,561)
U08-031	remove는 원소를 이동할 뿐 실제 삭제하지 않으므로, 컨테이너에서 지우려면 erase를 호출한다. 근거 대조 문구: 실제로원소를삭제하는연산을수행하지는	(B 원소 제거(remove, remove_if), 인쇄 p.568)
U08-032	remove_if는 조건을 나타내는 함수 객체를 받으며, 원문은 그 객체가 여러 번 복사될 수 있어 내부 상태에 의존하는 결과를 주의하도록 한다. 근거 대조 문구: 해당함수객체가여러번복사될수	(B remove_if에 조건 추가하기, 인쇄 p.569,572-573)
U08-033	람다는 이름 없는 함수 객체를 만들며, 캡쳐 목록으로 외부 변수를 사용하고 &로 참조를 캡쳐한다. 근거 대조 문구: 레퍼런스를캡쳐한다는의미	(B 람다 함수(lambda function), 인쇄 p.575-576,579)
U08-034	transform은 각 원소에 함수를 적용한 결과를 결과 시작 반복자부터 저장하며, 결과를 담는 공간이 충분해야 한다. 근거 대조 문구: 결과를저장할컨테이너의시작반복자	(B 원소 수정하기(transform), 인쇄 p.582)
U08-035	find는 값이 같은 첫 원소를 찾고, find_if는 조건이 참인 원소를 찾는다. 근거 대조 문구: 그결과가참인원소들을찾게	(B 원소를 탐색하는 함수(find, find_if, any_of, all_of 등등), 인쇄 p.585,588)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-036	any_of는 범위에서 하나라도 조건을 충족하면 true, all_of는 모든 원소가 조건을 충족해야 true를 반환한다. 근거 대조 문구: 조건을하나라도충족	(B 원소를 탐색하는 함수 (find, find_if, any_of, all_of 등등), 인쇄 p.589)
U08-037	std::string은 basic_string<char>의 인스턴스화 버전이며, basic_string은 문자를 연속 저장하고 Traits는 기본 문자열 연산을 정의한다. 근거 대조 문구: 문자열연산들을정의해놓은클래스	(B basic_string, 인쇄 p.495,591-592)
U08-038	짧은 문자열 최적화(SSO)는 짧은 문자열 데이터를 따로 할당하는 대신 객체 자체에 저장하는 방식이다. 근거 대조 문구: 그냥객체자체에저장	(B 짧은 문자열 최적화 (SSO), 인쇄 p.595)
U08-039	원문의 "hello"s는 string으로 추론되는 리터럴이며, Raw string literal은 이스케이프 없이 문자들을 그대로 넣고 구분자를 지정할 수 있다. 근거 대조 문구: 문자들은모두문자그대로	(B 리터럴 연산자 / 그 외의 여러 가지 리터럴 정의 방법, 인쇄 p.597-599)
U08-040	유니코드는 문자에 고유 값을 부여하며, UTF-8은 1~4바이트, UTF-16은 2 또는 4바이트, UTF-32는 4바이트로 문자를 표현한다. 근거 대조 문구: UTF-16 : 문자를2 혹은4 바이트	(B C++에서 한글 다루기, 인쇄 p.600)
U08-041	std::string의 size()는 실제 문자 수가 아니라 char 원소 수를 반환하므로, UTF-8 문자열의 문자 수와 다를 수 있다. 근거 대조 문구: 그냥char 이몇개가있는지	(B UTF-8 인코딩, 인쇄 p.602)
U08-042	string_view는 문자열을 소유하지 않고 참조해서 읽으며, 가리키는 문자열이 소멸한 뒤 사용하면 Undefined behavior가 발생한다. 근거 대조 문구: 문자열을소유하고있지않습니다.	(B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)
U08-043	string의 substr은 새 부분 문자열을 만들어 O(n), string_view의 substr은 다른 view를 만들어 O(1)에 수행된다. 근거 대조 문구: 또다른view 를생성하므로O(1)	(B string_view / 소유하지 않고 읽기만 한다!, 인쇄 p.609)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-044	remove가 남길 원소를 앞으로 옮긴 뒤 그 끝을 가리키는 반복자를 반환한다. 근거 대조 문구: 원소의이동만을수행	(B 원소 제거(remove, remove_if), 인쇄 p.567-568)
U08-045	remove 결과부터 기존 end까지 erase하여 실제 원소를 지운다. 근거 대조 문구: 반드시erase 함수를호출	(B 원소 제거(remove, remove_if), 인쇄 p.568)
U08-046	equal_range(key)로 시작과 끝 바로 뒤 반복자를 담은 pair를 얻는다. 근거 대조 문구: 시작과끝바로다음을가리키는반복자	(B 멀티셋(multiset)과 멀티맵(multimap), 인쇄 p.545)
U08-047	range.first부터 range.second 직전까지 반복자를 증가시키며 각 원소를 읽는다. 근거 대조 문구: range.first; itr != range.second	(B 멀티셋(multiset)과 멀티맵(multimap), 인쇄 p.546)

U08-048	반복자 범위 출력: 원문 발췌 (원문 코드 발췌)
	template <typename Iter>
	void print(Iter begin, Iter end) {
	while (begin != end) {
	std::cout << *begin << " ";
	begin++;
	}
	std::cout << std::endl;
	}
	근거 대조 문구: template <typename Iter>
	void print(Iter begin, Iter end) {
	while (begin != end) {
	std::cout << *begin << " ";
	begin++;
	}
	std::cout << std::endl;
	}
		(B 정렬(sort, stable_sort, partial_sort), 인쇄 p.557)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U08-049	람다와 remove_if: 원문 발췌 (원문 코드 발췌)
	std::cout << "벡터에서홀수인원소제거---" << std::endl;
	vec.erase(std::remove_if(vec.begin(), vec.end(),
	[](int i) -> bool { return i % 2 == 1; }),
	vec.end());
	print(vec.begin(), vec.end());
	근거 대조 문구: std::cout << "벡터에서홀수인원소제거---" << std::endl;
	vec.erase(std::remove_if(vec.begin(), vec.end(),
	[](int i) -> bool { return i % 2 == 1; }),
	vec.end());
	print(vec.begin(), vec.end());
		(B 람다 함수(lambda function), 인쇄 p.575)

U08-050	partial_sort 복잡도: O(N log M); N: 전체 원소 수, M: 정렬하려는 부분의 크기. 근거 대조 문구: partial_sort 의복잡도는O(N log M)	(B 정렬(sort, stable_sort, partial_sort), 인쇄 p.561)
U08-051	vector 근거 대조 문구: vector	(B 벡터(std::vector), 인쇄 p.500)
U08-052	연속 저장; 임의 접근 O(1); 임의 위치 추가·제거 O(n) 근거 대조 문구: 연속 저장	(B 벡터(std::vector), 인쇄 p.495,500)
U08-053	list 근거 대조 문구: list	(B 리스트(list), 인쇄 p.514)
U08-054	양방향 연결; []·at 없음; 위치를 찾은 뒤 추가·제거 O(1) 근거 대조 문구: 양방향 연결	(B 리스트(list), 인쇄 p.513-514)
U08-055	deque 근거 대조 문구: deque	(B 덱(deque), 인쇄 p.517)
U08-056	양 끝 추가·제거 O(1); 임의 접근 O(1); 임의 위치 추가·제거 O(n) 근거 대조 문구: 양 끝 추가·제거 O(1)	(B 덱(deque), 인쇄 p.517)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U08-057	STL은 컨테이너·알고리즘·반복자로 구성되며 반복자는 역참조를 통해 값에 접근한다. 근거 대조 문구: 컨테이너, 알고리즘, 반복자	(L99 Summary of 2023-2 C++ Programming Class / STL, 슬라이드 8)
U08-058	std::string은 basic_string의 char 인자 인스턴스화 버전이며 char_traits로 원하는 기능을 가진 문자열을 만들 수 있다. 근거 대조 문구: std::char_traits 를사용해서	(B 뭘 배웠지?, 인쇄 p.612)
U08-059	문자 인코딩 방식에 따라 문자별로 인코딩되는 길이가 다를 수 있다. 근거 대조 문구: 문자별로인코딩되는길이가다르다	(B 뭘 배웠지?, 인쇄 p.612)
U08-060	string_view로 불필요한 복사를 막고 const char*과 const string& 사이의 처리를 할 수 있다. 근거 대조 문구: 불필요한복사를막고	(B 뭘 배웠지?, 인쇄 p.612)

09 예외 처리

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-001	C++에서 예외를 발생시키고 처리하는 방식을 다룬다. 근거 대조 문구: 예외처리를어떠한방식	(B C++에서의 예외 처리, 인쇄 p.613)
U09-002	스택 풀기, 여러 종류의 예외, 모든 예외 받기와 noexcept를 다룬다. 근거 대조 문구: 스택풀기(stack unwinding)	(B C++에서의 예외 처리, 인쇄 p.618,621,626-627)
U09-003	예외(exception)는 정상적인 상황에서 벗어난 예외적인 상황이다. 근거 대조 문구: 정상적인상황에서벗어난모든예외적인상황	(B 예외란?, 인쇄 p.613)
U09-004	원문은 범위를 벗어난 vector 접근과 큰 메모리 할당을 실행 중 오류가 발생할 수 있는 예로 든다. 근거 대조 문구: 실제로실행해보면오류	(B 예외란?, 인쇄 p.613)
U09-005	기존 방식은 작업 후 결과값을 확인하며, 깊은 함수에서 발생한 문제를 바깥에서 확인하려면 각 함수가 처리 결과를 전달해야 한다. 근거 대조 문구: 각각의함수들에서처리결과를모두리턴	(B 기존의 예외 처리 방식, 인쇄 p.614-615)
U09-006	C++은 throw로 예외가 발생했다는 사실을 명시적으로 나타낼 수 있다. 근거 대조 문구: 명시적으로나타낼수있습니다.	(B 예외 발생시키기 - throw, 인쇄 p.615)
U09-007	throw 다음에는 예외로 전달할 객체를 쓰며, 원문은 std::out_of_range 객체를 던진다. 근거 대조 문구: 예외로전달하고싶은객체	(B 예외 발생시키기 - throw, 인쇄 p.616)
U09-008	표준 라이브러리에는 out_of_range·overflow_error·length_error·runtime_error 등의 예외가 정의되어 있다. 근거 대조 문구: overflow_error, length_error	(B 예외 발생시키기 - throw, 인쇄 p.616)
U09-009	throw를 실행하면 그 위치에서 즉시 함수를 빠져나가 예외 처리 부분으로 이동하므로 아래 문장들은 실행하지 않는다. 근거 대조 문구: throw 밑에있는모든문장은실행되지	(B 예외 발생시키기 - throw, 인쇄 p.616)
U09-010	예외를 처리하는 부분까지 함수를 빠져나가는 동안 스택에 생성된 객체들의 소멸자를 호출한다. 근거 대조 문구: stack 에생성되었던객체들을빠짐없이소멸	(B 예외 발생시키기 - throw, 인쇄 p.616)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-011	try 안에는 예외가 발생할 만한 코드를 넣으며, 예외가 없으면 catch는 무시한다. 근거 대조 문구: 밑에있는catch 문은무시	(B 예외 처리하기 - try와 catch, 인쇄 p.618)
U09-012	catch는 던진 예외를 받는 부분이며, 괄호 안에 정의된 예외의 꼴에 맞는 객체를 받는다. 근거 대조 문구: 예외의꼴에맞는객체	(B 예외 처리하기 - try와 catch, 인쇄 p.618)
U09-013	out_of_range의 what()으로 예외에 관해 저장된 문자열을 확인할 수 있다. 근거 대조 문구: what() 함수로그값을들여다	(B 예외 처리하기 - try와 catch, 인쇄 p.618)
U09-014	스택 풀기(stack unwinding)는 catch로 이동하면서 스택상에 정의된 객체들을 소멸시키는 과정이다. 근거 대조 문구: 스택상에서정의된객체들을소멸시키는과정	(B 스택 풀기(stack unwinding), 인쇄 p.621)
U09-015	원문의 호출 예에서는 func3에서 던진 예외가 func2·func1을 지나 main의 catch까지 전달된다. 근거 대조 문구: main 함수에있는catch	(B 스택 풀기(stack unwinding), 인쇄 p.619-620)
U09-016	생성자에서 예외가 발생하면 해당 객체의 소멸자가 호출되지 않으므로, 원문은 이전에 획득한 자원을 catch에서 해제하도록 주의시킨다. 근거 대조 문구: 생성자에서예외가발생시에소멸자가호출되지	(B 스택 풀기(stack unwinding), 인쇄 p.621)
U09-017	하나의 try에 여러 catch를 붙여 char·int·string·const char* 등 여러 종류의 예외를 받을 수 있다. 근거 대조 문구: 여러종류의throw 된객체	(B 여러 종류의 예외 받기, 인쇄 p.621-623)
U09-018	catch는 가장 먼저 대입될 수 있는 객체를 받으므로, Parent catch가 앞에 있으면 Child 예외도 먼저 받는다. 근거 대조 문구: 가장먼저대입될수있는객체	(B 여러 종류의 예외 받기, 인쇄 p.624)
U09-019	원문은 파생 클래스 예외를 받는 catch를 기반 클래스 catch보다 앞에 두어 각각 처리한다. 근거 대조 문구: Parent catch 를Child catch 보다뒤	(B 여러 종류의 예외 받기, 인쇄 p.624-625)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-020	던진 예외를 받는 catch가 없으면 원문 예제는 프로그램이 비정상 종료된다. 근거 대조 문구: 프로그램이비정상적으로종료	(B 모든 예외 받기, 인쇄 p.626)
U09-021	catch(...)는 try 안에서 발생한 모든 예외를 받지만 특정 타입의 객체로 받을 수는 없다. 근거 대조 문구: 모든예외들을받게	(B 모든 예외 받기, 인쇄 p.627)
U09-022	noexcept는 함수가 예외를 발생시키지 않음을 명시하는 키워드이다. 근거 대조 문구: 예외를발생시키지않는다면noexcept	(B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.627)
U09-023	noexcept 함수가 실제 예외를 발생시키면 예외가 제대로 처리되지 않고 프로그램이 종료된다. 근거 대조 문구: 프로그램이종료됩니다.	(B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.628)
U09-024	원문은 C++ 11부터 소멸자가 기본적으로 noexcept이므로 소멸자에서 예외를 던지지 않도록 주의시킨다. 근거 대조 문구: 소멸자들은기본적으로noexcept	(B 예외를 발생시키지 않는 함수 - noexcept, 인쇄 p.629)
U09-025	try 안에서 vec.at(index)를 호출한다. 근거 대조 문구: data = vec.at(index);	(B 예외 처리하기 - try와 catch, 인쇄 p.617)
U09-026	index가 범위를 벗어나면 std::out_of_range를 throw한다. 근거 대조 문구: throw std::out_of_range	(B 예외 처리하기 - try와 catch, 인쇄 p.617)
U09-027	catch(std::out_of_range& e)가 받아 e.what()을 출력한다. 근거 대조 문구: e.what()	(B 예외 처리하기 - try와 catch, 인쇄 p.617)
U09-028	func3에서 예외를 던진다. 근거 대조 문구: Exception from 3!	(B 스택 풀기(stack unwinding), 인쇄 p.619)
U09-029	catch가 없는 func2·func1을 빠져나가면서 각 함수의 객체를 소멸시킨다. 근거 대조 문구: 각함수들에정의되어있던객체들이잘소멸	(B 스택 풀기(stack unwinding), 인쇄 p.619-621)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U09-030	main에 있는 catch에서 전달된 예외를 처리한다. 근거 대조 문구: 예외가main 함수에까지잘전달	(B 스택 풀기(stack unwinding), 인쇄 p.620)

U09-031	out_of_range 처리: 원문 발췌 (원문 코드 발췌)
	try {
	data = vec.at(index);
	} catch (std::out_of_range& e) {
	std::cout << "예외발생! " << e.what() << std::endl;
	}
	근거 대조 문구: try {
	data = vec.at(index);
	} catch (std::out_of_range& e) {
	std::cout << "예외발생! " << e.what() << std::endl;
	}
		(B 예외 처리하기 - try와 catch, 인쇄 p.617)

U09-032	throw 근거 대조 문구: throw	(B 예외 발생시키기 - throw, 인쇄 p.616)
U09-033	예외 객체를 던지고 처리 부분으로 이동한다. 근거 대조 문구: 예외처리하는부분까지점프	(B 예외 발생시키기 - throw, 인쇄 p.616)
U09-034	try 근거 대조 문구: try	(B 예외 처리하기 - try와 catch, 인쇄 p.618)
U09-035	catch 근거 대조 문구: catch	(B 예외 처리하기 - try와 catch, 인쇄 p.618)

10 우측값·이동·완벽한 전달

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-001	우측값 레퍼런스와 이동 생성자를 다룬다. 근거 대조 문구: 우측값레퍼런스에대해서	(B 우측값 레퍼런스와 이동 생성자, 인쇄 p.630)
U10-002	좌측값을 이동할 수 있게 하는 move와 인자 전달을 위한 forward를 다룬다. 근거 대조 문구: 좌측값도이동을시키고싶다면	(B move 문법과 완벽한 전달, 인쇄 p.649,655,669)
U10-003	복사 생략(copy elision)은 컴파일러가 불필요한 복사 생성을 생략하는 작업이다. 근거 대조 문구: 컴파일러자체에서복사를생략	(B 복사 생략(Copy Elision), 인쇄 p.631)
U10-004	원문의 MyString 덧셈 예는 반환된 임시 문자열을 복사할 때 추가적인 자원이 소모되는 문제를 다룬다. 근거 대조 문구: 쓸데없는복사를두번	(B 복사 생략(Copy Elision), 인쇄 p.634-635)
U10-005	nullptr는 원문에서 포인터 주소값 0을 명확히 나타내며 기존 NULL을 대체하는 키워드로 설명한다. 근거 대조 문구: 포인터주소값0	(B 복사 생략(Copy Elision), 인쇄 p.634)
U10-006	원문은 주소값을 취할 수 있는 값을 좌측값(lvalue), 주소값을 취할 수 없는 값을 우측값(rvalue)으로 설명한다. 근거 대조 문구: 주소값을취할수없는값을우측값	(B 좌측값(lvalue)과 우측값(rvalue), 인쇄 p.635)
U10-007	& 하나로 정의하는 좌측값 레퍼런스(lvalue reference)는 좌측값을 참조하며, 좌측값 레퍼런스 자체도 좌측값이다. 근거 대조 문구: 좌측값레퍼런스자체도좌측값	(B 좌측값(lvalue)과 우측값(rvalue), 인쇄 p.636)
U10-008	const T&는 예외적으로 우측값도 참조할 수 있지만 그 임시 객체의 값을 변경할 수 없다. 근거 대조 문구: 우측값도레퍼런스로받을수있습니다.	(B 그렇다면 이동은 어떻게?, 인쇄 p.638)
U10-009	우측값 레퍼런스는 &&로 정의하며, 원문의 MyString(MyString&& str)은 우측값을 인자로 받는다. 근거 대조 문구: & 를두개사용해서정의	(B 우측값 레퍼런스, 인쇄 p.641)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-010	우측값 레퍼런스 매개변수 str은 이름이 있으므로 표현식 자체는 좌측값이다. 근거 대조 문구: 타입이'MyString 의우측값레퍼런스' 인좌측값	(B 우측값 레퍼런스, 인쇄 p.641)
U10-011	원문의 이동 생성자는 문자열 전체를 새로 복사하는 대신 string_content의 주소를 새 객체에 전달한다. 근거 대조 문구: 단순히주소값하나만	(B 우측값 레퍼런스, 인쇄 p.642)
U10-012	이동한 원본의 string_content를 nullptr로 바꾸어 원본 소멸자가 새 객체도 가리키는 메모리를 해제하지 않게 한다. 근거 대조 문구: string_content 를nullptr 로바꿔	(B 우측값 레퍼런스, 인쇄 p.642)
U10-013	원문의 일반적인 int&& 예제는 우측값을 받을 수 있고 좌측값 a를 직접 받을 수 없다. 근거 대조 문구: a 가좌측값이기때문에컴파일되지	(B 우측값 레퍼런스, 인쇄 p.642)
U10-014	우측값 레퍼런스는 참조하는 임시 객체가 소멸되지 않도록 붙들 수 있다. 근거 대조 문구: 임시객체가소멸되지않도록	(B 우측값 레퍼런스, 인쇄 p.642)
U10-015	원문 vector 예제는 이동 생성자에 noexcept를 명시하면 재할당할 때 이동 생성자를 사용함을 보여준다. 근거 대조 문구: 이동생성자에noexcept 를추가	(B 이동 생성자 작성 시 주의할 점, 인쇄 p.643,645, 647)
U10-016	std::move는 <utility>에서 제공되며, 이동 자체를 수행하지 않고 받은 객체를 우측값으로 변환한다. 근거 대조 문구: 실제로는단순한타입변환	(B move 함수(move semantics), 인쇄 p.655-657)
U10-017	실제 이동은 우측값을 받는 이동 생성자나 이동 대입 연산자가 호출될 때 수행된다. 근거 대조 문구: 이동생성자나이동대입연산자를호출할때	(B move 함수(move semantics), 인쇄 p.661)
U10-018	원문의 MyString 이동 대입은 주소·길이·용량을 전달하고 원본의 주소를 nullptr로, 길이와 용량을 0으로 바꾼다. 근거 대조 문구: s.memory_capacity = 0;	(B move 함수(move semantics), 인쇄 p.658-660)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-019	이동 대입 연산자를 정의하지 않은 원문 예제에서는 move를 써도 일반적인 대입 연산자의 복사가 수행된다. 근거 대조 문구: 일반적인대입연산자가오버로딩	(B move 함수(move semantics), 인쇄 p.661)
U10-020	원문의 my_swap은 a로 tmp를 이동 생성하고 b를 a로, tmp를 b로 이동 대입한다. 근거 대조 문구: b 에다시tmp 를이동	(B move 함수(move semantics), 인쇄 p.660)
U10-021	완벽한 전달(perfect forwarding)은 wrapper가 받은 인자를 원래 호출과 같은 형태로 다른 함수에 전달하는 문제를 다룬다. 근거 대조 문구: 원본과Wrapper 을사용했을때모두호출되는함수가동일	(B 완벽한 전달(perfect forwarding), 인쇄 p.661-663,670)
U10-022	emplace_back은 생성자 인자를 직접 받아 내부에서 객체를 생성하여 vector 뒤에 추가한다. 근거 대조 문구: 내부에서A 의생성자를호출	(B 완벽한 전달(perfect forwarding), 인쇄 p.662)
U10-023	wrapper(T u)는 원문의 좌측값·상수 좌측값·우측값 호출에서 모두 좌측값 레퍼런스 g를 호출하여 원래 형태를 보존하지 못한다. 근거 대조 문구: 세경우모두좌측값레퍼런스	(B 완벽한 전달(perfect forwarding), 인쇄 p.663)
U10-024	wrapper(T& u)는 원문의 우측값 A()를 받지 못하고, const T&와의 오버로드도 우측값을 상수 좌측값 레퍼런스로 전달한다. 근거 대조 문구: const T& 로추론되면서	(B 완벽한 전달(perfect forwarding), 인쇄 p.664-666)
U10-025	원문은 템플릿 인자 T에 대한 T&&를 보편적 레퍼런스(Universal reference)라 부르며 좌측값과 우측값을 모두 받을 수 있다고 설명한다. 근거 대조 문구: 좌측값역시받아낼수	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.668)
U10-026	레퍼런스 겹침 규칙에서 &와 겹치면 &가 되고, &&와 &&가 겹친 경우만 &&가 된다. 근거 대조 문구: r4 는int&&	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.669)
U10-027	원문의 wrapper에서는 a에 대해 T가 A&, ca에 대해 const A&, A()에 대해 A로 추론된다. 근거 대조 문구: T 가각각A& 와const A&	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.669)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-028	std::forward<T>(u)는 원래 우측값인 인자는 우측값으로, 좌측값인 인자는 좌측값으로 전달하여 원문과 같은 g 오버로드가 호출되게 한다. 근거 대조 문구: 성공적으로인자를전달	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.669-670)
U10-029	원본의 길이·문자열 주소·용량을 새 객체에 복사한다. 근거 대조 문구: string_content = str.string_content;	(B 우측값 레퍼런스, 인쇄 p.640)
U10-030	원본 문자열 주소를 nullptr로 바꾼다. 근거 대조 문구: str.string_content = nullptr;	(B 우측값 레퍼런스, 인쇄 p.640)
U10-031	소멸자는 문자열 주소가 nullptr가 아닐 때에만 delete[]를 수행한다. 근거 대조 문구: nullptr 가아닐때에만	(B 우측값 레퍼런스, 인쇄 p.642)
U10-032	T&&로 인자를 받아 원래 인자의 형태에 맞게 T를 추론한다. 근거 대조 문구: 보편적레퍼런스	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.668-669)
U10-033	이름 있는 매개변수 u를 std::forward<T>(u)를 거쳐 g에 전달한다. 근거 대조 문구: g(std::forward<T>(u));	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.667,669)
U10-034	원래 호출과 wrapper 호출이 같은 g 오버로드를 선택한다. 근거 대조 문구: 호출되는함수가동일	(B 보편적 레퍼런스 (Universal reference), 인쇄 p.670)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U10-035	MyString 이동 생성자: 원문 발췌 (원문 코드 발췌)
	MyString::MyString(MyString &&str) {
	std::cout << "이동생성자호출!" << std::endl;
	string_length = str.string_length;
	string_content = str.string_content;
	memory_capacity = str.memory_capacity;
	
	// 임시객체소멸시에메모리를해제하지
	// 못하게한다.
	str.string_content = nullptr;
	}
	근거 대조 문구: MyString::MyString(MyString &&str) {
	std::cout << "이동생성자호출!" << std::endl;
	string_length = str.string_length;
	string_content = str.string_content;
	memory_capacity = str.memory_capacity;
	
	// 임시객체소멸시에메모리를해제하지
	// 못하게한다.
	str.string_content = nullptr;
	}
		(B 우측값 레퍼런스, 인쇄 p.640)

U10-036	move를 사용하는 my_swap: 원문 발췌 (원문 코드 발췌)
	template <typename T>
	void my_swap(T &a, T &b) {
	T tmp(std::move(a));
	a = std::move(b);
	b = std::move(tmp);
	}
	근거 대조 문구: template <typename T>
	void my_swap(T &a, T &b) {
	T tmp(std::move(a));
	a = std::move(b);
	b = std::move(tmp);
	}
		(B move 함수(move semantics), 인쇄 p.659)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U10-037	int& & 근거 대조 문구: int&&	(B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
U10-038	int& 근거 대조 문구: int&	(B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
U10-039	int& && 근거 대조 문구: int&&&	(B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
U10-040	int&& & 근거 대조 문구: int&&&	(B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
U10-041	int&& && 근거 대조 문구: int&&&&	(B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)
U10-042	int&& 근거 대조 문구: int&&	(B 보편적 레퍼런스 / 레퍼런스 겹침 규칙, 인쇄 p.669)

11 스마트 포인터·함수 객체

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-001	C++의 자원 관리와 unique_ptr·shared_ptr·weak_ptr를 다룬다. 근거 대조 문구: 자원을관리하는방법	(B 스마트 포인터, 인쇄 p.671,674,703)
U11-002	호출 가능한 모든 것을 포괄하는 Callable과 function·mem_fn·bind를 다룬다. 근거 대조 문구: Callable 에대해서	(B 함수 객체, 인쇄 p.705,718)
U11-003	원문은 할당한 메모리와 연 파일을 자원(resource)으로 들며, 사용이 끝난 자원은 반환해야 한다고 설명한다. 근거 대조 문구: 사용이끝난자원은반드시반환	(B 자원(resource) 관리의 중요성, 인쇄 p.671)
U11-004	메모리를 사용한 뒤 해제하지 않는 경우를 메모리 누수(memory leak)라고 한다. 근거 대조 문구: 메모리누수(memory leak)	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.674)
U11-005	예외로 인해 delete가 있는 문장까지 실행되지 않으면 자원 해제를 놓칠 수 있다. 근거 대조 문구: delete pa 가실행되지않고	(B 자원(resource) 관리의 중요성, 인쇄 p.673-674)
U11-006	RAII(Resource Acquisition Is Initialization)는 자원 관리를 스택에 할당한 객체를 통해 수행하는 방식이다. 근거 대조 문구: 자원관리를스택에할당한객체를통해	(B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)
U11-007	스마트 포인터(smart pointer)는 자신이 소멸할 때 가리키는 자원도 해제하도록 하는 포인터 객체이다. 근거 대조 문구: 자신이소멸될때자신이가리키고있는데이터도같이delete	(B Resource Acquisition Is Initialization - RAII, 인쇄 p.674)
U11-008	double free는 이미 소멸한 객체를 다시 소멸시켜 발생하는 버그이다. 근거 대조 문구: 이미소멸된객체를다시소멸	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)
U11-009	unique_ptr는 특정 객체의 유일한 소유권을 나타내며, 자신이 소멸할 때 가리키던 객체도 소멸한다. 근거 대조 문구: 객체의유일한소유권	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675,682)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-010	unique_ptr는 -> 연산자를 오버로드하여 일반 포인터처럼 사용할 수 있다. 근거 대조 문구: -> 연산자를오버로드	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.676)
U11-011	= delete는 함수 사용을 명시적으로 금지하며, unique_ptr의 복사 생성자는 유일한 소유권을 위해 삭제되어 있다. 근거 대조 문구: 복사생성자가명시적으로삭제	(B 삭제된 함수, 인쇄 p.678)
U11-012	unique_ptr는 이동 생성이 가능하며, std::move로 소유권을 옮기면 원래 포인터는 아무것도 가리키지 않는다. 근거 대조 문구: pa 는아무것도가리키고있지	(B unique_ptr 소유권 이전하기, 인쇄 p.679-680)
U11-013	get은 실제 객체의 주소를 반환하며, 원문은 소유권을 옮기지 않고 잠시 접근할 함수에는 그 주소를 전달한다. 근거 대조 문구: 실제객체의주소값을리턴	(B unique_ptr를 함수 인자로 전달하기, 인쇄 p.682)
U11-014	make_unique는 클래스 생성자에 인자를 완벽하게 전달하여 unique_ptr를 생성한다. 근거 대조 문구: 생성자에인자들에직접완벽한전달	(B unique_ptr를 쉽게 생성하기, 인쇄 p.683)
U11-015	unique_ptr를 vector에 넣는 원문 예는 push_back(std::move(pa)) 또는 내부에서 직접 생성하는 emplace_back을 사용한다. 근거 대조 문구: 명시적으로pa 를vector 안으로이동	(B unique_ptr를 원소로 가지는 컨테이너, 인쇄 p.685-687)
U11-016	shared_ptr는 여러 스마트 포인터가 같은 객체를 소유할 수 있게 하며, 참조 개수가 0이 되면 객체를 해제한다. 근거 대조 문구: 참조개수가0 이되어야	(B shared_ptr, 인쇄 p.688-689)
U11-017	use_count는 현재 shared_ptr의 참조 개수를 알려준다. 근거 대조 문구: use_count 함수를통해	(B shared_ptr, 인쇄 p.691)
U11-018	shared_ptr들은 동적으로 할당한 제어 블록(control block)을 공유하여 참조 개수 등의 정보를 공유한다. 근거 대조 문구: 제어블록(control block)	(B shared_ptr, 인쇄 p.692)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-019	make_shared는 객체와 shared_ptr의 제어 블록을 한 번에 동적 할당하여 shared_ptr를 반환한다. 근거 대조 문구: 제어블록까지한번에동적할당	(B make_shared로 생성하자, 인쇄 p.693)
U11-020	같은 일반 포인터 주소로 shared_ptr를 각각 생성하면 제어 블록이 따로 만들어져 같은 객체를 중복 해제할 수 있다. 근거 대조 문구: 두개의제어블록이따로생성	(B shared_ptr 생성 시 주의할 점, 인쇄 p.693-695)
U11-021	enable_shared_from_this를 상속한 클래스의 shared_from_this는 이미 있는 제어 블록을 사용하며, 그 객체를 가리키는 shared_ptr가 먼저 있어야 한다. 근거 대조 문구: 이미정의되어있는제어블록	(B enable_shared_from_this, 인쇄 p.696-697)
U11-022	shared_ptr가 서로를 소유하는 순환 참조에서는 참조 개수가 0이 되지 않아 소멸자가 호출되지 않는 문제가 생긴다. 근거 대조 문구: 순환참조	(B 서로 참조하는 shared_ptr, 인쇄 p.698-699)
U11-023	weak_ptr는 객체를 참조하지만 shared_ptr의 참조 개수를 늘리지 않아 순환 참조 문제를 해결하는 데 사용한다. 근거 대조 문구: 참조개수를늘리지는않습니다.	(B weak_ptr, 인쇄 p.699-700)
U11-024	weak_ptr는 shared_ptr 또는 다른 weak_ptr로 생성하며 일반 포인터 주소로 만들 수 없다. 근거 대조 문구: 평범한포인터주소값으로weak_ptr 를생성할수는없습니다.	(B weak_ptr, 인쇄 p.702)
U11-025	weak_ptr의 lock은 객체가 살아 있으면 그 객체의 shared_ptr, 이미 해제되었으면 빈 shared_ptr를 반환한다. 근거 대조 문구: 아무것도가리키지않는shared_ptr	(B weak_ptr, 인쇄 p.702)
U11-026	제어 블록은 weak_ptr도 더 이상 가리키지 않을 때 해제할 수 있으므로 참조 개수와 약한 참조 개수(weak count)를 기록한다. 근거 대조 문구: 약한참조개수(weak count)	(B weak_ptr, 인쇄 p.703)
U11-027	Callable은 ()를 붙여 호출할 수 있는 모든 것을 뜻하며, 함수·함수 객체·람다가 예로 나온다. 근거 대조 문구: () 를붙여서호출할수있는모든것	(B Callable, 인쇄 p.705-706)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-028	std::function은 Callable을 객체 형태로 보관하며, 템플릿 인자로 반환형과 함수 인자를 나타내는 함수 타입을 받는다. 근거 대조 문구: 리턴값과함수의인자들	(B std::function, 인쇄 p.706-707)
U11-029	멤버 함수를 std::function으로 보관할 때 원래 인자에 호출할 객체를 받는 인자를 더하고 &로 멤버 함수 주소를 명시한다. 근거 대조 문구: 추가적으로객체를받는인자	(B 멤버 함수를 가지는 std::function, 인쇄 p.709-710)
U11-030	mem_fn은 전달한 멤버 함수를 함수 객체로 만들어 반환하며, 원문은 transform에 사용한다. 근거 대조 문구: 전달된멤버함수를function 객체로	(B 멤버 함수들을 함수 객체로 - mem_fn, 인쇄 p.714)
U11-031	std::bind는 원래 함수의 인자를 특정 값으로 붙이고, placeholders로 새 함수 객체의 인자를 전달하거나 순서를 바꾼다. 근거 대조 문구: 첫번째인자와두번째인자의순서를바꿔	(B std::bind, 인쇄 p.715-716)
U11-032	bind에 전달한 값은 복사되므로 참조 전달이 필요하면 std::ref를 사용하고, const 레퍼런스에는 std::cref를 사용한다. 근거 대조 문구: const 레퍼런스의경우cref	(B std::bind, 인쇄 p.717-718)
U11-033	unique_ptr가 객체의 유일한 소유권을 갖게 한다. 근거 대조 문구: 유일한소유권	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)
U11-034	원문은 std::move로 다른 unique_ptr에 이동 생성한다. 근거 대조 문구: std::unique_ptr<A> pb = std::move(pa);	(B unique_ptr 소유권 이전하기, 인쇄 p.679)
U11-035	이전 후 원래 unique_ptr는 nullptr이므로 그 객체에 접근하지 않는다. 근거 대조 문구: 이동시킨이후에기존의unique_ptr 을접근하지	(B unique_ptr 소유권 이전하기, 인쇄 p.680)
U11-036	lock으로 shared_ptr를 얻는다. 근거 대조 문구: other.lock()	(B weak_ptr, 인쇄 p.702)
U11-037	반환값을 if로 확인한다. 근거 대조 문구: false 로형변환	(B weak_ptr, 인쇄 p.702-703)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-038	객체가 살아 있으면 접근하고, 빈 shared_ptr이면 이미 소멸된 것으로 처리한다. 근거 대조 문구: 이미해제가되었다면아무것도가리키지	(B weak_ptr, 인쇄 p.702)

U11-039	weak_ptr의 lock과 확인: 원문 발췌 (원문 코드 발췌)
	void access_other() {
	std::shared_ptr<A> o = other.lock();
	if (o) {
	std::cout << "접근: " << o->name() << std::endl;
	} else {
	std::cout << "이미소멸됨ㅠ" << std::endl;
	}
	}
	근거 대조 문구: void access_other() {
	std::shared_ptr<A> o = other.lock();
	if (o) {
	std::cout << "접근: " << o->name() << std::endl;
	} else {
	std::cout << "이미소멸됨ㅠ" << std::endl;
	}
	}
		(B weak_ptr, 인쇄 p.701)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U11-040	Callable을 std::function에 보관: 원문 발췌 (원문 코드 발췌)
	std::function<int(const std::string&)> f1 = some_func1;
	std::function<void(char)> f2 = S();
	std::function<void()> f3 = []() { std::cout << "Func3 호출! " << std::endl; };
	
	f1("hello");
	f2('c');
	f3();
	근거 대조 문구: std::function<int(const std::string&)> f1 = some_func1;
	std::function<void(char)> f2 = S();
	std::function<void()> f3 = []() { std::cout << "Func3 호출! " << std::endl; };
	
	f1("hello");
	f2('c');
	f3();
		(B std::function, 인쇄 p.707)

U11-041	unique_ptr 근거 대조 문구: unique_ptr	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)
U11-042	특정 객체의 유일한 소유권 근거 대조 문구: 유일한소유권	(B 객체의 유일한 소유권 - unique_ptr, 인쇄 p.675)
U11-043	shared_ptr 근거 대조 문구: shared_ptr	(B shared_ptr, 인쇄 p.688)
U11-044	여러 포인터가 같은 객체를 소유 근거 대조 문구: 다른shared_ptr 역시그객체	(B shared_ptr, 인쇄 p.688)
U11-045	weak_ptr 근거 대조 문구: weak_ptr	(B weak_ptr, 인쇄 p.700)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U11-046	참조 개수를 늘리지 않으며 shared_ptr로 변환하여 접근 근거 대조 문구: 참조개수를늘리지는않습니다.	(B weak_ptr, 인쇄 p.700)

12 쓰레드·동기화·atomic·비동기·ThreadPool

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-001	thread 객체를 생성하면 인자로 받은 함수를 새로운 쓰레드에서 실행한다. 근거 대조 문구: 새로운쓰레드에서실행	(B 16장 / C++ 에서 쓰레드 생성하기, 인쇄 p.729)
U12-002	쓰레드풀 사용자는 쓰레드들을 생성해 두고 수행할 일을 쓰레드풀에 추가한다. 근거 대조 문구: 그냥쓰레드풀에추가	(B 16장 / ThreadPool, 인쇄 p.806)
U12-003	프로세스는 운영체제에서 실행되는 프로그램의 최소 단위이다. 근거 대조 문구: 운영체제에서실행되는프로그램의최소단위	(B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.720)
U12-004	같은 프로세스 안의 쓰레드들은 메모리를 공유한다. 근거 대조 문구: 같은메모리를공유하게됩니다	(B 16장 / 멀티 쓰레드 프로그램, 인쇄 p.722)
U12-005	쓰레드가 CPU 코어에 어떻게 할당되고 언제 컨텍스트 스위치를 할지는 운영체제가 결정한다. 근거 대조 문구: 전적으로운영체제의마음	(B 16장 / C++ 에서 쓰레드 생성하기, 인쇄 p.729)
U12-006	join 은 해당 쓰레드가 실행을 종료하면 리턴한다. 근거 대조 문구: 해당하는쓰레드들이실행을종료하면리턴	(B 16장 / join, 인쇄 p.730)
U12-007	쓰레드를 detach 하면 main 함수에서는 그 쓰레드가 종료될 때까지 기다리지 않는다. 근거 대조 문구: 더이상쓰레드들이종료될때까지기다리지않습니다	(B 16장 / detach, 인쇄 p.733)
U12-008	thread 생성자의 첫 인자는 Callable 이며, 이어서 그 함수에 전달할 인자들을 적는다. 근거 대조 문구: thread 생성자의첫번째인자로함수	(B 16장 / 쓰레드에 인자 전달하기, 인쇄 p.735)
U12-009	this_thread::get_id 로 현재 쓰레드의 아이디를 얻을 수 있다. 근거 대조 문구: 현재내가돌아가고있는쓰레드의아이디	(B 16장 / 쓰레드에 인자 전달하기, 인쇄 p.735)
U12-010	원문의 counter 예제는 mutex 의 lock 과 unlock 사이에서 result += 1 을 수행한다. 근거 대조 문구: m.lock();	(B 16장 / 뮤텍스와 조건 변수, 인쇄 p.744)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-011	뮤텍스의 lock 과 unlock 사이에서 한 쓰레드만 실행할 수 있는 코드 부분을 임계 영역(critical section)이라고 부른다. 근거 대조 문구: 임계영역(critical section)	(B 16장 / 뮤텍스 (mutex), 인쇄 p.745)
U12-012	lock_guard 는 생성자에서 뮤텍스를 lock 하고 소멸할 때 unlock 한다. 근거 대조 문구: 생성자에서뮤텍스를lock	(B 16장 / lock_guard, 인쇄 p.748)
U12-013	원문의 데드락 예제에서는 worker1 이 m1 을, worker2 가 m2 를 잠근 채 서로 상대가 가진 뮤텍스의 해제를 기다려 진행하지 못한다. 근거 대조 문구: 데드락상황	(B 16장 / 데드락 (deadlock), 인쇄 p.749-750)
U12-014	여러 Lock 을 획득해야 한다면 정해진 순서로 획득한다. 근거 대조 문구: 정해진순서로획득	(B 16장 / Lock 들을 언제나 정해진 순서로 획득해라, 인쇄 p.754)
U12-015	생산자(producer)는 처리할 일을 받아오는 쓰레드이고, 소비자 (consumer)는 받은 일을 처리하는 쓰레드이다. 근거 대조 문구: 받은일을처리하는쓰레드	(B 16장 / 생산자와 소비자 패턴, 인쇄 p.754)
U12-016	조건 변수(condition_variable)는 특정 조건이 만족될 때까지 쓰레드를 기다리게 하는 데 사용한다. 근거 대조 문구: 어떠어떠한조건을만족할때까지	(B 16장 / condition_variable, 인쇄 p.760)
U12-017	wait 에 전달한 조건이 거짓이면 unique_lock 을 unlock 한 뒤 sleep 한다. 근거 대조 문구: lk 를unlock 한뒤에	(B 16장 / condition_variable, 인쇄 p.763)
U12-018	unique_lock 은 unlock 후 다시 lock 할 수 있고, cv->wait 는 unique_lock 을 인자로 받는다. 근거 대조 문구: unlock 후에다시lock	(B 16장 / condition_variable, 인쇄 p.764)
U12-019	notify_one 은 기다리는 쓰레드 중 하나를 깨워 조건을 다시 검사하게 한다. 근거 대조 문구: 쓰레드중하나를깨워서조건을다시검사	(B 16장 / condition_variable, 인쇄 p.764)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-020	원문의 생산자·소비자 예제는 생산자들을 join 한 뒤 notify_all 로 잠든 소비자들을 모두 깨운다. 근거 대조 문구: 나머지자고있는쓰레드들을모두깨운다	(B 16장 / condition_variable, 인쇄 p.764)
U12-021	원자적 연산은 중간에 다른 쓰레드가 끼어들 여지가 없으며, 연산을 했거나 하지 않았거나만 존재한다. 근거 대조 문구: 이연산을했다혹은안했다	(B 16장 / 원자성(atomicity), 인쇄 p.774-775)
U12-022	atomic 의 템플릿 인자로 원자적으로 만들고 싶은 타입을 전달한다. 근거 대조 문구: 원자적으로만들고싶은타입	(B 16장 / 원자성(atomicity), 인쇄 p.776)
U12-023	memory_order_relaxed 는 단일 쓰레드 관점의 결과가 같다면 서로 다른 변수의 relaxed 메모리 연산을 재배치할 수 있다. 근거 대조 문구: 서로다른변수의relaxed 메모리	(B 16장 / memory_order_relaxed, 인쇄 p.779)
U12-024	memory_order_release 는 이전 메모리 명령이 뒤로 재배치되는 것을 금지하고, memory_order_acquire 는 뒤의 메모리 명령이 앞으로 재배치되는 것을 금지한다. 근거 대조 문구: 해당명령뒤에오는모든메모리명	(B 16장 / release 와 acquire, 인쇄 p.783)
U12-025	memory_order_seq_cst 를 사용하는 메모리 연산들은 모든 쓰레드에서 동일한 연산 순서로 관찰되며, atomic 의 기본 memory_order 이다. 근거 대조 문구: 모든쓰레드에서동일한연산순서	(B 16장 / memory_order_seq_cst, 인쇄 p.787)
U12-026	프로그램 실행이 여러 갈래로 갈라져 동시에 진행되는 것을 비동기적(asynchronous) 실행이라고 부른다. 근거 대조 문구: 비동기적(asynchronous) 실행	(B 16장 / 비동기 연산을 위한 도구들, 인쇄 p.793)
U12-027	promise 가 future 에 값을 전달하기 전까지 wait 가 기다리며, get 으로 전달된 객체를 얻는다. 근거 대조 문구: promise 가future 에값을전달하기전까지	(B 16장 / std::promise 와 std::future, 인쇄 p.795)
U12-028	future 의 get 은 안의 데이터를 이동하므로 두 번 호출하면 안 된다. 근거 대조 문구: get 을두번호출	(B 16장 / std::promise 와 std::future, 인쇄 p.795)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-029	promise 의 set_exception 으로 전달된 예외는 future 의 get 을 호출할 때 던져진다. 근거 대조 문구: get 함수를호출하였을때	(B 16장 / 예외 전달, 인쇄 p.797)
U12-030	wait_for 은 정해진 시간 동안 기다리며, future_status 는 ready, timeout, deferred 의 세 상태를 갖는다. 근거 대조 문구: future_status 는총3 가지상태	(B 16장 / wait_for, 인쇄 p.797,799)
U12-031	여러 쓰레드에서 future 를 get 해야 하는 경우 shared_future 를 사용한다. 근거 대조 문구: 여러개의다른쓰레드에서future	(B 16장 / shared_future, 인쇄 p.799)
U12-032	packaged_task 는 전달받은 함수를 실행하여 리턴값을 promise 에 설정하고, get_future 로 대응하는 future 를 얻는다. 근거 대조 문구: 그함수의리턴값을promise 에설정	(B 16장 / packaged_task, 인쇄 p.801)
U12-033	std::async 는 실행 결과를 future 로 돌려주며, launch::async 는 바로 새 쓰레드에서 실행하고 launch::deferred 는 future::get 시 새 쓰레드 없이 실행한다. 근거 대조 문구: 새로운 쓰레드를생성하지않음	(B 16장 / std::async, 인쇄 p.803)
U12-034	원문의 jobs_ 는 std::queue<std::function<void()>> 이며, 가장 오래전에 추가된 작업을 먼저 얻기 위해 큐를 사용한다. 근거 대조 문구: std::queue<std::function<void()>> jobs_;	(B 16장 / ThreadPool 만들기, 인쇄 p.808)
U12-035	cv_job_q_ 와 m_job_q_ 는 생산자·소비자 패턴을 구현하고, stop_all 은 Worker 쓰레드의 종료 조건을 나타낸다. 근거 대조 문구: 생산자-소비자패턴을구현	(B 16장 / ThreadPool 만들기, 인쇄 p.808)
U12-036	WorkerThread 는 작업이 생기거나 stop_all 이 설정될 때까지 기다리고, stop_all 이 참이며 큐가 비면 종료한다. 근거 대조 문구: stop_all && this->jobs_.empty()	(B 16장 / ThreadPool 첫 번째 버전, 인쇄 p.809)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-037	EnqueueJob 은 큐에 작업을 넣은 뒤 notify_one 으로 쓰레드 하나를 깨운다. 근거 대조 문구: jobs_.push(std::move(job));	(B 16장 / ThreadPool 첫 번째 버전, 인쇄 p.810)
U12-038	원문의 확장 버전은 packaged_task 를 shared_ptr 에 보관하고 람다에 복사본을 전달해 작업 실행 시점까지 객체를 유지한다. 근거 대조 문구: 람다함수에	(B 16장 / ThreadPool 확장, 인쇄 p.818)
U12-039	EnqueueJob 의 반환형에는 std::result_of 로 구한 함수의 리턴 타입을 가지는 future 를 사용한다. 근거 대조 문구: 함수F 의리턴값은std::result_of	(B 16장 / ThreadPool 확장, 인쇄 p.814)
U12-040	큐 맨 앞의 job 을 꺼내고 pop 한 뒤 lock 을 unlock 한다. 근거 대조 문구: jobs_.pop();	(B 16장 / ThreadPool 첫 번째 버전, 인쇄 p.809)
U12-041	꺼낸 job 을 실행하고 while 루프에서 다음 작업을 기다린다. 근거 대조 문구: job();	(B 16장 / ThreadPool 첫 번째 버전, 인쇄 p.809)

U12-042	lock_guard 를 사용하는 원문 worker 발췌 (원문 코드 발췌)
	void worker(int& result, std::mutex& m) {
	for (int i = 0; i < 10000; i++) {
	std::lock_guard<std::mutex> lock(m);
	result += 1;
	}
	}
	근거 대조 문구: std::lock_guard<std::mutex> lock(m);
		(B 16장 / lock_guard, 인쇄 p.747)

U12-043	쓰기(store) 근거 대조 문구: 쓰기(store)	(B 16장 / memory order 표, 인쇄 p.789)
U12-044	memory_order_relaxed, memory_order_release, memory_order_seq_cst 근거 대조 문구: memory_order_release	(B 16장 / memory order 표, 인쇄 p.789)
U12-045	읽기(load) 근거 대조 문구: 읽기(load)	(B 16장 / memory order 표, 인쇄 p.789)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U12-046	memory_order_relaxed, memory_order_consume, memory_order_acquire, memory_order_seq_cst 근거 대조 문구: memory_order_consume	(B 16장 / memory order 표, 인쇄 p.789)
U12-047	읽고-수정하고-쓰기 근거 대조 문구: read - modify - write	(B 16장 / memory order 표, 인쇄 p.789)
U12-048	memory_order_relaxed, memory_order_consume, memory_order_acquire, memory_order_release, memory_order_acq_rel, memory_order_seq_cst 근거 대조 문구: memory_order_acq_rel	(B 16장 / memory order 표, 인쇄 p.789)
U12-049	shared_future 근거 대조 문구: shared_future	(B 16장 / shared_future, 인쇄 p.799)
U12-050	packaged_task 근거 대조 문구: packaged_task	(B 16장 / packaged_task, 인쇄 p.801)
U12-051	std::async 근거 대조 문구: std::async	(B 16장 / std::async, 인쇄 p.802)

13 유니폼 초기화·constexpr·decltype

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-001	유니폼 초기화는 {} 를 이용한 생성자 호출이며, 인자 없는 생성자가 함수의 정의로 오해되는 일을 막는다. 근거 대조 문구: 유니폼초기화({} 를이용한생성자호출)	(B 17장 / 뭘 배웠지?, 인쇄 p.836)
U13-002	decltype 는 전달된 식의 타입으로 치환된다. 근거 대조 문구: 식의타입으로치환	(B 19장 / decltype, 인쇄 p.852)
U13-003	A a(); 는 객체 a 를 만드는 대신, A 를 리턴하고 인자를 받지 않는 함수 a 로 해석된다. 근거 대조 문구: 인자를받지않는함수	(B 17장 / C++ 유니폼 초기화, 인쇄 p.826)
U13-004	중괄호 초기화는 데이터 손실이 있는 Narrowing 변환을 불허하며, 원문의 A b{3.5}; 는 double 에서 int 로 변환할 수 없어 오류가 난다. 근거 대조 문구: 데이터손실이있는(Narrowing) 변환	(B 17장 / 유니폼 초기화, 인쇄 p.829)
U13-005	함수의 리턴 시 {} 를 사용하면 생성하는 객체의 타입을 다시 명시하지 않아도 된다. 근거 대조 문구: 객체의타입을다시명시하지	(B 17장 / 유니폼 초기화, 인쇄 p.829)
U13-006	map 의 초기화 리스트는 pair<Key, Value> 원소를 받으며, pair 의 첫 원소는 키이고 두 번째 원소는 값이다. 근거 대조 문구: pair 의첫번째원소로키	(B 17장 / initializer_list, 인쇄 p.832)
U13-007	{} 로 객체를 생성할 때 initializer_list 를 받는 생성자가 있으면 오버로딩에서 최우선으로 고려한다. 근거 대조 문구: 최우선으로	(B 17장 / initializer_list 사용시 주의할 점, 인쇄 p.832)
U13-008	원문에서 vector v{10}; 은 10 을 보관한 원소 한 개를 만들고, vector v(10); 은 count 생성자를 호출한다. 근거 대조 문구: 10 을보관하고있는벡터	(B 17장 / initializer_list 사용시 주의할 점, 인쇄 p.832-833)
U13-009	C++ 17 기준 auto x = {arg1, arg2...} 는 인자들이 같은 타입이면 std::initializer_list<T> 로 추론된다. 근거 대조 문구: 모두같은타입이라면	(B 17장 / initializer_list 와 auto, 인쇄 p.835)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-010	C++ 17 기준 auto x {arg1, arg2, ...} 는 인자 한 개이면 그 타입으로 추론하고 여러 개이면 오류가 난다. 근거 대조 문구: 인자가단1 개라면	(B 17장 / initializer_list와 auto, 인쇄 p.835)
U13-011	auto list = {"a", "b", "cc"}; 의 타입은 initializer_list<const char*> 이다. 근거 대조 문구: initializer_list<const char*>	(B 17장 / initializer_list와 auto, 인쇄 p.836)
U13-012	constexpr 은 식이 상수식이라고 명시하는 키워드이며, 객체 정의에 붙이면 그 객체를 상수식에 사용할 수 있다. 근거 대조 문구: 어떠한식이상수식이라고명시	(B 18장 / constexpr, 인쇄 p.838)
U13-013	const 로 정의된 상수는 컴파일 타임에 값을 알 필요가 없지만, constexpr 변수의 초기화에는 다른 상수식이 와야 한다. 근거 대조 문구: 반드시오른쪽에다른상수식	(B 18장 / constexpr vs const, 인쇄 p.838-839)
U13-014	constexpr 함수는 조건이 맞을 때 반환값을 컴파일 타임 상수로 만들 수 있다. 근거 대조 문구: 조건이맞을때	(B 18장 / constexpr 함수, 인쇄 p.842)
U13-015	원문의 Factorial(10) 은 컴파일 타임에 계산되어 클래스 A 의 템플릿 인자로 들어가며 출력은 3628800 이다. 근거 대조 문구: 3628800	(B 18장 / constexpr 함수, 인쇄 p.842)
U13-016	원문은 C++ 14 에서 constexpr 함수의 제약이 완화되었다고 설명하며, goto·리터럴 타입이 아닌 변수·초기화되지 않은 변수 등의 제한을 열거한다. 근거 대조 문구: C++ 14 부터위와같은제약조건들이완화	(B 18장 / constexpr 함수, 인쇄 p.843)
U13-017	리터럴(Literal) 타입은 컴파일러가 컴파일 타임에 정의할 수 있는 타입이다. 근거 대조 문구: 컴파일타임에정의할수있는타입	(B 18장 / 리터럴 타입?, 인쇄 p.845)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-018	constexpr 생성자의 인자는 리터럴 타입이어야 하며, 해당 클래스는 다른 클래스를 가상 상속받을 수 없다. 근거 대조 문구: 다른클래스를가상상속받을수없습니다	(B 18장 / constexpr 생성자, 인쇄 p.845)
U13-019	if constexpr 의 조건은 bool 로 변환될 수 있는 컴파일 타임 상수식이어야 한다. 근거 대조 문구: 컴파일타임상수식이어야	(B 18장 / if constexpr, 인쇄 p.850)
U13-020	if constexpr 이 참이면 else 문은 컴파일되지 않고, 거짓이면 else 부분만 컴파일된다. 근거 대조 문구: 완전히무시	(B 18장 / if constexpr, 인쇄 p.850)
U13-021	괄호로 둘러싸이지 않은 식별자 표현식을 decltype 에 전달하면 해당 식의 타입을 얻는다. 근거 대조 문구: 괄호로둘러쌓이지않은식별자표현식	(B 19장 / decltype, 인쇄 p.853)
U13-022	식별자 표현식이 아닌 경우, 식의 타입이 T 이고 값 카테고리가 xvalue 이면 T&&, lvalue 이면 T&, prvalue 이면 T 가 된다. 근거 대조 문구: decltype 는T&&	(B 19장 / decltype, 인쇄 p.853)
U13-023	모든 C++ 식에는 타입과 값 카테고리(value category)가 따른다. 근거 대조 문구: 식의타입과값카테고리	(B 19장 / Value Category, 인쇄 p.853)
U13-024	정체를 알 수 있는 식들을 glvalue 라고 하며, 이동시킬 수 있는 식들을 rvalue 라고 한다. 근거 대조 문구: 정체를알수있는모든식들을glvalue	(B 19장 / Value Category, 인쇄 p.854)
U13-025	이름을 가진 대부분의 객체는 lvalue 이며, 우측값 레퍼런스 인자 a 도 식 a 의 값 카테고리는 lvalue 이다. 근거 대조 문구: 식a 의 값카테고리는lvalue	(B 19장 / lvalue, 인쇄 p.855)
U13-026	prvalue 는 우측값 레퍼런스와 상수 좌측값 레퍼런스의 초기화에 사용할 수 있다. 근거 대조 문구: 상수좌측값레퍼런스를초기화	(B 19장 / prvalue, 인쇄 p.857)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-027	우측값 레퍼런스를 리턴하는 함수의 호출식은 xvalue 의 대표적인 경우이며 std::move(x) 가 예이다. 근거 대조 문구: 대표적으로std::move(x)	(B 19장 / xvalue, 인쇄 p.857)
U13-028	원문에서 decltype((a)) 는 lvalue 에 대한 타입 추론이므로 int& 가 된다. 근거 대조 문구: int& 로추론	(B 19장 / decltype 의 쓰임새, 인쇄 p.858)
U13-029	원문의 auto j = i; 는 const 를 없애지만, decltype(i) 는 const 를 보존하며 배열 타입도 그대로 전달한다. 근거 대조 문구: decltype 의경우이를그대로보존	(B 19장 / decltype 의 쓰임새, 인쇄 p.858-859)
U13-030	decltype 안의 식은 런타임에 실행되지 않는다. 근거 대조 문구: 런타임시에실행되는것이아닙니다	(B 19장 / std::declval, 인쇄 p.860)
U13-031	std::declval<T>() 는 T 의 생성자를 직접 호출하지 않고 타입 연산에 객체를 나타낼 수 있으며, 런타임에서 사용하면 오류가 난다. 근거 대조 문구: 실제로런타임에사용하면오류	(B 19장 / std::declval, 인쇄 p.863)

U13-032	원문 constexpr 함수 발췌 (원문 코드 발췌)
	constexpr int Factorial(int n) {
	int total = 1;
	for (int i = 1; i <= n; i++) {
	total *= i;
	}
	return total;
	}
	근거 대조 문구: constexpr int Factorial(int n)
		(B 18장 / constexpr 함수, 인쇄 p.842)

U13-033	xvalue 근거 대조 문구: xvalue	(B 19장 / decltype, 인쇄 p.853)
U13-034	T&& 근거 대조 문구: T&&	(B 19장 / decltype, 인쇄 p.853)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U13-035	lvalue 근거 대조 문구: lvalue	(B 19장 / decltype, 인쇄 p.853)
U13-036	T& 근거 대조 문구: T&	(B 19장 / decltype, 인쇄 p.853)
U13-037	prvalue 근거 대조 문구: prvalue	(B 19장 / decltype, 인쇄 p.853)
U13-038	T 근거 대조 문구: T 가됩니다	(B 19장 / decltype, 인쇄 p.853)

14 type_traits·정규표현식·난수·시간·파일시스템·유틸리티

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-001	type_traits 는 타입에 대한 여러 연산을 수행하는 메타 함수들을 제공한다. 근거 대조 문구: 타입들에대해서여러가지	(B 20장 / is_void, 인쇄 p.870)
U14-002	fstream 은 파일 데이터의 입출력을 담당하고, filesystem 은 파일 메타데이타에 대한 접근을 돕는다. 근거 대조 문구: 파일메타데이타	(B 20장 / 파일시스템 라이브러리 소개, 인쇄 p.921)
U14-003	템플릿 메타 함수는 함수처럼 동작하는 템플릿 클래스이며 타입에 대한 연산을 수행한다. 근거 대조 문구: 타입에대해연산	(B 20장 / 템플릿 메타 함수, 인쇄 p.867)
U14-004	is_void<void> 는 특수화된 템플릿으로 value 가 true 이고, 다른 타입은 false 이다. 근거 대조 문구: value 가false	(B 20장 / is_void, 인쇄 p.870)
U14-005	is_integral 은 정수 타입인지 확인하며, static_assert 는 전달한 식의 참·거짓을 컴파일 타임에 확인한다. 근거 대조 문구: 컴파일타임에확인	(B 20장 / static_assert, 인쇄 p.870-871)
U14-006	is_class 는 전달받은 타입이 클래스인지 확인하는 메타 함수이다. 근거 대조 문구: 클래스인지아닌지확인	(B 20장 / is_class, 인쇄 p.872)
U14-007	SFINAE 는 Substitution Failure Is Not An Error 이며, 문법에 맞지 않는 템플릿 인자 치환 결과를 함수 오버로딩 후보에서 제외한다. 근거 대조 문구: Substitution Failure Is Not An Error	(B 20장 / SFINAE, 인쇄 p.878)
U14-008	원문은 SFINAE 에서 함수의 인자와 리턴 타입을 확인하며, 함수 내부 전체를 확인하지는 않는다고 설명한다. 근거 대조 문구: 함수의인자들과리턴타입만	(B 20장 / SFINAE, 인쇄 p.878)
U14-009	enable_if 는 참으로 특수화된 버전에 type 을 정의하고, 나머지에는 type 을 정의하지 않는 방식으로 후보를 제한한다. 근거 대조 문구: type 이정의되어있고	(B 20장 / enable_if, 인쇄 p.882)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-010	정규표현식은 문자열에서 패턴을 찾아 규칙 확인, 검색, 치환에 사용한다. 근거 대조 문구: 문자열에서패턴을찾는데	(B 20장 / 정규표현식 라이브러리 소개, 인쇄 p.894)
U14-011	regex_match 는 문자열 전체가 정규표현식의 패턴에 부합하면 true 를 리턴한다. 근거 대조 문구: 문자열전체가	(B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)
U14-012	부분 매칭 결과는 match[i].str() 로 접근하며, regex_match 의 첫 결과는 전체 문자열이고 그 뒤는 () 안의 문자열이다. 근거 대조 문구: 첫번째결과에전체문자	(B 20장 / 부분 매칭 뽑아내기, 인쇄 p.899)
U14-013	match.suffix() 는 검색된 패턴 바로 뒤부터 원래 문자열 끝까지를 나타내므로 그 뒤에서 다시 검색할 수 있다. 근거 대조 문구: 검색된패턴바로뒤부터	(B 20장 / 원하는 패턴 검색하기, 인쇄 p.901)
U14-014	regex_iterator 는 생성될 때와 ++ 될 때 regex_search 를 실행하고, 역참조하면 현재 패턴의 match_results 를 얻는다. 근거 대조 문구: ++ 연산자로증감될때마다regex_search	(B 20장 / std::regex_iterator, 인쇄 p.903)
U14-015	regex_replace 는 패턴에 맞는 문자열을 다른 문자열로 치환한다. 근거 대조 문구: 다른문자열로치환	(B 20장 / 원하는 패턴 치환하기, 인쇄 p.903)
U14-016	치환 문자열의 $1 은 첫 번째 캡쳐 그룹을 나타내며, 중첩된 그룹의 번호는 괄호가 열리는 순서로 정해진다. 근거 대조 문구: 괄호가열리는순서대로	(B 20장 / 캡쳐 그룹, 인쇄 p.904,906)
U14-017	의사 난수(pseudo random number)는 시드(seed)를 기반으로 난수처럼 보이는 수열을 생성하는 방식이다. 근거 대조 문구: 의사난수(pseudo random number)	(B 20장 / 난수 생성, 인쇄 p.910)
U14-018	원문은 random_device 로 시드값을 얻고, 그 이후의 난수열은 난수 엔진으로 생성하도록 설명한다. 근거 대조 문구: 시드값처럼난수엔진을초기화	(B 20장 / random_device, 인쇄 p.912)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-019	std::mt19937 은 메르센 트위스터 알고리즘을 사용하는 난수 생성 엔진이다. 근거 대조 문구: 메르센 트위스터	(B 20장 / 난수 엔진, 인쇄 p.912)
U14-020	uniform_int_distribution<int> 의 생성자에 범위를 지정하고, 그 분포에 난수 엔진을 전달하여 샘플을 뽑는다. 근거 대조 문구: 생성자에원하는범위	(B 20장 / 분포, 인쇄 p.913)
U14-021	정규분포 예제는 std::normal_distribution<double> dist(0, 1) 로 평균 0, 표준편차 1 을 지정한다. 근거 대조 문구: 평균= */ 0	(B 20장 / 정규분포, 인쇄 p.913)
U14-022	clock 의 now() 는 해당 clock 에 맞는 time_point 객체를 리턴한다. 근거 대조 문구: time_point 객체를리턴	(B 20장 / chrono 에서 지원하는 clock 들, 인쇄 p.917)
U14-023	원문의 end - start 는 duration 객체를 얻고, count() 는 시간 차이 동안의 틱 횟수를 알려준다. 근거 대조 문구: 시간차이동안몇번의틱	(B 20장 / duration, 인쇄 p.917)
U14-024	실제 시간 단위로 변환하기 위해 duration_cast 를 사용한다. 근거 대조 문구: duration_cast 를사용	(B 20장 / duration_cast, 인쇄 p.917)
U14-025	날짜 출력 예제는 system_clock::now, to_time_t, localtime, put_time 의 순서로 변환한다. 근거 대조 문구: std::put_time	(B 20장 / 현재 시간을 날짜로, 인쇄 p.919)
U14-026	path 는 파일과 폴더의 경로를 다루는 객체이다. 근거 대조 문구: 파일과폴더의경로	(B 20장 / 뭘 배웠지?, 인쇄 p.937)
U14-027	exists, is_regular_file, is_directory 는 각각 경로의 존재, 파일 여부, 디렉토리 여부를 확인한다. 근거 대조 문구: std::filesystem::is_regular_file	(B 20장 / 파일을 찾아보자, 인쇄 p.921)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-028	current_path() 는 프로그램 실행 경로를 리턴하며, 상대경로는 이 경로를 기준으로 구한다. 근거 대조 문구: 프로그램의현재실행경로	(B 20장 / 경로, 인쇄 p.925)
U14-029	directory_iterator 로 디렉토리 안의 파일을 살피고, recursive_directory_iterator 로 안의 모든 파일·폴더를 탐색한다. 근거 대조 문구: recursive_directory_iterator	(B 20장 / 뭘 배웠지?, 인쇄 p.937)
U14-030	create_directory 는 디렉토리를 생성하고 copy 와 remove 는 파일·폴더를 복사·삭제한다. 근거 대조 문구: copy, remove	(B 20장 / 뭘 배웠지?, 인쇄 p.937)
U14-031	std::optional 은 원하는 값을 보관할 수도, 보관하지 않을 수도 있는 클래스이다. 근거 대조 문구: 원하는값을보관할수도	(B 20장 / std::optional, 인쇄 p.940)
U14-032	optional 의 has_value() 로 값의 존재를 확인하고, value() 또는 * 로 값에 접근한다. 근거 대조 문구: has_value 함수	(B 20장 / std::optional, 인쇄 p.942)
U14-033	값이 없는 optional 은 보관할 타입의 디폴트 객체를 가질 필요가 없다. 근거 대조 문구: 디폴트객체를가질필요가없다	(B 20장 / std::optional, 인쇄 p.942)
U14-034	std::optional<std::reference_wrapper<A>> 로 A 객체의 레퍼런스를 보관할 수 있으며, std::ref 로 wrapper 를 만들고 get() 으로 참조한 객체를 얻는다. 근거 대조 문구: std::reference_wrapper	(B 20장 / 레퍼런스를 가지는 std::optional, 인쇄 p.945)
U14-035	std::variant 는 컴파일 타임에 정한 여러 타입 중 한 타입의 객체를 보관하는 one-of 클래스이다. 근거 대조 문구: one-of	(B 20장 / std::variant, 인쇄 p.946)
U14-036	std::monostate 를 사용해 variant 가 아무것도 들고 있지 않은 상태를 표현할 수 있다. 근거 대조 문구: 아무것도들고있지않은상태	(B 20장 / std::monostate, 인쇄 p.949)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-037	std::tuple 은 서로 다른 타입의 객체들을 묶으며, make_tuple 로 만들고 std::get 으로 원소에 접근한다. 근거 대조 문구: std::make_tuple	(B 20장 / std::tuple, 인쇄 p.950-951)
U14-038	structured binding 은 auto [age, name, is_male] = student; 처럼 tuple 의 원소들을 받으며, 모든 원소를 받아야 한다. 근거 대조 문구: 모든원소들을반드시받아야	(B 20장 / Structured binding, 인쇄 p.953)
U14-039	structured binding 에서 auto& 를 사용하면 복사 대신 레퍼런스를 취할 수 있으며, 구조체나 pair 에도 사용할 수 있다. 근거 대조 문구: 그냥레퍼런스만	(B 20장 / Structured binding, 인쇄 p.953-954)
U14-040	system_clock::now() 로 현재 time_point 를 얻는다. 근거 대조 문구: system_clock::now()	(B 20장 / 현재 시간을 날짜로, 인쇄 p.919)
U14-041	system_clock::to_time_t 로 time_t 로 변환한다. 근거 대조 문구: to_time_t(now)	(B 20장 / 현재 시간을 날짜로, 인쇄 p.919)
U14-042	localtime 으로 tm 객체를 얻고 put_time 으로 출력 형식을 만든다. 근거 대조 문구: std::localtime(&t)	(B 20장 / 현재 시간을 날짜로, 인쇄 p.919)
U14-043	원문 정규분포 초기화 발췌 (원문 코드 발췌) std::random_device rd; std::mt19937 gen(rd()); std::normal_distribution<double> dist(/* 평균= */ 0, /* 표준편차= */ 1); 근거 대조 문구: std::normal_distribution<double> dist	(B 20장 / 정규분포, 인쇄 p.913)
U14-044	regex_match 근거 대조 문구: regex_match	(B 20장 / 전체 문자열 매칭하기, 인쇄 p.896)
U14-045	regex_iterator 근거 대조 문구: regex_iterator	(B 20장 / std::regex_iterator, 인쇄 p.903)
U14-046	regex_replace 근거 대조 문구: regex_replace	(B 20장 / 원하는 패턴 치환하기, 인쇄 p.903)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U14-047	optional 근거 대조 문구: std::optional	(B 20장 / std::optional, 인쇄 p.940)
U14-048	variant 근거 대조 문구: std::variant	(B 20장 / std::variant, 인쇄 p.946)
U14-049	tuple 근거 대조 문구: std::tuple	(B 20장 / std::tuple, 인쇄 p.950)

15 Visual Studio 디버깅·Make

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-001	중단점은 임의의 위치에서 프로그램을 정지시키는 기능이다. 근거 대조 문구: 임의의위치에서프로그램을정지	(L02D Debugging Tool 1 - Break point, 슬라이드 5)
U15-002	make 는 주어진 쉘 명령어를 조건에 맞게 실행하며, 조건을 담은 파일을 Makefile 이라고 한다. 근거 대조 문구: 조건에맞게실행	(B 22장 / make, 인쇄 p.964)
U15-003	디버깅은 버그(오류)를 제거하는 것이다. 근거 대조 문구: 버그(=오류)를제거	(L02D What is debugging?, 슬라이드 3)
U15-004	강의 자료는 디버깅 가능한 Debug 구성을 확인하며, Release 모드에서는 값을 제대로 볼 수 없다고 설명한다. 근거 대조 문구: Release 모드에서는값을제대로볼수없음	(L02D Preface, 슬라이드 4)
U15-005	중단할 위치의 왼쪽 끝 클릭 또는 F9 으로 중단점을 만들고, Debug 구성에서 Ctrl+F5 가 아닌 F5 로 빌드 후 디버깅을 실행한다. 근거 대조 문구: F9을통해중단점생성	(L02D Debugging Tool 1 - Break point, 슬라이드 5)
U15-006	노란 화살표는 현재 중단된 위치를 표시하며, 그 명령문은 아직 실행되지 않은 상태이다. 근거 대조 문구: 해당명령문은아직실행되지않은상태	(L02D Debugging Tool 1 - Break point, 슬라이드 5)
U15-007	F10 은 명령문을 실행하고 다음 줄로 진행하는 step over 이고, F11 은 호출되는 추가 코드로 들어가는 step into 이다. 근거 대조 문구: step over	(L02D Debugging Tool 1 - Break point, 슬라이드 6)
U15-008	F5 는 다음 중단점까지 진행하고 Shift+F11 은 상위 stack 으로 나가는 step out 이다. 근거 대조 문구: step out	(L02D Debugging Tool 1 - Break point, 슬라이드 6)
U15-009	변수 위에 마우스를 올리거나 하단 창에서 값을 확인하며, 방금 명령줄로 값이 변한 변수는 빨간색으로 표시된다. 근거 대조 문구: 값이변한변수가빨간색	(L02D Debugging Tool 1 - Break point, 슬라이드 7)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-010	메모리 윈도우는 변수가 메모리에 저장된 모습을 살피며, 주소 부분에 &변수이름 을 입력한다. 근거 대조 문구: 주소부분에	(L02D Debugging Tool 2 - Memory Window, 슬라이드 8-9)
U15-011	디버깅에서 의도한 명령 실행 순서와 메모리 데이터가 맞는지 확인한다. 근거 대조 문구: 내가생각하는순서대로	(L02D How to debugging?, 슬라이드 10)
U15-012	Conditional & Action breakpoint 는 특정 조건에서 중단점을 활성화할 수 있고 재컴파일이 필요 없다. 근거 대조 문구: 재컴파일이필요없음	(L02D Conditional & Action breakpoint, 슬라이드 12)
U15-013	디스어셈블리 창은 변환된 어셈블리 코드를, 레지스터 창은 현재 레지스터에 저장된 값을 보여준다. 근거 대조 문구: 레지스터에저장된값	(L02D Disassembly & Register, 슬라이드 13)
U15-014	컴파일은 소스코드를 컴퓨터가 이해할 어셈블리어로 변환하는 과정이다. 근거 대조 문구: 어셈블리어로변환	(B 22장 / 컴파일(Compile), 인쇄 p.960)
U15-015	링킹은 서로 다른 파일에 흩어진 함수나 클래스들을 묶어 연결하는 과정이다. 근거 대조 문구: 한데묶어서링크	(B 22장 / 링킹(Linking), 인쇄 p.962)
U15-016	Makefile 은 target, recipe, prerequisites 의 세 요소로 구성된다. 근거 대조 문구: 3 가지요소	(B 22장 / make, 인쇄 p.964)
U15-017	target 은 make 할 대상으로 전달하는 이름이며, recipe 는 그 타겟을 만들 때 실행할 명령어의 나열이다. 근거 대조 문구: 명령어들의나열	(B 22장 / target·recipes, 인쇄 p.964)
U15-018	recipe 의 명령어는 반드시 탭으로 한 번 들여쓴다. 근거 대조 문구: 반드시탭한번	(B 22장 / 실행할 명령어(recipes), 인쇄 p.964)
U15-019	prerequisites 는 타겟에 사용할 파일의 목록이며 의존 파일(dependency)이라고도 한다. 근거 대조 문구: 의존파일(dependency)	(B 22장 / 필요조건들(prerequisites), 인쇄 p.964)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-020	타겟이 의존 파일들의 수정 시점보다 나중에 수정되었다면 그 타겟의 명령어를 실행하지 않는다. 근거 대조 문구: 명령어를실행하지않습니다	(B 22장 / 필요조건들 (prerequisites), 인쇄 p.964)
U15-021	변수는 $(CC) 와 같이 $() 안에 이름을 써서 사용하며, 정의하지 않은 변수는 빈 문자열로 치환된다. 근거 대조 문구: 빈문자열로치환	(B 22장 / 변수, 인쇄 p.966-967)
U15-022	원문의 = 는 참조 변수가 나중에 정의될 때까지 결정을 미루고, := 는 정의하는 시점의 값만 확인한다. 근거 대조 문구: 해당시점에의변수의값만	(B 22장 / 변수를 정의하는 두 가지 방법, 인쇄 p.967)
U15-023	.PHONY: clean 으로 등록하면 clean 파일의 존재와 관계없이 타겟의 명령을 실행한다. 근거 대조 문구: clean 파일의유무와상관없이	(B 22장 / PHONY, 인쇄 p.969)
U15-024	%.o: %.cc %.h 패턴은 대응하는 파일의 규칙을 나타내며, 패턴은 target 과 prerequisite 부분에 사용한다. 근거 대조 문구: 패턴은타겟과prerequisite 부분에만	(B 22장 / 패턴 사용하기, 인쇄 p.970)
U15-025	$@ 는 타겟 이름, $< 는 첫 의존 파일, $^ 는 의존 파일 목록 전체에 대응한다. 근거 대조 문구: $@ : 타겟이름	(B 22장 / 자동 변수, 인쇄 p.971)
U15-026	$? 는 타겟보다 최신인 의존 파일이며, $+ 는 중복된 파일 이름도 포함한 의존 파일 목록이다. 근거 대조 문구: 중복된파일이름들까지	(B 22장 / 자동 변수, 인쇄 p.971)
U15-027	-MD 옵션으로 컴파일하면 의존 파일 목록을 담은 .d 파일이 생성된다. 근거 대조 문구: -MD 옵션	(B 22장 / 자동으로 prerequisite 만들기, 인쇄 p.971-972)
U15-028	include main.d 는 해당 파일 내용을 포함하고, -include 는 포함할 파일이 없어도 make 메시지를 출력하지 않는다. 근거 대조 문구: 파일이존재하지않아도	(B 22장 / 자동으로 prerequisite 만들기, 인쇄 p.972-973)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-029	patsubst 는 $(patsubst 패턴,치환후형태,변수) 꼴로 패턴을 치환한다. 근거 대조 문구: patsubst 패턴,치환후형태,변수	(B 22장 / 최종 정리, 인쇄 p.976)
U15-030	원문의 분리된 헤더 파일 경로 예제는 컴파일러 옵션에 -Iinclude/ 를 추가한다. 근거 대조 문구: -Iinclude/	(B 22장 / 헤더 파일들을 따로 뽑는 경우, 인쇄 p.978)
U15-031	원문은 make 의 -j 뒤에 사용할 쓰레드 수를 적고 make -j8 을 예로 든다. 근거 대조 문구: make -j8	(B 22장 / 멀티코어를 활용해서 Make 속도를 올리자, 인쇄 p.978)
U15-032	Makefile 에서 main 타겟을 찾고 의존 파일 foo.o, bar.o, main.o 의 타겟을 찾는다. 근거 대조 문구: 타겟이main	(B 22장 / Makefile 처리 순서, 인쇄 p.965)
U15-033	foo.o 가 없으므로 컴파일 명령을 실행하고, bar.o 와 main.o 도 컴파일한다. 근거 대조 문구: 마찬가지로bar.o, main.o 도컴파일	(B 22장 / Makefile 처리 순서, 인쇄 p.965-966)
U15-034	마지막으로 g++ foo.o bar.o main.o -o main 을 실행한다. 근거 대조 문구: g++ foo.o bar.o main.o -o main	(B 22장 / Makefile 처리 순서, 인쇄 p.966)
U15-035	원문 Makefile 일부 발췌 (원문 코드 발췌) foo.o : foo.h foo.cc g++ -c foo.cc bar.o : bar.h bar.cc g++ -c bar.cc 근거 대조 문구: foo.o : foo.h foo.cc	(B 22장 / Makefile, 인쇄 p.965)
U15-036	원문 주의사항에 따라 recipe 앞의 들여쓰기는 탭이다. 근거 대조 문구: 탭으로바꾸어서	(B 22장 / 주의사항, 인쇄 p.965)
U15-037	F10 근거 대조 문구: F10	(L02D Break point, 슬라이드 6)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U15-038	step over: 해당 명령문 실행 후 다음 명령줄로 진행 근거 대조 문구: 다음명령줄로진행	(L02D Break point, 슬라이드 6)
U15-039	F11 근거 대조 문구: F11	(L02D Break point, 슬라이드 6)
U15-040	step into: 명령문에서 호출하는 추가 코드로 들어감 근거 대조 문구: 추가코드	(L02D Break point, 슬라이드 6)
U15-041	Shift+F11 근거 대조 문구: Shift+F11	(L02D Break point, 슬라이드 6)
U15-042	step out: 상위 stack 으로 나가기 근거 대조 문구: 상위stack으로나가기	(L02D Break point, 슬라이드 6)

16 리스트, 스택, 큐

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-001	프로그램에 적합한 자료 구조를 사용하는 것이 얼마나 중요한지 설명한다. 근거 대조 문구: 응용 프로그램에 적합한 자료 구조를 사용하는 것이 얼마나 중요한지	(K 1장 학습 목표, PDF p.11)
U16-002	C++ 내장 자료 구조와 사용자 정의 선형 자료 구조를 다룬다. 근거 대조 문구: 적절한 사용자 정의 선형 자료 구조를 구현	(K 1장 학습 목표, PDF p.11)
U16-003	시간 복잡도는 특정 작업을 수행하는 데 걸리는 시간을 데이터 크기에 대한 수식으로 표현하는 방식이다. 근거 대조 문구: 시간 복잡도는 특정 작업을 수행하는 데 걸리는 시간을 데이터 크기에 대한 수식으로 표현	(K 1.2 연속된 자료 구조와 연결된 자료 구조, PDF p.12)
U16-004	연속된 자료 구조는 모든 원소를 단일 메모리 청크에 저장한다. 근거 대조 문구: 모든 원소를 단일 메모리 청크(chunk)에 저장	(K 1.2.1 연속된 자료 구조, PDF p.12)
U16-005	연속된 자료 구조에서 i번째 원소의 주소는 BA + i * sizeof(type)으로 계산하며 접근 시간은 O(1)이다. 근거 대조 문구: i번째 원소에 접근하려면 BA + i * sizeof(type) 수식을 사용	(K 1.2.1 연속된 자료 구조, PDF p.13)
U16-006	연결된 자료 구조는 노드라고 하는 여러 메모리 청크에 데이터를 저장한다. 근거 대조 문구: 노드(node)라고 하는 여러 개의 메모리 청크에 데이터를 저장	(K 1.2.2 연결된 자료 구조, PDF p.14)
U16-007	연결 리스트의 노드는 데이터와 다음 노드를 가리키는 next 포인터를 가지며 마지막 노드의 next는 NULL이다. 근거 대조 문구: 맨 마지막 노드에서는 ... NULL을 가집니다.	(K 1.2.2 연결된 자료 구조, PDF p.14)
U16-008	연결 리스트의 원소에 접근하려면 헤드부터 next를 따라 이동하므로 원소 접근은 O(n)이다. 근거 대조 문구: 시간 복잡도로 표현하면 O(n)	(K 1.2.2 연결된 자료 구조, PDF p.14)
U16-009	C 스타일 배열은 메모리 할당·해제, 범위 검사, 깊은 복사에서 제약이 있다. 근거 대조 문구: 메모리 할당과 해제를 수동으로 처리해야 합니다.	(K 1.2.4 C 스타일 배열의 제약 사항, PDF p.16-17)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-010	std::array는 원소 타입과 배열 크기를 매개변수로 사용하는 클래스 템플릿이며 메모리를 자동으로 할당하고 해제한다. 근거 대조 문구: 원소의 타입과 배열 크기를 매개변수로 사용하는 클래스 템플릿	(K 1.3 std::array, PDF p.17)
U16-011	std::vector는 고정 크기 문제를 해결하며 초기화할 때 데이터 크기를 제공하지 않아도 된다. 근거 대조 문구: 초기화 과정에 데이터의 크기를 제공하지 않아도 됩니다.	(K 1.4.1 std::vector - 가변 크기 배열, PDF p.29)
U16-012	벡터의 크기는 실제로 저장된 원소 개수이며 용량(capacity)과 다른 의미이다. 근거 대조 문구: 크기는 벡터에 실제로 저장된 원소 개수	(K 1.4.1 std::vector - 가변 크기 배열, PDF p.29)
U16-013	push_back()은 맨 뒤에 원소를 추가하고 insert()는 반복자로 지정한 위치에 원소를 추가한다. 근거 대조 문구: 삽입할 위치를 나타내는 반복자를 첫 번째 인자로	(K 1.4.1 std::vector - 가변 크기 배열, PDF p.30)
U16-014	push_back()은 남은 공간이 있으면 O(1), 공간이 부족하여 전체 원소를 복사·이동하면 O(n)이며 평균 시간 복잡도는 O(1)에 가깝다. 근거 대조 문구: 평균 시간 복잡도는 O(1)에 가깝습니다.	(K 1.4.1 std::vector - 가변 크기 배열, PDF p.30)
U16-015	pop_back()은 O(1), erase()는 뒤쪽 원소를 앞으로 이동해야 하므로 O(n)이다. 근거 대조 문구: 뒤쪽의 원소들을 모두 앞으로 이동해야 하기 때문에 O(n)	(K 1.4.1 std::vector - 가변 크기 배열, PDF p.32)
U16-016	reserve(capacity)는 벡터에서 사용할 용량을 지정하며 크기를 변경하지 않는다. 근거 대조 문구: 이 함수는 벡터의 크기를 변경하지는 않습니다.	(K 1.4.1 std::vector - 가변 크기 배열, PDF p.32)
U16-017	사용자 정의 할당자는 allocate(), deallocate(), construct(), destroy() 등의 함수를 제공해야 한다. 근거 대조 문구: allocate(), deallocate(), construct(), destroy()	(K 1.4.2 std::vector 할당자, PDF p.32)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-018	std::forward_list는 기본적인 연결 리스트의 성능을 유지하면서 추가 기능을 제공하며, 전체 크기 반환과 마지막 원소 직접 접근은 제공하지 않는다. 근거 대조 문구: 전체 리스트의 크기를 반환하거나 ... 직접 접근하는 기능을 제공하지 않습니다.	(K 1.5 std::forward_list, PDF p.33)
U16-019	std::forward_list는 push_front()와 insert_after()로 원소를 삽입하며 이들 삽입 함수의 시간 복잡도는 O(1)이다. 근거 대조 문구: 시간 복잡도는 O(1)	(K 1.5.1 std::forward_list에서 원소 삽입과 삭제, PDF p.34)
U16-020	unique()는 인접한 원소끼리 비교하므로 리스트 전체에서 유일한 원소만 남기려면 먼저 정렬한 후 호출한다. 근거 대조 문구: 먼저 리스트를 정렬한 후 unique() 함수를 사용	(K 1.5.2 std::forward_list의 기타 멤버 함수, PDF p.39)
U16-021	반복자는 포인터와 비슷하지만 STL 컨테이너에 대해 공통의 인터페이스를 제공한다. 근거 대조 문구: STL 컨테이너에 대해 공통의 인터페이스	(K 1.6 반복자, PDF p.39)
U16-022	배열·벡터에는 임의 접근 반복자, std::forward_list에는 증가 연산만 가능한 순방향 반복자가 제공된다. 근거 대조 문구: 이러한 반복자를 순방향 반복자(forward iterator)	(K 1.6 반복자, PDF p.40)
U16-023	std::list는 prev와 next를 가지는 이중 연결 리스트이며 빠른 push_back()과 size()를 지원한다. 근거 대조 문구: 이중 연결 리스트(doubly-linked list) 구조	(K 1.7 std::list, PDF p.48)
U16-024	벡터의 메모리 재할당은 기존 반복자·포인터·참조를 무효화한다. 근거 대조 문구: 모든 반복자와 포인터, 참조는 모두 무효화	(K 1.7.4 반복자 무효화, PDF p.51)
U16-025	std::list와 std::forward_list의 삽입은 반복자의 유효성에 영향을 미치지 않으며 삭제 시에는 삭제된 원소의 반복자가 무효화된다. 근거 대조 문구: 삭제되는 원소를 가리키던 반복자는 당연히 무효화	(K 1.7.4 반복자 무효화, PDF p.51)
U16-026	std::deque은 양방향 큐이며 양 끝의 삽입·삭제와 임의 접근은 O(1), 중간의 삽입·삭제는 O(n)이다. 근거 대조 문구: 모든 원소에 대해 임의 접근 동작이 O(1)	(K 1.8.1 덱의 구조, PDF p.53)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-027	컨테이너 어댑터는 기존 컨테이너를 감싸 해당 자료 구조에 필요한 인터페이스만 제공한다. 근거 대조 문구: 스택 자료 구조에서 꼭 필요한 인터페이스만을 제공	(K 1.9 컨테이너 어댑터, PDF p.57)
U16-028	스택은 LIFO(Last In First Out, 후입선출) 구조이며 한쪽 끝에서만 데이터 삽입·삭제를 수행한다. 근거 대조 문구: LIFO(Last In First Out, 후입선출)	(K 1.9 컨테이너 어댑터, PDF p.56)
U16-029	std::queue는 FIFO(First In First Out, 선입선출) 구조이고 std::deque을 기본 컨테이너로 사용한다. 근거 대조 문구: FIFO(First In First Out, 선입선출)	(K 1.9.2 std::queue, PDF p.58)
U16-030	std::priority_queue는 기본적으로 std::vector와 std::less를 사용하며 최대 힙으로 생성된다. 근거 대조 문구: 기본적으로는 최대 힙(max heap)	(K 1.9.3 std::priority_queue, PDF p.58-59)
U16-031	STL은 스택·큐·우선순위 큐 어댑터에 반복자를 지원하지 않는다. 근거 대조 문구: 이들 어댑터에 대해서는 반복자를 지원하지 않습니다.	(K 1.9.4 어댑터 반복자, PDF p.59)
U16-032	벤치마킹은 통계 데이터를 기반으로 더 나은 접근 방식을 결정하는 방법이다. 근거 대조 문구: 통계 데이터를 기반으로 더 나은 접근 방식을 결정	(K 1.10 벤치마킹, PDF p.59)
U16-033	새로운 노드를 생성한다. 근거 대조 문구: 새로운 노드를 생성	(K 1.2.2 연결된 자료 구조, PDF p.15)
U16-034	새 노드의 next가 다음 노드를 가리키게 한다. 근거 대조 문구: next 포인터가 다음 노드(i = 3)를 가리키게	(K 1.2.2 연결된 자료 구조, PDF p.15)
U16-035	이전 노드의 next가 새 노드를 가리키도록 설정한다. 근거 대조 문구: 새로운 노드(i = 2)를 가리키도록	(K 1.2.2 연결된 자료 구조, PDF p.15)
U16-036	실제 모델과 비슷한 작은 프로토타입을 각 컨테이너로 구현한다. 근거 대조 문구: 실제 모델과 비슷한 작은 프로토타입	(K 1.10 벤치마킹, PDF p.60)
U16-037	작업을 여러 번 반복하여 수행 시간을 측정하고 비교한다. 근거 대조 문구: 작업을 여러 번 ... 반복하여 수행 시간을 측정	(K 1.10 벤치마킹, PDF p.60)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-038	성능 테스트에서 더 나은 결과를 내는 구현 방법을 선택한다. 근거 대조 문구: 더 나은 결과를 나타내는 구현 방법을 선택	(K 1.10 벤치마킹, PDF p.60)

U16-039	큐 사용 예제 — 원문 발췌 (원문 코드 발췌)
	std::queue<int> q;
	q.push(1);
	q.push(2);
	q.push(3);
	q.pop();
	q.push(4);
	근거 대조 문구: 원문 예제의 q 선언과 push/pop 문장
		(K 1.9.2 std::queue, PDF p.58)

U16-040	연속된 자료 구조의 i번째 원소 주소: BA + i * sizeof(type); BA: 시작 주소, sizeof(type): 원소 하나에 필요한 메모리 크기 근거 대조 문구: BA + i * sizeof(type)	(K 1.2.1 연속된 자료 구조, PDF p.13)
U16-041	임의 접근 근거 대조 문구: 임의 접근	(K 1.2.3 비교, PDF p.16)
U16-042	O(1) 근거 대조 문구: O(1)	(K 1.2.3 비교, PDF p.16)
U16-043	O(n) 근거 대조 문구: O(n)	(K 1.2.3 비교, PDF p.16)
U16-044	맨 뒤에 원소 삽입 근거 대조 문구: 맨 뒤에 원소 삽입	(K 1.2.3 비교, PDF p.16)
U16-045	중간에 원소 삽입 근거 대조 문구: 중간에 원소 삽입	(K 1.2.3 비교, PDF p.16)
U16-046	데이터에 적용할 연산과 사용 빈도에 따라 다양한 자료 구조를 비교한다. 근거 대조 문구: 이들 연산의 사용 빈도에 따라 다양한 자료 구조를 비교	(K 1.11 나가며, PDF p.61)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U16-047	컨테이너 어댑터는 의도를 파악하기 쉬운 코드를 작성하고 코딩 실수를 줄이는 데 도움이 된다. 근거 대조 문구: 프로그래머의 코딩 실수도 줄일 수 있습니다.	(K 1.11 나가며, PDF p.61)
U16-048	이 장에서 설명한 구조는 논리적 관점에서 선형이다. 근거 대조 문구: 모든 구조는 논리적 관점에서 선형	(K 1.11 나가며, PDF p.61)
U16-049	std::queue는 FIFO, 스택은 LIFO이다. 근거 대조 문구: LIFO / FIFO	(K 1.9 컨테이너 어댑터 ·1.9.2 std::queue, PDF p.56,58)
U16-050	스택 근거 대조 문구: 스택	(K 1.9, PDF p.56)
U16-051	LIFO, 한쪽 끝에서 삽입·삭제 근거 대조 문구: LIFO, 한쪽 끝에서 삽입·삭제	(K 1.9, PDF p.56)
U16-052	큐 근거 대조 문구: 큐	(K 1.9.2, PDF p.58)
U16-053	FIFO, 맨 뒤에 추가·맨 앞에서 제거 근거 대조 문구: FIFO, 맨 뒤에 추가·맨 앞에서 제거	(K 1.9.2, PDF p.58)

17 트리, 힙, 그래프

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-001	비선형 자료 구조가 필요한 경우를 판별하고 트리 구조를 구현·순회한다. 근거 대조 문구: 비선형 자료 구조를 사용해야 하는 경우를 판별	(K 2장 학습 목표, PDF p.63)
U17-002	그래프 구조를 구현하고 상황에 맞는 방법으로 그래프를 표현한다. 근거 대조 문구: 주어진 상황에 맞게 다양한 방법으로 그래프를 표현	(K 2장 학습 목표, PDF p.63)
U17-003	데이터가 저장된 부분을 노드(node), 노드 사이를 잇는 선을 에지(edge, 간선)라고 한다. 근거 대조 문구: 데이터가 저장된 부분을 보통 노드(node)라고 부르고	(K 2.2.1 계층적 문제, PDF p.65)
U17-004	트리는 노드와 에지로 계층을 구성하며 중심이 되는 루트 노드는 보통 맨 위에 표시한다. 근거 대조 문구: 트리의 중심이 되는 노드를 루트 노드	(K 2.3 트리: 상하 반전된 형태, PDF p.66)
U17-005	전위 순회는 현재 노드, 왼쪽 하위 노드, 오른쪽 하위 노드의 순서로 재귀적으로 방문한다. 근거 대조 문구: 현재 노드를 먼저 방문	(K 2.3.2 트리 순회, PDF p.70)
U17-006	중위 순회는 왼쪽 노드, 현재 노드, 오른쪽 노드의 순서로 방문한다. 근거 대조 문구: 왼쪽 노드를 먼저 방문	(K 2.3.2 트리 순회, PDF p.71)
U17-007	후위 순회는 두 자식 노드를 먼저 방문한 후 현재 노드를 방문한다. 근거 대조 문구: 두 자식 노드를 먼저 방문한 후	(K 2.3.2 트리 순회, PDF p.71)
U17-008	레벨 순서 순회는 맨 위 레벨부터 아래 레벨까지 왼쪽에서 오른쪽으로 방문한다. 근거 대조 문구: 맨 위 레벨부터 아래 레벨까지	(K 2.3.2 트리 순회, PDF p.71)
U17-009	이진 트리의 각 노드는 최대 두 개의 자식 노드를 가진다. 근거 대조 문구: 각 노드가 최대 두 개의 자식 노드를	(K 2.4 다양한 트리 구조, PDF p.73)
U17-010	BST는 왼쪽 노드 ≤ 부모 노드 ≤ 오른쪽 노드의 관계를 가진다. 근거 대조 문구: 왼쪽 노드 ≤ 부모 노드 ≤ 오른쪽 노드	(K 2.4.1 이진 검색 트리, PDF p.74)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-011	BST의 모양은 원소 삽입 순서에 따라 결정되므로 O(log N) 검색이 항상 성립하지는 않는다. 근거 대조 문구: 시간 복잡도가 O(log N)이라는 것도 항상 정확하다고 볼 수 없습니다.	(K 2.4.2 트리 연산의 시간 복잡도, PDF p.78)
U17-012	BST의 레벨 수를 높이(height)라고 하며 원소 검색의 실제 시간 복잡도는 O(높이)이다. 근거 대조 문구: 원소 검색의 실제 시간 복잡도는 O(높이)	(K 2.4.4 균형 트리, PDF p.85)
U17-013	원소 삽입·삭제 후 트리 구성을 조정하여 편향성을 줄인 이진 검색 트리를 높이-균형 BST라고 한다. 근거 대조 문구: 이렇게 조정되어 편향성이 줄어든 이진 검색 트리를 높이-균형 BST	(K 2.4.4 균형 트리, PDF p.85)
U17-014	N-항 트리는 각 노드가 N개의 자식을 가질 수 있으며 자식 노드를 벡터로 저장할 수 있다. 근거 대조 문구: N개의 자식 노드는 벡터를 이용하여 저장	(K 2.4.5 N-항 트리, PDF p.86)
U17-015	힙은 최대 원소 접근 O(1), 원소 삽입과 최대 원소 삭제 O(log N)을 만족해야 한다. 근거 대조 문구: O(log N): 최대 원소 삭제에 대한 시간 복잡도	(K 2.5 힙, PDF p.88)
U17-016	힙은 완전 이진 트리를 사용하며 루트부터 레벨 순서로 배열이나 벡터에 저장할 수 있다. 근거 대조 문구: 루트 노드를 배열 또는 벡터의 맨 처음에 저장	(K 2.5 힙, PDF p.88)
U17-017	부모 노드가 i번째 배열 원소이면 자식 노드는 2*i + 1, 2*i + 2이며 자식 i의 부모는 (i - 1)/2이다. 근거 대조 문구: 자식 노드는 2*i + 1 또는 2*i + 2	(K 2.5 힙, PDF p.89)
U17-018	최대 힙은 최대 원소를 루트에 두고 부모 노드가 두 자식 노드보다 항상 커야 한다는 불변성을 유지한다. 근거 대조 문구: 부모 노드가 두 자식 노드보다 항상 커야 한다	(K 2.5 힙, PDF p.89)
U17-019	모든 비교 연산을 반대로 설정하면 최소 원소에 빠르게 접근하는 최소 힙을 만들 수 있다. 근거 대조 문구: 모든 비교 연산을 반대로 설정	(K 2.5 힙, PDF p.89)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-020	힙 생성 알고리즘은 아래쪽 서브 트리부터 위로 올라가며 불변성을 맞추고 O(N) 시간에 힙을 초기화한다. 근거 대조 문구: O(N) 시간에 힙을 초기화	(K 2.5.1 힙 연산, PDF p.91)
U17-021	std::make_heap()은 배열이나 벡터의 반복자를 인자로 받아 힙을 구성한다. 근거 대조 문구: 배열 또는 벡터의 반복자를 인자로 받아	(K 2.5.1 힙 연산, PDF p.91)
U17-022	트리에서는 한 노드에서 다른 노드로 이동하는 경로가 하나만 존재하여 순환적 종속성을 표현할 수 없다. 근거 대조 문구: 이동하는 경로가 하나만 존재	(K 2.6 그래프, PDF p.96)
U17-023	그래프는 노드 데이터와 노드 사이의 에지 데이터를 함께 저장한다. 근거 대조 문구: 노드 사이의 에지 데이터도 저장	(K 2.6 그래프, PDF p.96)
U17-024	에지에 가중치나 더 많은 정보를 부여한 그래프를 가중 그래프라고 한다. 근거 대조 문구: 이러한 형태의 그래프를 가중 그래프	(K 2.6 그래프, PDF p.96)
U17-025	무방향 그래프의 에지는 양방향이며 방향 그래프에서 양방향을 표현하려면 서로 반대 방향의 두 에지를 사용한다. 근거 대조 문구: A에서 B, 그리고 B에서 A로 향하는 두 개의 에지	(K 2.6 그래프, PDF p.96)
U17-026	인접 행렬은 N개 노드를 N×N 배열로 표현하고 배열 원소에 에지 가중치를 저장하며 에지가 없으면 -1을 설정한다. 근거 대조 문구: 두 노드 사이에 에지가 존재하지 않으면 해당 원소에 -1	(K 2.6.1 인접 행렬로 그래프 표현하기, PDF p.97)
U17-027	인접 리스트는 각 노드에 직접 연결된 노드의 ID만 저장한다. 근거 대조 문구: 직접 연결되어 있는 노드의 ID만 저장	(K 2.6.3 인접 리스트로 그래프 표현하기, PDF p.101)
U17-028	루트 노드를 방문하고 자식 노드를 큐에 추가한다. 근거 대조 문구: 해당 노드의 자식 노드를 모두 큐에 추가	(K 2.3.3 연습 문제 8, PDF p.73)
U17-029	현재 레벨 방문이 끝나면 큐에서 다음 레벨 노드를 꺼내 방문한다. 근거 대조 문구: 큐에 저장된 노드를 꺼내어 방문	(K 2.3.3 연습 문제 8, PDF p.73)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-030	큐가 완전히 빌 때까지 반복한다. 근거 대조 문구: 큐가 완전히 빌 때까지 반복	(K 2.3.3 연습 문제 8, PDF p.73)
U17-031	배열의 맨 마지막 위치에 새 원소를 추가하여 완전 이진 트리를 유지한다. 근거 대조 문구: 배열의 맨 마지막 위치에 원소를 추가	(K 2.5.1 힙 연산, PDF p.89)
U17-032	새 원소의 부모 노드가 더 작으면 서로 교환한다. 근거 대조 문구: 부모 노드가 더 작으면 서로 교환	(K 2.5.1 힙 연산, PDF p.89)
U17-033	전체 트리가 불변 조건을 만족하도록 교환을 반복한다. 근거 대조 문구: 불변 조건이 만족하도록 교환 작업을 반복	(K 2.5.1 힙 연산, PDF p.90)
U17-034	루트와 마지막 노드를 교환한 후 마지막 노드를 삭제한다. 근거 대조 문구: 루트 노드와 트리의 맨 마지막 노드를 서로 교환	(K 2.5.1 힙 연산, PDF p.90)
U17-035	루트와 두 자식 중 더 큰 노드를 비교·교환한다. 근거 대조 문구: 그중 더 큰 노드와 서로 교환	(K 2.5.1 힙 연산, PDF p.90-91)
U17-036	불변성이 깨진 서브 트리에서도 재귀적으로 교환을 반복한다. 근거 대조 문구: 서브 트리에 대해서도 노드 교환 작업을 재귀적으로 반복	(K 2.5.1 힙 연산, PDF p.91)

U17-037	N-항 트리 노드 — 원문 발췌 (원문 코드 발췌)
	struct nTree
	{
	int data;
	std::vector<nTree*> children;
	};
	근거 대조 문구: 원문 struct nTree 정의
		(K 2.4.5 N-항 트리, PDF p.86)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U17-038	중위 순회 — 원문 발췌 (원문 코드 발췌)
	static void inOrder(node* start)
	{
	if (!start)
	return;
	inOrder(start->first);
	std::cout << start->position << ", ";
	inOrder(start->second);
	}
	근거 대조 문구: 원문 static void inOrder 정의
		(K 2.3.2 트리 순회, PDF p.71)

U17-039	BST 검색의 재귀식과 조건: T(N) = T(N/2) + 1; 원문은 이어서 검색 범위가 절반으로 줄어드는 가정이 항상 성립하지 않는다고 밝힌다. 근거 대조 문구: T(N) = T(N/2) + 1	(K 2.4.2 트리 연산의 시간 복잡도, PDF p.77-78)
U17-040	배열에 저장한 힙의 인덱스: 자식: 2*i + 1, 2*i + 2 / 부모: (i - 1)/2; 부모 또는 자식이 i번째 인덱스일 때 근거 대조 문구: 2*i + 1 또는 2*i + 2	(K 2.5 힙, PDF p.89)
U17-041	전위 순회 근거 대조 문구: 전위 순회	(K 2.3.2, PDF p.70)
U17-042	현재 → 왼쪽 → 오른쪽 근거 대조 문구: 현재 → 왼쪽 → 오른쪽	(K 2.3.2, PDF p.70)
U17-043	중위 순회 근거 대조 문구: 중위 순회	(K 2.3.2, PDF p.71)
U17-044	왼쪽 → 현재 → 오른쪽 근거 대조 문구: 왼쪽 → 현재 → 오른쪽	(K 2.3.2, PDF p.71)
U17-045	후위 순회 근거 대조 문구: 후위 순회	(K 2.3.2, PDF p.71)
U17-046	왼쪽 → 오른쪽 → 현재 근거 대조 문구: 왼쪽 → 오른쪽 → 현재	(K 2.3.2, PDF p.71)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U17-047	이 장은 트리와 그래프라는 두 자료 구조를 중점적으로 다뤘다. 근거 대조 문구: 트리와 그래프라는 두 자료 구조	(K 2.7 나가며, PDF p.105)
U17-048	다양한 상황에서 사용할 수 있는 여러 종류의 트리를 설명했다. 근거 대조 문구: 여러 종류의 트리	(K 2.7 나가며, PDF p.105)
U17-049	프로그램에서 이들 자료 구조를 표현하는 여러 방법을 검토했다. 근거 대조 문구: 자료 구조를 표현하는 다양한 방법	(K 2.7 나가며, PDF p.105)
U17-050	전위: 현재·왼쪽·오른쪽, 중위: 왼쪽·현재·오른쪽, 후위: 왼쪽·오른쪽·현재이다. 근거 대조 문구: 전위 순회 / 중위 순회 / 후위 순회	(K 2.3.2 트리 순회, PDF p.70-71)

18 해시 테이블과 블룸 필터

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-001	대용량 데이터의 룩업 문제를 이해하고 결정적·비결정적 솔루션을 구분한다. 근거 대조 문구: 결정적 룩업 솔루션이 적합한지, 또는 비결정적 룩업 솔루션	(K 3장 학습 목표, PDF p.107)
U18-002	상황에 맞는 효율적인 룩업과 C++ STL에서 제공되는 솔루션을 구현한다. 근거 대조 문구: C++ STL에서 제공되는 일반적인 솔루션	(K 3장 학습 목표, PDF p.107)
U18-003	룩업(lookup, 조회)은 특정 원소의 존재를 확인하거나 특정 키에 해당하는 값을 찾는 작업이다. 근거 대조 문구: 특정 키(key)에 해당하는 값(value)을 찾는 작업	(K 3.1 들어가며, PDF p.107)
U18-004	해싱은 데이터를 가급적 고유한 숫자 값으로 표현하고 같은 숫자를 이용하여 유무를 확인하거나 원본 데이터를 추출하는 작업이다. 근거 대조 문구: 각각의 데이터를 가급적 고유한 숫자 값으로 표현	(K 3.2 해시 테이블, PDF p.108)
U18-005	해시 함수는 데이터로부터 고유한 숫자 값을 계산하는 함수이며 해시 값은 그 함수가 반환한 숫자 값이다. 근거 대조 문구: 주어진 데이터로부터 고유한 숫자 값을 계산	(K 3.2 해시 테이블·3.2.1 해싱, PDF p.108-109)
U18-006	모듈로 함수 x % n은 0부터 n - 1 사이의 정수를 반환한다. 근거 대조 문구: 이 값은 0부터 (n - 1) 사이의 정수	(K 3.2.1 해싱, PDF p.109)
U18-007	충돌은 서로 다른 키에 대해 해시 함수가 같은 해시 값을 반환하는 현상이다. 근거 대조 문구: 서로 다른 키에 대해 같은 해시 값을 반환	(K 3.2.1 해싱, PDF p.109)
U18-008	체이닝은 해시 테이블의 특정 위치에 연결 리스트를 두어 충돌한 키들을 함께 저장한다. 근거 대조 문구: 하나의 연결 리스트를 저장	(K 3.3.1 체이닝, PDF p.114)
U18-009	체이닝에서 삽입은 O(1)이지만 모든 키가 같은 해시 값을 가지면 룩업은 O(N)이 된다. 근거 대조 문구: 룩업 연산은 연결 리스트에서 선형 검색을 수행	(K 3.3.2 연습 문제 14, PDF p.118)
U18-010	부하율은 해시 테이블의 각 리스트에 저장되는 키의 평균 개수이다. 근거 대조 문구: 각각의 리스트에 저장되는 키의 평균 개수	(K 3.3.2 연습 문제 14, PDF p.118)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-011	부하율은 해시 테이블 성능을 결정하는 유일한 지표가 아니며 해시 값이 골고루 분포되어야 한다. 근거 대조 문구: 부하율이 해시 테이블의 성능을 결정하는 유일한 지표는 아닙니다.	(K 3.3.2 연습 문제 14, PDF p.119)
U18-012	열린 주소 지정은 모든 원소를 해시 테이블 내부에 저장하고 충돌 시 다른 빈 위치를 탐색한다. 근거 대조 문구: 모든 원소를 해시 테이블 내부에 저장	(K 3.3.3 열린 주소 지정, PDF p.119)
U18-013	선형 탐색은 충돌한 위치에서 다음 셀로 하나씩 이동하여 빈 셀을 찾는다. 근거 대조 문구: 하나씩 다음 셀(cell) 위치로 이동	(K 3.3.3 열린 주소 지정, PDF p.120)
U18-014	이차함수 탐색은 선형 방정식 대신 이차 방정식을 사용하여 이동 폭을 증가시킨다. 근거 대조 문구: 이동 폭을 이차함수 형태로 증가	(K 3.3.3 열린 주소 지정, PDF p.121)
U18-015	뻐꾸기 해싱은 크기가 같고 해시 함수가 서로 다른 두 해시 테이블을 사용한다. 근거 대조 문구: 크기가 같은 두 개의 해시 테이블	(K 3.3.4 뻐꾸기 해싱, PDF p.122)
U18-016	각 원소는 두 위치 중 하나에 저장되며 삽입 후 다른 위치로 이동할 수 있다. 근거 대조 문구: 원소가 나중에 다른 위치로 이동	(K 3.3.4 뻐꾸기 해싱, PDF p.122)
U18-017	룩업은 저장 가능한 두 위치만 확인하므로 시간 복잡도가 항상 O(1)이다. 근거 대조 문구: 저장 가능한 위치 두 군데만 확인	(K 3.3.4 뻐꾸기 해싱, PDF p.122)
U18-018	삽입 중 순환이 발생하면 새 해시 함수로 재해싱해야 하며 여러 번 필요할 수 있다. 근거 대조 문구: 여러 번 재해싱을 수행해야 할 수도	(K 3.3.4 뻐꾸기 해싱, PDF p.123)
U18-019	원문은 높은 성능을 보장하려면 부하율을 0.5보다 작게 설정해야 한다고 설명한다. 근거 대조 문구: 부하율이 0.5보다 작게끔 설정	(K 3.3.4 뻐꾸기 해싱, PDF p.123)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-020	std::unordered_set은 키만, std::unordered_map은 키와 값을 함께 저장한다. 근거 대조 문구: std::unordered_set은 키만 저장	(K 3.4 C++ 해시 테이블, PDF p.131)
U18-021	기본 최대 부하율은 1이며 max_load_factor(float)로 최대 한계치를 변경할 수 있다. 근거 대조 문구: 기본값이 1로 되어 있는 부하율 최대 한계치	(K 3.4 C++ 해시 테이블, PDF p.131)
U18-022	std::unordered_map의 [] 연산자는 참조를 반환하며 키가 없으면 기본값을 추가하여 반환한다. 근거 대조 문구: 해당 키가 없다면 해당 위치에 기본값을 추가	(K 3.4.1 연습 문제 16, PDF p.135)
U18-023	std::unordered_set과 std::unordered_map은 중복 키를 허용하지 않으며 중복 저장에는 unordered_multiset 또는 unordered_multimap을 사용한다. 근거 대조 문구: 중복된 키를 허용하지 않습니다.	(K 3.4.1 연습 문제 16, PDF p.135)
U18-024	블룸 필터는 공간 효율이 높지만 거짓-긍정(false positive)이 나올 수 있으며 거짓-부정(false negative)은 없다. 근거 대조 문구: 거짓-부정(false negative)이 없다는 것은 보장	(K 3.5 블룸 필터, PDF p.138)
U18-025	블룸 필터의 긍정 답변은 실제로 있을 수도 없을 수도 있음을 뜻하며 부정 답변은 확실히 없음을 뜻한다. 근거 대조 문구: 이 원소는 확실히 없습니다.	(K 3.5 블룸 필터, PDF p.138)
U18-026	블룸 필터는 실제 값 대신 부울 타입 배열을 사용하고 여러 해시 함수에 대응하는 비트를 설정한다. 근거 대조 문구: 실제 값을 저장하지는 않으며	(K 3.5 블룸 필터, PDF p.138)
U18-027	원소가 많아질수록 거짓-긍정이 발생할 가능성이 증가한다. 근거 대조 문구: 원소 개수가 많아질수록 거짓-긍정이 발생할 가능성은 증가	(K 3.5 블룸 필터, PDF p.138)
U18-028	부울 배열이 모두 1이면 포화 상태가 되어 룩업은 항상 true를 반환하고 삽입은 필터 상태를 바꾸지 못한다. 근거 대조 문구: 룩업 함수는 항상 true를 반환	(K 3.5 블룸 필터, PDF p.138)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-029	첫 번째 테이블에서 새 원소를 저장할 위치가 비었는지 검사한다. 근거 대조 문구: 현재 비어 있는지를 검사	(K 3.3.4 뻐꾸기 해싱, PDF p.122)
U18-030	이미 원소가 있으면 새 원소를 저장하고 기존 원소를 두 번째 테이블로 옮긴다. 근거 대조 문구: B를 두 번째 해시 테이블로 옮깁니다.	(K 3.3.4 뻐꾸기 해싱, PDF p.122)
U18-031	빈 셀이 나타날 때까지 다른 테이블로 옮기는 과정을 반복하고 순환 시 재해싱한다. 근거 대조 문구: 완전히 비어 있는 셀이 나타날 때까지	(K 3.3.4 뻐꾸기 해싱, PDF p.122-123)
U18-032	삽입 시 모든 해시 함수 값을 계산하고 대응 비트를 1로 설정한다. 근거 대조 문구: 대응되는 위치의 비트 값을 1로 설정	(K 3.5 블룸 필터, PDF p.138)
U18-033	룩업 시 모든 해시 함수 값을 계산하여 해당 비트를 검사한다. 근거 대조 문구: 모든 해시 함수 값을 계산	(K 3.5 블룸 필터, PDF p.138)
U18-034	모든 비트가 1이면 true, 1이 아닌 비트가 하나라도 있으면 false를 반환한다. 근거 대조 문구: 1이 아닌 비트가 하나라도 있다면 false	(K 3.5 블룸 필터, PDF p.138)

U18-035	블룸 필터의 해시 함수 — 원문 발췌 (원문 코드 발췌)
	int hash(int num, int key)
	{
	switch (num)
	{
	case 0: return key % nBits;
	case 1: return (key / 7) % nBits;
	case 2: return (key / 11) % nBits;
	}
	return 0;
	}
	근거 대조 문구: 원문 int hash(int num, int key) 정의
		(K 3.5.1 연습 문제 17, PDF p.140)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-036	부하율: 부하율 = 전체 키 개수 / 해시 테이블 크기; 각 리스트에 저장되는 키의 평균 개수 근거 대조 문구: 그림 3-6 부하율	(K 3.3.2 연습 문제 14, PDF p.118)
U18-037	std::unordered_set 근거 대조 문구: std::unordered_set	(K 3.4, PDF p.131)
U18-038	키 근거 대조 문구: 키	(K 3.4, PDF p.131)
U18-039	std::unordered_map 근거 대조 문구: std::unordered_map	(K 3.4, PDF p.131)
U18-040	키와 값 근거 대조 문구: 키와 값	(K 3.4, PDF p.131)
U18-041	unordered_multiset / unordered_multimap 근거 대조 문구: unordered_multiset / unordered_multimap	(K 3.4.1, PDF p.135)
U18-042	중복 저장 허용 근거 대조 문구: 중복 저장 허용	(K 3.4.1, PDF p.135)
U18-043	룩업 문제에서는 필요에 따라 결정적 솔루션과 확률적 솔루션을 선택할 수 있다. 근거 대조 문구: 결정적 솔루션과 확률적 솔루션	(K 3.6 나가며, PDF p.144)
U18-044	C++의 해싱 관련 컨테이너를 이용하면 해시 테이블을 직접 작성하지 않아도 된다. 근거 대조 문구: 직접 해시 테이블을 작성하지 않아도	(K 3.6 나가며, PDF p.144)
U18-045	컨테이너에서 find()를 많이 사용한다면 룩업 기반 솔루션을 고려한다. 근거 대조 문구: find() 함수를 많이 사용한다면	(K 3.6 나가며, PDF p.144)
U18-046	true 근거 대조 문구: true	(K 3.5, PDF p.138)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U18-047	있을 수도 있고 없을 수도 있음 근거 대조 문구: 있을 수도 있고 없을 수도 있음	(K 3.5, PDF p.138)
U18-048	false 근거 대조 문구: false	(K 3.5, PDF p.138)
U18-049	확실히 없음 근거 대조 문구: 확실히 없음	(K 3.5, PDF p.138)

19 분할 정복

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-001	분할 정복 설계 패러다임과 병합 정렬·퀵 정렬·선형 시간 선택을 구현한다. 근거 대조 문구: 병합 정렬, 퀵 정렬, 선형 시간 선택	(K 4장 학습 목표, PDF p.147)
U19-002	맵리듀스 모델과 멀티스레드를 사용하는 C++ 맵리듀스 라이브러리를 다룬다. 근거 대조 문구: 멀티스레드를 사용하는 C++ 맵리듀스 라이브러리	(K 4장 학습 목표, PDF p.147)
U19-003	알고리즘은 입력을 받아 일련의 변환을 수행하여 결과를 출력한다. 근거 대조 문구: 일련의 변환을 수행하여 결과를 출력	(K 4.1 들어가며, PDF p.147)
U19-004	분할 정복은 문제를 작은 부분 문제로 나누고 각각의 해답을 구한 뒤 합쳐 전체 해답을 얻는다. 근거 대조 문구: 각 부분 문제 솔루션을 합쳐서 전체 문제	(K 4.1 들어가며, PDF p.149)
U19-005	선형 검색은 전체 원소를 방문하여 대상을 찾으며 정렬 여부와 관계없이 동작하고 시간 복잡도는 O(N)이다. 근거 대조 문구: 선형 검색 알고리즘의 시간 복잡도는 O(N)	(K 4.2 이진 검색, PDF p.150)
U19-006	이진 검색은 시퀀스가 정렬되어 있다는 사실을 이용하여 가운데 원소와 대상을 비교하고 검색 범위를 수정한다. 근거 대조 문구: 주어진 시퀀스가 정렬되어 있다는 사실	(K 4.2 이진 검색, PDF p.150)
U19-007	분할 정복은 분할(divide), 정복(conquer), 결합(combine)의 세 단계로 이루어진다. 근거 대조 문구: 분할(divide) ... 정복(conquer) ... 결합(combine)	(K 4.3 분할 정복 이해하기, PDF p.157)
U19-008	이진 검색은 작은 범위에서 구한 해답이 전체 해답과 같으므로 일반적인 결합 과정이 필요하지 않다. 근거 대조 문구: 결합 과정이 필요하지 않습니다.	(K 4.3 분할 정복 이해하기, PDF p.158)
U19-009	정렬 알고리즘은 여러 데이터 타입, 많은 데이터, 빠른 동작이라는 요구 사항을 고려한다. 근거 대조 문구: 정렬 알고리즘 구현에 있어 필요한 요구 사항	(K 4.4 분할 정복을 이용한 정렬 알고리즘, PDF p.158)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-010	외부 정렬은 전체 데이터가 메인 메모리에 상주하지 않은 상태에서 수행되는 정렬이다. 근거 대조 문구: 메인 메모리에 데이터가 상주하지 않은 상태	(K 4.4 분할 정복을 이용한 정렬 알고리즘, PDF p.159)
U19-011	병합 정렬은 집합을 작은 부분집합으로 나누어 각각을 정렬한 뒤 순서를 유지하며 합친다. 근거 대조 문구: 정렬된 부분집합을 오름차순 또는 내림차순 순서를 유지	(K 4.4.1 병합 정렬, PDF p.159)
U19-012	병합 정렬의 분할은 각 부분 배열이 하나의 원소만 가질 때 멈춘다. 근거 대조 문구: 각 부분 배열이 하나의 원소만 가질 때 멈춥니다.	(K 4.4.1 병합 정렬, PDF p.160)
U19-013	퀵 정렬의 목표는 평균 실행 시간을 줄이는 것이며 핵심 연산은 분할이다. 근거 대조 문구: 퀵 정렬의 핵심 연산은 ... 분할	(K 4.4.3 퀵 정렬, PDF p.163)
U19-014	퀵 정렬의 분할은 피벗 P를 기준으로 작거나 같은 원소의 L과 큰 원소의 R을 만들고 L, P, R 순서로 재구성한다. 근거 대조 문구: 입력 배열을 L, P, R 순서로 재구성	(K 4.4.3 퀵 정렬, PDF p.163)
U19-015	분할 후 피벗은 배열 전체를 정렬했을 때 실제로 있어야 하는 위치로 이동한다. 근거 대조 문구: P가 실제로 있어야 하는 위치로 이동	(K 4.4.3 퀵 정렬, PDF p.164)
U19-016	퀵 정렬은 두 개 이상의 원소가 있는 배열을 분할하고 L과 R에 재귀적으로 같은 작업을 적용한다. 근거 대조 문구: 생성된 부분 배열에 재귀적으로 적용	(K 4.4.3 퀵 정렬, PDF p.164)
U19-017	그림 4-9에서 병합 정렬의 최선·평균·최악 시간 복잡도는 모두 O(nlogn)이고, 퀵 정렬은 최선·평균 O(nlogn), 최악 $O(n^2)$이다. 근거 대조 문구: 병합 정렬과 퀵 정렬의 점근적 시간 복잡도	(K 4.4.4 그림 4-9, PDF p.168)
U19-018	선형 시간 선택은 정렬되지 않은 집합에서 i번째로 작은 원소를 O(n)에 찾는 문제를 다룬다. 근거 대조 문구: O(n) 시간 복잡도를 갖는 솔루션	(K 4.4.6 선형 시간 선택, PDF p.170)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-019	원문의 선형 시간 선택은 다섯 원소씩 나눈 부분 벡터의 중앙값을 모으고 그 중앙값을 피벗으로 사용한다. 근거 대조 문구: 각각의 부분 벡터 ... 다섯 개의 원소	(K 4.4.6 선형 시간 선택, PDF p.170-171)
U19-020	선형 시간 선택은 분할된 벡터 중 하나에만 재귀 호출하며 벡터 크기는 매번 적어도 30% 줄어든다. 근거 대조 문구: 벡터 크기는 적어도 30% 줄어듭니다.	(K 4.4.6~4.4.7 선형 시간 선택, PDF p.177)
U19-021	std::stable_sort()는 순위가 같은 원소의 원본 순서를 유지하며 정렬한다. 근거 대조 문구: 원본 순서가 변경되지 않도록	(K 4.5 그림 4-13, PDF p.178)
U19-022	std::partial_sort()는 컨테이너의 일부 구간에 대해 정렬한다. 근거 대조 문구: 일부 구간에 대해서 정렬	(K 4.5 그림 4-13, PDF p.178)
U19-023	std::transform()은 모든 원소에 함수를 적용하고 std::accumulate()는 누적 연산을 수행한다. 근거 대조 문구: 컨테이너 원소의 누적 합을 계산	(K 4.5 그림 4-13, PDF p.178)
U19-024	std::reduce()는 지정한 범위의 원소에 리듀스 연산을 수행하고 결과를 반환한다. 근거 대조 문구: 리듀스 연산을 수행하고 결과를 반환	(K 4.5 그림 4-13, PDF p.178)
U19-025	맵은 컨테이너 C의 모든 원소에 함수 f(x)를 적용하는 연산이다. 근거 대조 문구: 컨테이너의 모든 원소에 함수 f(x)를 적용	(K 4.6.1 맵과 리듀스 추상화, PDF p.180)
U19-026	리듀스는 모든 원소 x에 함수 f(acc,x)를 적용하여 하나의 값으로 축약하는 연산이다. 근거 대조 문구: 하나의 값으로 축약하는 연산	(K 4.6.1 맵과 리듀스 추상화, PDF p.180)
U19-027	맵리듀스 모델은 맵 단계에서 중간 키·값 쌍을 만들고 리듀스 단계에서 결합하여 최종 결과를 생성한다. 근거 대조 문구: 중간 단계의 〈키, 값〉 쌍	(K 4.6.3 맵리듀스 프레임워크를 이용한 부분 통합, PDF p.183)
U19-028	워드카운트는 여러 텍스트 파일에서 각각의 단어가 나타나는 횟수를 구하는 문제이다. 근거 대조 문구: 각각의 단어가 나타나는 횟수	(K 4.6.5 실습 문제 10, PDF p.188)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-029	동일한 방식으로 해결할 수 있는 여러 부분 문제로 분할한다. 근거 대조 문구: 동일한 방식으로 해결할 수 있는 여러 부분 문제	(K 4.3 분할 정복 이해하기, PDF p.157)
U19-030	각 부분 문제의 해답을 구한다. 근거 대조 문구: 각 부분 문제에 대한 해답	(K 4.3 분할 정복 이해하기, PDF p.157)
U19-031	각 해답을 결합하여 전체 문제의 해답을 구한다. 근거 대조 문구: 각 부분 문제의 해답을 결합	(K 4.3 분할 정복 이해하기, PDF p.157)
U19-032	입력 벡터를 다섯 개 이하의 원소를 갖는 부분 벡터들로 나누고 각각 정렬한다. 근거 대조 문구: 각각의 Vi를 정렬	(K 4.4.6 선형 시간 선택, PDF p.170)
U19-033	각 부분 벡터의 중앙값들을 모아 집합 M을 만들고 M의 중앙값 q를 찾는다. 근거 대조 문구: 집합 M에서 중앙값 q	(K 4.4.6 선형 시간 선택, PDF p.171)
U19-034	q를 피벗으로 전체 벡터를 L과 R로 나눈다. 근거 대조 문구: q를 피벗으로 삼아	(K 4.4.6 선형 시간 선택, PDF p.171)
U19-035	피벗의 순위와 i를 비교하여 반환하거나 L 또는 R에서 반복한다. 근거 대조 문구: 만약 i = k이면, q는 V의 i번째 원소	(K 4.4.6 선형 시간 선택, PDF p.172)

U19-036	중앙값 위치 찾기 — 원문 발췌 (원문 코드 발췌)
	template<typename T>
	auto find_median(typename std::vector<T>::iterator begin,
	typename std::vector<T>::iterator last)
	{
	// 정렬
	quick_sort<T>(begin, last);
	// 가운데(median) 원소 반복자를 반환
	return begin + (std::distance(begin, last) / 2);
	}
	근거 대조 문구: 원문 find_median 정의
		(K 4.4.7 연습 문제 21, PDF p.172)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U19-037	선형 시간 선택의 복잡도: O(n); 원문의 다섯 원소 묶음과 중앙값 피벗을 사용하는 알고리즘 근거 대조 문구: 시간 복잡도를 구해보면 O(n)	(K 4.4.6~4.4.7 선형 시간 선택, PDF p.177)
U19-038	병합 정렬 근거 대조 문구: 병합 정렬	(K 4.4.4 그림 4-9, PDF p.168)
U19-039	O(nlogn) 근거 대조 문구: O(nlogn)	(K 4.4.4 그림 4-9, PDF p.168)
U19-040	퀵 정렬 근거 대조 문구: 퀵 정렬	(K 4.4.4 그림 4-9, PDF p.168)
U19-041	O(n²) 근거 대조 문구: O(n²)	(K 4.4.4 그림 4-9, PDF p.168)
U19-042	분할 정복을 알고리즘 설계 패러다임과 소프트웨어 확장 도구의 두 관점으로 살펴봤다. 근거 대조 문구: 분할 정복을 두 가지 관점	(K 4.7 나가며, PDF p.192)
U19-043	분할 연산은 부분 정렬과 선형 시간 선택 같은 문제의 기초가 된다. 근거 대조 문구: 부분 정렬과 선형 시간 선택	(K 4.7 나가며, PDF p.192)
U19-044	알고리즘 구현은 데이터 타입에 종속적이지 않아야 하며 템플릿과 표준 라이브러리 함수를 사용할 수 있다. 근거 대조 문구: 데이터 타입에 종속적이지 않아야	(K 4.7 나가며, PDF p.192)
U19-045	맵은 각 원소에 f(x)를 적용하고 리듀스는 f(acc,x)를 적용하여 하나의 값으로 축약한다. 근거 대조 문구: 맵은 … 리듀스는	(K 4.6.1 맵과 리듀스 추상화, PDF p.180)

20 그리디 알고리즘

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-001	그리디 접근 방식과 최적 부분 구조·그리디 선택 속성을 판별한다. 근거 대조 문구: 최적 부분 구조와 그리디 선택 속성	(K 5장 학습 목표, PDF p.195)
U20-002	분할 가능 배낭·그래프 컬러링·디스조인트-셋을 이용한 크루스칼 MST를 구현한다. 근거 대조 문구: 크루스칼 최소 신장 트리 알고리즘	(K 5장 학습 목표, PDF p.195)
U20-003	그리디 알고리즘은 매 단계에서 가장 좋아 보이는 해답을 선택하여 지역적 최적 해법으로 전역적 최적 해법을 구성한다. 근거 대조 문구: 매 단계에서 '가장 좋아 보이는' 해답	(K 5.1 들어가며, PDF p.195)
U20-004	최단 작업 우선 스케줄링은 평균 대기 시간을 최소화하기 위해 일 처리가 빨리 끝나는 사람부터 배치한다. 근거 대조 문구: 일 처리가 가장 빨리 끝나는 사람을 맨 앞	(K 5.2.1 최단 작업 우선 스케줄링, PDF p.197)
U20-005	i번째 사람의 대기 시간은 첫 번째부터 i - 1번째 사람까지의 처리 시간 합이다. 근거 대조 문구: 일 처리 시간의 합만큼 대기 시간	(K 5.2.1 최단 작업 우선 스케줄링, PDF p.197)
U20-006	원문의 대기열은 순서 조정 후 평균 대기 시간이 15.25에서 8.875로 줄어든다. 근거 대조 문구: 평균 대기 시간이 8.875로 줄어들었습니다.	(K 5.2.2 연습 문제 24, PDF p.198)
U20-007	0-1 배낭 문제는 물건들의 무게 합이 T를 넘지 않으면서 가격 합이 최대가 되는 선택을 구한다. 근거 대조 문구: 물건들의 무게 합이 T를 넘지 않아야	(K 5.3.1 0-1 배낭 문제, PDF p.200)
U20-008	분할 가능 배낭 문제는 물건을 분할하여 일부만 배낭에 담을 수 있다. 근거 대조 문구: 각 물건의 일부분만을 배낭에 담을 수	(K 5.3.2 분할 가능 배낭 문제, PDF p.201)
U20-009	분할 가능 배낭은 단위 무게당 가격이 높은 순서로 선택한다. 근거 대조 문구: 단위 무게당 가격이 높은 순서	(K 5.3.2 분할 가능 배낭 문제, PDF p.201)
U20-010	작업 스케줄링 실습은 종료 시간 순으로 정렬하여 가장 빨리 끝나는 작업을 고르고 겹치는 작업을 제거한다. 근거 대조 문구: 종료 시간을 기준으로 전체 작업 리스트를 정렬	(K 5.3.4 실습 문제 11, PDF p.206)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-011	최적 부분 구조는 문제의 최적 솔루션이 부분 문제들의 최적 솔루션으로 구성되는 속성이다. 근거 대조 문구: 부분 문제들의 최적의 솔루션으로 구성	(K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)
U20-012	그리디 선택은 지역적 최적 솔루션을 반복 선택하여 전체 최적 솔루션을 구할 수 있는 속성이다. 근거 대조 문구: 지역적 최적 솔루션을 반복적으로 선택	(K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)
U20-013	원문은 최적 부분 구조와 그리디 선택을 모두 가진 문제에 그리디 방식으로 최적 솔루션을 구할 수 있다고 설명한다. 근거 대조 문구: 두 가지 속성을 모두 갖는 문제만	(K 5.3.5 그리디 알고리즘의 요구 조건, PDF p.207)
U20-014	최소 신장 트리는 그래프의 모든 정점을 연결하며 에지 가중치 합이 최소인 트리이다. 근거 대조 문구: 모든 정점을 연결하고 연결된 에지의 가중치 합이 최소	(K 5.3.6 최소 신장 트리 문제, PDF p.208)
U20-015	크루스칼 알고리즘은 매번 최소 가중치 에지를 선택하고 사이클이 생기지 않을 때 솔루션에 추가한다. 근거 대조 문구: 사이클이 발생하지 않으면 해당 에지와 양 끝 정점을	(K 5.3.6 최소 신장 트리 문제, PDF p.209)
U20-016	디스조인트-셋 또는 유니온-파인드는 트리 형태의 원소로 구성된 포레스트이다. 근거 대조 문구: 트리 형태의 원소로 구성된 포레스트	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)
U20-017	디스조인트-셋의 원소는 숫자 ID, 랭크와 부모 포인터를 가진다. 근거 대조 문구: 랭크(rank)와 부모에 대한 포인터	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)
U20-018	make-set(x)는 랭크 0이고 부모가 자기 자신인 새 원소를 추가한다. 근거 대조 문구: 새로 추가한 원소의 랭크는 0	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)
U20-019	find(x)는 부모 포인터를 따라 이동하여 트리의 루트를 반환한다. 근거 대조 문구: 트리의 루트를 반환	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)
U20-020	union(x, y)는 두 원소의 루트가 같으면 아무 작업도 하지 않고 다르면 높은 랭크 루트를 낮은 랭크 루트의 부모로 설정한다. 근거 대조 문구: 높은 랭크 루트를 낮은 랭크 루트의 부모로 설정	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.211)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-021	크루스칼에서 에지 양 끝의 루트가 같으면 사이클이 발생하므로 그 에지를 MST에 추가하지 않는다. 근거 대조 문구: 해당 에지를 MST에 추가하지 않습니다.	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.213)
U20-022	그래프 컬러링은 인접한 정점끼리 같은 색을 갖지 않도록 모든 정점에 색상을 지정하는 문제이다. 근거 대조 문구: 서로 인접한 정점끼리 같은 색을 가지지 않도록	(K 5.4 그래프 컬러링, PDF p.222)
U20-023	그래프 컬러링에서는 에지 가중치를 사용하지 않는다. 근거 대조 문구: 에지의 가중치는 사용하지 않으므로	(K 5.4 그래프 컬러링, PDF p.222)
U20-024	원문은 최소 색상 수를 찾는 그래프 컬러링을 NP-완전 문제로 소개한다. 근거 대조 문구: 최소 개수의 색상 수를 찾는 것은 NP-완전	(K 5.4 그래프 컬러링, PDF p.222)
U20-025	차수(degree)는 정점에 연결된 에지의 개수이다. 근거 대조 문구: 차수는 정점에 연결된 에지의 개수	(K 5.4.2 실습 문제 12, PDF p.227)
U20-026	웰시-포웰 알고리즘은 차수가 높은 정점부터 차례대로 그래프 컬러링을 수행한다. 근거 대조 문구: 차수(degree)가 높은 정점부터 차례대로	(K 5.4.2 실습 문제 12, PDF p.227)
U20-027	작업 리스트를 종료 시간으로 정렬한다. 근거 대조 문구: 종료 시간을 기준으로	(K 5.3.4 실습 문제 11, PDF p.206)
U20-028	가장 빠른 종료 시간을 갖는 작업을 선택한다. 근거 대조 문구: 가장 빠른 종료 시간을 갖는 작업	(K 5.3.4 실습 문제 11, PDF p.206)
U20-029	선택된 작업과 겹치는 작업을 제거하고 남은 작업에 대해 선택을 반복한다. 근거 대조 문구: 현재 선택된 작업과 시간이 겹치는 작업은 모두 제거	(K 5.3.4 실습 문제 11, PDF p.206)
U20-030	그래프의 모든 에지를 최소 힙에 넣고 모든 정점으로 디스조인트-셋을 초기화한다. 근거 대조 문구: 그래프 G의 모든 에지를 최소 힙 H에 추가	(K 5.3.6~5.3.7, PDF p.208,213)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-031	최소 가중치 에지를 하나 꺼낸다. 근거 대조 문구: H로부터 에지 e를 하나 꺼냅니다.	(K 5.3.6 최소 신장 트리 문제, PDF p.208)
U20-032	양 끝 정점의 루트가 같으면 버리고 다르면 트리를 병합하고 에지를 MST에 추가한다. 근거 대조 문구: 실제로 두 트리가 병합하면 해당 에지를 MST에 추가	(K 5.3.7 디스조인트-셋 자료 구조, PDF p.213)
U20-033	모든 정점을 차수 내림차순으로 정렬한다. 근거 대조 문구: 모든 정점을 차수에 대한 내림차순	(K 5.4.2 실습 문제 12, PDF p.227)
U20-034	색이 없는 첫 정점에 이웃이 사용하지 않은 색상 C를 지정한다. 근거 대조 문구: 아직 사용되지 않은 색상을 해당 정점에 지정	(K 5.4.2 실습 문제 12, PDF p.227)
U20-035	색이 없는 다른 정점도 이웃이 C를 갖지 않으면 C를 지정한다. 근거 대조 문구: 이 정점의 이웃이 C 색상을 가지고 있지 않다면	(K 5.4.2 실습 문제 12, PDF p.227)
U20-036	색이 없는 정점이 남아 있으면 반복하고 없으면 종료한다. 근거 대조 문구: 남아 있는 정점이 없다면 종료	(K 5.4.2 실습 문제 12, PDF p.227)

U20-037	대기 시간 계산 — 원문 발췌 (원문 코드 발췌)
	template<typename T>
	auto compute_waiting_times(std::vector<T>& service_times)
	{
	std::vector<T> W(service_times.size());
	W[0] = 0;
	for (auto i = 1; i < service_times.size(); i++)
	W[i] = W[i - 1] + service_times[i - 1];
	return W;
	}
	근거 대조 문구: 원문 compute_waiting_times 정의
		(K 5.2.2 연습 문제 24, PDF p.198)

U20-038	원문 대기 시간 계산식: W[0] = 0; W[i] = W[i - 1] + service_times[i - 1];; service_times: 일 처리 시간 벡터, W: 대기 시간 벡터 근거 대조 문구: W[i] = W[i - 1] + service_times[i - 1];	(K 5.2.2 연습 문제 24, PDF p.198)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U20-039	0-1 배낭 근거 대조 문구: 0-1 배낭	(K 5.3.1, PDF p.200)
U20-040	물건들을 선택하여 담음 근거 대조 문구: 물건들을 선택하여 담음	(K 5.3.1, PDF p.200)
U20-041	분할 가능 배낭 근거 대조 문구: 분할 가능 배낭	(K 5.3.2, PDF p.201)
U20-042	물건의 일부분만 담을 수 있음 근거 대조 문구: 물건의 일부분만 담을 수 있음	(K 5.3.2, PDF p.201)
U20-043	그리디 방식은 부분적인 최선의 선택이 전체 최선으로 이어지는 문제에 적용할 수 있다. 근거 대조 문구: 전체적인 '최선의' 방법으로 귀결	(K 5.5 나가며, PDF p.230)
U20-044	최적 부분 구조와 그리디 선택을 크루스칼 MST로 살펴보며 디스조인트-셋을 설명했다. 근거 대조 문구: 디스조인트-셋 자료 구조	(K 5.5 나가며, PDF p.230)
U20-045	그래프 컬러링 솔루션으로 웰시-포웰 알고리즘을 다뤘다. 근거 대조 문구: 웰시-포웰 알고리즘	(K 5.5 나가며, PDF p.230)
U20-046	최단 작업 우선 근거 대조 문구: 최단 작업 우선	(K 5.2.1, PDF p.197)
U20-047	처리 시간이 짧은 순 근거 대조 문구: 처리 시간이 짧은 순	(K 5.2.1, PDF p.197)
U20-048	단위 무게당 가격이 높은 순 근거 대조 문구: 단위 무게당 가격이 높은 순	(K 5.3.2, PDF p.201)
U20-049	크루스칼 MST 근거 대조 문구: 크루스칼 MST	(K 5.3.6, PDF p.209)
U20-050	사이클을 만들지 않는 최소 가중치 에지 근거 대조 문구: 사이클을 만들지 않는 최소 가중치 에지	(K 5.3.6, PDF p.209)

21 그래프 알고리즘 I

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-001	그래프에서 적절한 탐색 방법을 선택하여 구현하고, 프림 알고리즘으로 최소 신장 트리를 구한다. 근거 대조 문구: 그래프에서 특정 원소를 찾기 위해 적절한 탐색 방법을 선택하여 구현	(K 6장 학습 목표, PDF p.233)
U21-002	프림과 크루스칼 알고리즘을 상황에 따라 선택하고, 다익스트라 알고리즘으로 두 정점 사이의 최단 경로를 찾는다. 근거 대조 문구: 프림 알고리즘과 크루스칼 알고리즘 중에서 주어진 상황에 적합한 알고리즘을 선택	(K 6장 학습 목표, PDF p.233)
U21-003	그래프는 정점(vertex)의 집합과 정점들을 서로 잇는 에지(edge)의 집합으로 구성되며, G = 〈V, E〉에서 V는 정점 집합, E는 에지 집합이다. 근거 대조 문구: 그래프는 정점(vertex)의 집합과 정점들을 서로 잇는 에지(edge)의 집합으로 구성	(K 6.1, PDF p.233)
U21-004	에지가 특정 정점에서 다른 정점으로 향하는 방향이 있으면 방향 그래프(directed graph), 특정 방향이 없으면 무방향 그래프(undirected graph)이다. 근거 대조 문구: 방향이 있다면 방향 그래프	(K 6.1, PDF p.233)
U21-005	에지에 가중치가 있으면 가중 그래프(weighted graph), 가중치가 없으면 비가중 그래프(unweighted graph)이다. 근거 대조 문구: 가중치가 있으면 가중 그래프	(K 6.1, PDF p.233)
U21-006	그래프에는 두 정점 사이의 여러 에지, 한 에지의 여러 가중치, 자기 자신으로 연결되는 셀프 에지(self edge)가 있을 수 있다. 근거 대조 문구: 두 정점 사이에는 여러 개의 에지가 있을 수도 있고	(K 6.1, PDF p.234)
U21-007	그래프 순회 문제(graph traversal problem)는 특정 정점에서 시작하여 나머지 모든 정점을 방문하는 문제이며, 특정 정점을 찾는 용도로도 쓰여 그래프 탐색 문제라고도 부른다. 근거 대조 문구: 특정 정점에서 시작하여 나머지 모든 정점을 방문하는 문제	(K 6.2, PDF p.235)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-008	너비 우선 탐색(BFS, Breadth-First Search)은 시작 정점을 경계(frontier)에 추가한 뒤, 현재 경계에 인접한 정점을 반복적으로 탐색한다. 근거 대조 문구: 시작 정점을 경계(frontier)에 추가하는 것으로 시작	(K 6.2.1, PDF p.236)
U21-009	BFS는 시작 정점에서 멀리 있는 정점보다 가까운 정점을 먼저 방문하며, 같은 거리에 있는 정점들의 방문 순서는 임의로 정할 수 있다. 근거 대조 문구: 시작 정점과 같은 거리에 있는 정점들의 방문 순서는 임의	(K 6.2.1, PDF p.236)
U21-010	BFS는 자식 정점을 손자 정점보다 먼저 방문하며, 보통 경계를 별도 자료 구조로 만들지 않고 정점 ID를 큐에 저장한다. 근거 대조 문구: 자식 정점을 손자 정점보다 먼저 방문	(K 6.2.1, PDF p.237)
U21-011	깊이 우선 탐색(DFS, Depth-First Search)은 시작 정점에서 특정 경로를 따라 가능한 멀리 있는 정점을 재귀적으로 먼저 방문한다. 근거 대조 문구: 특정 경로를 따라 가능한 멀리 있는 정점을 재귀적으로 먼저 방문	(K 6.2.3, PDF p.243)
U21-012	DFS는 더 방문할 정점이 없어지면 다른 경로를 찾아 다시 멀어지는 방향으로 탐색하며, 이러한 방법을 백트래킹(backtracking)이라고 한다. 근거 대조 문구: 더 방문할 정점이 없어지면 다른 경로를 찾아	(K 6.2.3, PDF p.243)
U21-013	BFS는 선입선출(FIFO) 큐를 사용하고, DFS는 후입선출(LIFO) 스택을 사용하여 인접한 정점들을 재귀적으로 이동하며 방문한다. 근거 대조 문구: 큐 대신에 스택을 사용한다는 점이 다릅니다	(K 6.2.3, PDF p.245)
U21-014	BFS는 시작 정점에서 가까운 정점을, DFS는 대체로 멀리 있는 정점을 찾을 때 적합하다. 근거 대조 문구: BFS는 시작 정점에서 가까운 정점을 찾는 데 적합	(K 6.2.4, PDF p.249)
U21-015	본문은 BFS의 검색 트리를 짧고 넓으며 많은 메모리를 필요로 하는 것으로, DFS의 검색 트리를 길고 좁으며 상대적으로 적은 메모리를 필요로 하는 것으로 비교한다. 근거 대조 문구: BFS에 의해 생성된 검색 트리는 짧고 넓은 편	(K 6.2.4, PDF p.249)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-016	이분 그래프(bipartite graph)는 정점을 두 집합으로 나눌 수 있고 모든 에지가 서로 다른 집합의 정점끼리 연결되는 그래프이다. 근거 대조 문구: 그래프의 모든 에지는 서로 다른 집합에 속한 정점끼리 연결	(K 6.2.5, PDF p.249)
U21-017	DFS로 이동하며 두 색을 번갈아 칠할 수 있으면 이분 그래프이며, 같은 색을 할당한 인접한 두 정점을 만나면 이분 그래프가 아니다. 근거 대조 문구: DFS 과정에서 같은 색상으로 할당된 두 개의 인접한 정점을 만나면	(K 6.2.5, PDF p.252)
U21-018	최소 신장 트리(MST) 문제는 주어진 그래프의 모든 정점을 연결하고, 연결된 에지의 가중치 합이 최소인 트리 T를 구하는 문제이다. 근거 대조 문구: 모든 정점을 연결하고 연결된 에지의 가중치 합이 최소인 트리 T	(K 6.3, PDF p.253)
U21-019	프림 알고리즘(Prim's algorithm)은 시작 정점으로 경계를 구성한 뒤 경계를 관통하는 에지 중 가중치가 가장 작은 에지를 선택하여 연결된 정점을 방문한다. 근거 대조 문구: 경계를 관통하는 에지 중에서 가장 가중치가 작은 에지를 선택	(K 6.3, PDF p.253)
U21-020	프림 알고리즘을 구현하려면 각 정점에 경계로부터의 거리(distance)를 기록해야 한다. 근거 대조 문구: 각 정점에 경계로부터의 거리(distance) 정보를 기록	(K 6.3, PDF p.253)
U21-021	프림 알고리즘은 인접한 정점의 거리 값이 현재 정점과 연결하는 에지 가중치보다 크면 그 가중치로 바꾸며, 이 과정을 현재 정점에 안착(settle)했다고 한다. 근거 대조 문구: 이러한 과정을 정점 U에 안착(settle)했다고	(K 6.3, PDF p.254)
U21-022	단일 시작(single source) 최단 경로 문제(shortest path problem)는 시작 정점에서 목적 정점으로 이어지는 최소 비용 경로를 구하는 문제이다. 근거 대조 문구: 시작 정점에서 목적 정점까지 이어지는 최소 비용 경로	(K 6.4, PDF p.259)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-023	다익스트라 알고리즘(Dijkstra's algorithm)은 음수 가중치가 없는 그래프에서 동작하며 프림의 MST 알고리즘을 약간 변경한 형태이다. 근거 대조 문구: 음수 가중치가 없는 그래프에서 동작하는 최단 경로 탐색 알고리즘	(K 6.4, PDF p.260)
U21-024	프림은 경계로부터의 최소 거리를 사용하지만, 다익스트라는 시작 정점으로부터 각 정점까지의 전체 거리를 사용한다. 근거 대조 문구: 시작 정점으로부터 각 정점까지의 전체 거리를 사용	(K 6.4, PDF p.260)
U21-025	다익스트라는 목적 정점이 나타나면 종료하지만, 프림은 모든 정점을 방문해야 종료한다. 근거 대조 문구: 다익스트라 알고리즘은 목적 정점이 나타나면 종료	(K 6.4, PDF p.260)
U21-026	다익스트라는 인접 정점의 거리 값이 현재 정점의 거리 값과 연결 에지 가중치의 합보다 크면 그 합으로 거리 값을 바꾼다. 근거 대조 문구: U의 거리 값 + (U, V) 에지 가중치	(K 6.4, PDF p.261)
U21-027	모든 정점의 거리 값을 무한대로 초기화하고 시작 정점의 거리만 0으로 설정한 뒤, 모든 거리 값을 최소 힙 H에 추가한다. 근거 대조 문구: 모든 거리 값을 최소 힙 H에 추가	(K 6.3, PDF p.253)
U21-028	최소 힙에서 정점 U를 꺼내 MST에 추가하고, U를 포함하도록 경계를 새로 설정한다. 근거 대조 문구: 이 정점을 MST에 추가하고	(K 6.3, PDF p.254)
U21-029	U에 인접한 각 정점 V의 거리 값이 (U, V)의 에지 가중치보다 크면 그 가중치로 설정한다. 근거 대조 문구: V의 거리 값을 (U, V)의 에지 가중치로 설정	(K 6.3, PDF p.254)
U21-030	방문하지 않은 정점이 남아 있으면 최소 힙에서 정점을 꺼내는 단계로 돌아간다. 근거 대조 문구: 방문하지 않은 정점이 남아 있다면 2단계로 이동	(K 6.3, PDF p.254)
U21-031	모든 정점의 거리 값을 무한대로 초기화하되 시작 정점만 0으로 설정하고, 모든 거리 값을 최소 힙 H에 추가한다. 근거 대조 문구: 시작 정점의 거리 값은 0으로 설정	(K 6.4, PDF p.260)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-032	최소 힙에서 정점 U를 꺼내고, U가 목적 정점이면 알고리즘을 종료한다. 근거 대조 문구: 만약 U가 목적 정점이면 최단 경로를 찾은 것	(K 6.4, PDF p.260)
U21-033	방문하지 않은 정점이 남아 있으면 최소 힙에서 정점을 꺼내는 단계로 돌아가며, 목적 정점을 꺼내면 종료한다. 근거 대조 문구: 최소 힙 H로부터 꺼낸 정점이 목적 정점	(K 6.4, PDF p.261)
U21-034	그래프를 이동하면서 두 색을 번갈아 정점에 칠한다. 근거 대조 문구: 두 개의 색상을 번갈아 칠	(K 6.2.5, PDF p.252)
U21-035	모든 정점에 이 방식으로 색을 칠할 수 있으면 이분 그래프로 간주한다. 근거 대조 문구: 모든 정점에 색상을 칠할 수 있다면 이분 그래프	(K 6.2.5, PDF p.252)
U21-036	DFS 과정에서 같은 색의 인접한 두 정점을 만나면 이분 그래프가 아닌 것으로 판별한다. 근거 대조 문구: 이분 그래프가 아닌 것으로 판별	(K 6.2.5, PDF p.252)

U21-037	연습 문제 28: Edge 구조체의 원문 발췌 (원문 코드 발췌)
	template <typename T>
	struct Edge
	{
	unsigned src;
	unsigned dst;
	T weight;
	};
	근거 대조 문구: 그래프의 에지를 표현하는 Edge 구조체 정의
		(K 6.2.2, PDF p.238)

U21-038	BFS와 DFS의 시간 복잡도: O(V + E); V는 정점의 개수, E는 에지의 개수이다. 근거 대조 문구: BFS와 DFS의 시간 복잡도는 모두 O(V + E)	(K 6.2.2·6.2.4, PDF p.243,249)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-039	프림 알고리즘의 시간 복잡도: O(E log V), O(E + V log V); 앞의 식은 이진 최소 힙과 인접 리스트를 사용한 경우, 뒤의 식은 피보나치 최소 힙을 사용한 경우이다. 근거 대조 문구: 이진 최소 힙과 MST 저장을 위해 인접 리스트를 사용	(K 6.3.1, PDF p.259)
U21-040	다익스트라 알고리즘의 시간 복잡도: O(E + V log V); 피보나치 최소 힙을 사용한 경우이다. 근거 대조 문구: 피보나치 최소 힙을 사용하여 구현한 다익스트라 알고리즘	(K 6.4.1, PDF p.266)
U21-041	MST 구성 방식 근거 대조 문구: 최소 가중치 에지를 차례대로 추가	(K 6.3.1, PDF p.259)
U21-042	최소 가중치 에지를 차례대로 추가한다. 근거 대조 문구: 그래프의 최소 가중치 에지를 차례대로 추가	(K 6.3.1, PDF p.259)
U21-043	아무 정점부터 시작한다. 근거 대조 문구: 그래프의 아무 정점부터 시작	(K 6.3.1, PDF p.259)
U21-044	가장 널리 알려진 시간 복잡도 근거 대조 문구: 가장 널리 알려진 시간 복잡도	(K 6.3.1, PDF p.259)
U21-045	O(E log V) 근거 대조 문구: O(E log V)	(K 6.3.1, PDF p.259)
U21-046	O(E + V log V) — 피보나치 최소 힙 사용 근거 대조 문구: 피보나치 최소 힙(Fibonacci min-heap)	(K 6.3.1, PDF p.259)
U21-047	주로 사용하는 그래프 근거 대조 문구: 희소 그래프	(K 6.3.1, PDF p.259)
U21-048	적은 수의 에지로 구성된 희소 그래프(sparse graph) 근거 대조 문구: 적은 수의 에지로 구성된 희소 그래프	(K 6.3.1, PDF p.259)
U21-049	많은 수의 에지로 구성된 밀집 그래프(dense graph) 근거 대조 문구: 많은 수의 에지로 구성된 밀집 그래프	(K 6.3.1, PDF p.259)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U21-050	그래프 순회 문제의 해결 방법으로 너비 우선 탐색(BFS)과 깊이 우선 탐색(DFS)을 소개했다. 근거 대조 문구: 그 솔루션으로 너비 우선 탐색(BFS)과 깊이 우선 탐색(DFS) 방법	(K 6.5, PDF p.268)
U21-051	MST 문제를 해결하는 프림 알고리즘을 설명하고 크루스칼 알고리즘과 비교했다. 근거 대조 문구: 프림 알고리즘과 크루스칼 알고리즘을 서로 비교	(K 6.5, PDF p.268)
U21-052	단일 시작 최단 경로 문제와 다익스트라 알고리즘을 설명하며, 6.4 본문은 음수 가중치가 없는 그래프에서 동작한다고 명시한다. 근거 대조 문구: 단일 시작 최단 경로 문제를 소개 / 음수 가중치가 없는 그래프에서 동작	(K 6.5, PDF p.268; K 6.4, PDF p.260)
U21-053	프림 근거 대조 문구: 프림 알고리즘	(K 6.4, PDF p.260)
U21-054	경계로부터 최소 거리·모든 정점 방문 후 종료 근거 대조 문구: 프림 알고리즘은 모든 정점을 방문해야 종료	(K 6.4, PDF p.260)
U21-055	다익스트라 (음수 가중치 없음) 근거 대조 문구: 음수 가중치가 없는 그래프에서 동작	(K 6.4, PDF p.260)
U21-056	시작 정점으로부터 전체 거리·목적 정점이 나타나면 종료 근거 대조 문구: 시작 정점으로부터 각 정점까지의 전체 거리를 사용	(K 6.4, PDF p.260)

22 그래프 알고리즘 II

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U22-001	다익스트라 알고리즘의 문제를 이해하고, 벨만-포드 알고리즘과 존슨 알고리즘으로 최단 경로를 찾으며 음수 가중치 사이클을 판단한다. 근거 대조 문구: 벨만-포드 알고리즘과 존슨 알고리즘을 사용하여 그래프에서 최단 경로	(K 7장 학습 목표, PDF p.271)
U22-002	방향·무방향 그래프의 연결성 차이를 설명하고, 코사라주 알고리즘으로 강한 연결 요소를 찾는다. 근거 대조 문구: 코사라주 알고리즘을 이용하여 그래프에서 강한 연결 요소	(K 7장 학습 목표, PDF p.271)
U22-003	다익스트라는 방문하지 않은 정점 중 가장 가까운 정점을 선택한 뒤 다시 검토하지 않으므로, 음수 에지를 고려하기 전에 다른 정점을 방문해 최적 경로를 찾지 못할 수 있다. 근거 대조 문구: 이렇게 방문한 정점은 나중에 다시 검토하지 않습니다	(K 7.2, PDF p.272)
U22-004	벨만-포드 알고리즘(Bellman-Ford algorithm)은 음수 가중치가 있는 그래프에서 사용할 수 있으며, 모든 에지에 대한 선택을 (V - 1)번 반복해 점진적으로 최단 거리를 찾는다. 근거 대조 문구: 그래프의 모든 에지에 대해 다익스트라의 그리디 선택 방법을 (V - 1)번 반복	(K 7.3, PDF p.273)
U22-005	연습 문제 32의 UNKNOWN은 무한대를 표현하는 정수 상수로, 에지 가중치의 합보다 충분히 큰 값을 사용하며 코드에서는 INT_MAX로 정의한다. 근거 대조 문구: UNKNOWN은 정수 상수 값으로, 그래프 에지 가중치의 합보다 충분히 큰 정수	(K 7.3.1, PDF p.274)
U22-006	BellmanFord()는 에지 리스트 edges, 정점 개수 V, 출발 정점 번호 start를 받고 거리 값 배열을 반환한다. 근거 대조 문구: 에지 리스트 edges, 정점 개수 V, 출발 정점 번호 start	(K 7.3.1, PDF p.274)
U22-007	각 에지의 시작 정점 거리 값이 UNKNOWN이 아닌 경우, 시작 정점 거리와 에지 가중치의 합이 목적 정점 거리보다 작으면 그 합으로 목적 정점 거리를 교체한다. 근거 대조 문구: 시작 정점의 거리 값이 UNKNOWN이 아닌 경우에 대해서만	(K 7.3.1, PDF p.275)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U22-008	음수 가중치 사이클(negative weight cycle)은 에지 가중치의 합이 음수가 되는 사이클이다. 근거 대조 문구: 에지 가중치의 합이 음수가 되는 사이클	(K 7.4, PDF p.277)
U22-009	음수 가중치 사이클이 있으면 그 사이클이 계산에 반복 사용되어 최종 결과가 왜곡될 수 있다. 근거 대조 문구: 해당 사이클이 반복적으로 계산에 사용되어 최종 결과가 왜곡	(K 7.4, PDF p.277)
U22-010	벨만-포드의 (V - 1)번 반복 후 전체 에지를 다시 검사하여 더 짧은 경로가 발견되면 음수 가중치 사이클이 있다고 판단한다. 근거 대조 문구: 더 짧은 경로가 발견된다면 음수 가중치 사이클이 존재	(K 7.4·7.4.1, PDF p.277-278)
U22-011	연습 문제 33은 음수 가중치 사이클이 발견되면 "음수 가중치 사이클 발견!"을 출력하고 빈 벡터를 반환하며 종료한다. 근거 대조 문구: 빈 벡터를 반환하면서 함수를 종료	(K 7.4.1, PDF p.278)
U22-012	실습 문제 15 '욕심쟁이 로봇'은 도착 시 남는 에너지를 최대화하는 경로를 선택하고, 충전소를 반복 방문하여 과충전될 가능성을 예측하는 문제이다. 근거 대조 문구: 목적지에 도달할 때 남아 있는 에너지(전력)가 최대	(K 7.4.2, PDF p.281)
U22-013	음수 에지 가중치가 없는 그래프에서는 그리디 접근을 사용하는 다익스트라 알고리즘이 벨만-포드보다 효율적이다. 근거 대조 문구: 음수 에지 가중치를 가지고 있지 않다면	(K 7.4.2, PDF p.284)
U22-014	존슨 알고리즘(Johnson's algorithm)은 벨만-포드와 다익스트라를 결합하여 모든 정점 사이의 최단 경로를 구하며, 전체 에지 가중치를 음수가 아닌 형태로 변환한다. 근거 대조 문구: 두 방법을 결합하여 모든 정점 사이의 최단 경로	(K 7.5, PDF p.285)
U22-015	새 더미(dummy) 정점을 추가하고 모든 정점에 가중치가 0인 에지를 연결한 뒤, 벨만-포드로 더미 정점에서 각 정점까지의 최단 거리를 기록한다. 근거 대조 문구: 새로운 '더미(dummy)' 정점을 추가	(K 7.5, PDF p.285)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U22-016	가중치 변환 수식을 연속 적용하면 각 줄의 세 번째 항이 다음 줄의 가운데 항에 의해 상쇄되며, 이를 망원(telescoping) 속성이라고 한다. 근거 대조 문구: 이를 '망원(telescoping)' 속성이라고	(K 7.5, PDF p.286)
U22-017	가중치 변환은 원래의 최단 경로 순서를 유지하면서 음이 아닌 가중치의 그래프를 만들며, 각 정점에서 다익스트라를 적용해 모든 정점 쌍의 최단 거리를 구한다. 근거 대조 문구: 원래의 최단 경로 순서는 그대로 유지한 채	(K 7.5, PDF p.287)
U22-018	연습 문제 34는 변환한 가중치로 구한 거리에 변환 수식을 역으로 적용하여 실제 최단 거리 값을 다시 계산한다. 근거 대조 문구: 가중치 변환 수식을 역으로 적용하여 실제 최단 거리 값을 다시 계산	(K 7.5.1, PDF p.292)
U22-019	실습 문제 16에서 정점의 평균 최단 거리는 그 정점에서 출발하는 최단 경로 거리 합을 도달 가능한 정점 개수로 나눈 값이며, 그래프의 평균 거리는 모든 정점의 평균 최단 거리의 평균이다. 근거 대조 문구: 정점의 평균 최단 거리는 해당 정점에서 출발하는 최단 경로의 거리 합	(K 7.5.2, PDF p.295)
U22-020	방향 그래프를 같은 구역의 정점끼리 서로 이동 가능한 경로를 갖도록 나눌 때, 각 구역이 강한 연결 요소를 나타낸다. 근거 대조 문구: 같은 구역 안의 정점끼리는 서로 이동 가능한 경로	(K 7.6, PDF p.297)
U22-021	무방향 그래프의 연결 요소(connected component)는 모든 정점이 서로 연결된 부분 그래프 중 최대 크기의 부분 그래프 집합이며, 그 안의 두 정점은 상호 접근이 가능하다. 근거 대조 문구: 모든 정점이 서로 연결되어 있는 부분 그래프 중에서 최대 크기	(K 7.6.1, PDF p.297)
U22-022	무방향 그래프의 연결 요소는 다른 부분 그래프와 완전히 단절되지만, 강한 연결 요소(strongly connected component)는 다른 부분 그래프와 완전히 단절될 필요가 없다. 근거 대조 문구: 강한 연결 요소(strongly connected component)는 그래프 내에서 다른 부분 그래프와 완전히 단절되어 있을 필요는	(K 7.6.1, PDF p.298)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U22-023	코사라주 알고리즘(Kosaraju's algorithm)은 강한 연결 요소를 찾기 위해 입력 그래프에 DFS를 한 번, 전치한 그래프에 DFS를 한 번 수행한다. 근거 대조 문구: 코사라주 알고리즘은 DFS를 두 번 수행	(K 7.7, PDF p.300)
U22-024	그래프의 전치(transpose)는 시작 정점과 목표 정점을 서로 뒤바꿔 각 에지의 방향을 반대로 하는 것이다. 근거 대조 문구: 각각의 에지 방향이 반대	(K 7.7, PDF p.300)
U22-025	첫 번째 DFS는 방문을 기록하고 미방문 인접 정점을 재귀적으로 탐색한 뒤, 인접 정점을 모두 탐색한 현재 정점을 스택에 추가한다. 근거 대조 문구: 인접한 정점을 모두 탐색한 후에는 현재 정점을 스택에 추가	(K 7.7, PDF p.300)
U22-026	전치 그래프에서는 스택에서 꺼낸 정점을 아직 방문하지 않았다면 DFS를 시작하며, 각각의 DFS에서 만나는 정점들이 강한 연결 요소를 구성한다. 근거 대조 문구: 각각의 DFS에서 만나는 정점들이 강한 연결 요소를 구성	(K 7.7, PDF p.300)
U22-027	본문은 코사라주 구현에서 인접 행렬을 권장하지 않으며, 각 정점의 이웃을 찾는 데 상당한 반복이 필요하기 때문이라고 설명한다. 근거 대조 문구: 각 정점의 이웃을 찾기 위해 상당한 양의 반복문	(K 7.7, PDF p.301)
U22-028	실습 문제 17의 미로에서 점수를 무한히 낮출 경로의 존재 여부는, 미로를 가중 그래프로 표현하고 음수 가중치 사이클이 있는지 판단하는 것으로 해석한다. 근거 대조 문구: 미로를 가중 그래프로 표현하고, 여기에 음수 가중치 사이클	(K 7.7.2, PDF p.310)
U22-029	본문은 모든 정점 사이의 최단 거리에는 존슨 알고리즘을, 하나의 시작 정점에서 나머지 정점까지의 최단 거리에는 존슨보다 벨만-포드를 사용하는 것이 효율적이라고 비교한다. 근거 대조 문구: 존슨 알고리즘보다 벨만-포드 알고리즘을 사용하는 것이 더 효율적	(K 7.8, PDF p.311)
U22-030	그래프 구현 방식은 데이터 특성과 문제 세부 사항에 따라 선택하며, 성능 외에 코드의 가독성과 간결성을 고려할 수도 있다. 근거 대조 문구: 구현 코드의 가독성 및 간결성 측면	(K 7.8, PDF p.311)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U22-031	구현에 앞서 기본 목적과 필요한 기능, 문제 정보를 나타내는 가장 기본적인 구성 요소를 숙지해야 한다. 근거 대조 문구: 주어진 문제와 연관된 정보를 나타내는 데 필요한 가장 기본적인 구성 요소	(K 7.8, PDF p.312)
U22-032	distance 배열을 UNKNOWN으로 초기화하고 출발 정점의 값만 0으로 설정한다. 근거 대조 문구: 출발 정점에 대한 distance 값은 0으로 초기화	(K 7.3.1, PDF p.274)
U22-033	전체 에지를 검토하며 거리 값을 갱신하는 반복을 (V - 1)번 수행한다. 근거 대조 문구: (V - 1)번 반복되는 루프	(K 7.3.1, PDF p.275)
U22-034	BellmanFord()의 결과 h가 비어 있으면 종료하고, 그렇지 않으면 모든 에지에 가중치 변환 수식을 적용한다. 근거 대조 문구: 만약 h가 비어 있으면 함수를 종료	(K 7.5.1, PDF p.291)
U22-035	각 정점에서 다익스트라를 호출하여 변환된 그래프의 모든 정점 간 최단 거리를 shortest에 저장한다. 근거 대조 문구: 다익스트라 알고리즘 구현 함수를 V번 호출	(K 7.5.1, PDF p.291-292)
U22-036	원본 그래프의 DFS가 끝나면 그래프를 전치한다. 근거 대조 문구: 같은 작업을 전치 그래프에 대해 수행	(K 7.7, PDF p.300)

U22-037	연습 문제 33: 음수 가중치 사이클 검사 원문 발췌 (원문 코드 발췌)
	for (auto& e : edges)
	{
	if (distance[e.src] == UNKNOWN)
	continue;
	
	if (distance[e.dst] > distance[e.src] + e.weight)
	{
	cout << "음수 가중치 사이클 발견!" << endl;
	return {};
	}
	}
	근거 대조 문구: 음수 가중치 사이클이 있는지 검사
		(K 7.4.1, PDF p.278)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U22-038	연습 문제 34: 에지 가중치 재설정 원문 발췌 (원문 코드 발췌)
	// 에지 가중치 재설정
	for (auto& e : edges)
	{
	e.weight += (h[e.src] - h[e.dst]);
	}
	근거 대조 문구: 모든 에지에 대해 가중치 변환 수식을 적용
		(K 7.5.1, PDF p.291)

U22-039	존슨 알고리즘의 에지 가중치 변환: W(uv) = w(uv) + d[s, u] - d[s, v]; w(uv)는 초기 에지 가중치, d[s, u]와 d[s, v]는 더미 정점 s로부터의 최단 거리, W(uv)는 변환된 가중치이다. 근거 대조 문구: 이제 각 에지 가중치에 다음 수식을 적용	(K 7.5, PDF p.286)
U22-040	코사라주 알고리즘의 시간 복잡도: O(V + E); 입력 그래프를 인접 리스트로 표현한 경우이다. 근거 대조 문구: 입력 그래프가 인접 리스트로 표현되어 있을 경우	(K 7.7, PDF p.301)
U22-041	모든 정점 사이의 최단 거리 근거 대조 문구: 그래프의 모든 정점 사이의 최단 거리	(K 7.8, PDF p.311)
U22-042	존슨 알고리즘 근거 대조 문구: 존슨 알고리즘을 사용하는 것이 가장 좋은 선택	(K 7.8, PDF p.311)
U22-043	하나의 시작 정점에서 나머지 정점까지의 최단 거리 근거 대조 문구: 하나의 시작 정점에서 나머지 정점들 사이의 최단 거리	(K 7.8, PDF p.311)
U22-044	존슨보다 벨만-포드가 효율적 근거 대조 문구: 존슨 알고리즘보다 벨만-포드 알고리즘을 사용하는 것이 더 효율적	(K 7.8, PDF p.311)
U22-045	그래프 기반 솔루션으로 복잡한 데이터를 비교적 쉽게 조작하고 다루는 방법을 살펴봤다. 근거 대조 문구: 그래프 기반 솔루션을 사용하여 복잡한 데이터를 비교적 쉽게 조작	(K 7.9, PDF p.312)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U22-046	6장의 그래프 구조와 순회 기본 원리를 확장하여 고급 문제를 해결했다. 근거 대조 문구: 좀 더 고급 문제를 해결하기 위해 기본 개념을 확장	(K 7.9, PDF p.312)
U22-047	기본 개념을 제대로 이해하는 것은 더 심도 있는 그래프 구현을 탐구하기 위해 중요하다. 근거 대조 문구: 이러한 기본 개념을 제대로 이해하는 것은 매우 중요	(K 7.9, PDF p.312)

23 동적 계획법 I

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U23-001	주어진 문제에 동적 계획법을 적용할 수 있는지 분석하고 메모이제이션과 타뷸레이션을 비교한다. 근거 대조 문구: 동적 계획법을 적용할 수 있는지 분석 / 메모이제이션과 타뷸레이션 기법을 서로 비교	(K 8장 학습 목표, PDF p.315)
U23-002	직관적인 전수 조사부터 점진적으로 최적화 알고리즘을 구현하며 동적 계획법 해법을 개발한다. 근거 대조 문구: 직관적인 전수 조사 / 점진적으로 최적화 알고리즘을 구현	(K 8장 학습 목표, PDF p.315)
U23-003	중복되는 부분 문제는 같은 부분 문제가 반복적으로 나타나는 특성이다. 근거 대조 문구: 같은 부분 문제가 반복적으로 나타납니다	(K 8.2 동적 계획법이란?, PDF p.318)
U23-004	최적 부분 구조는 전체 문제의 최적해를 부분 문제의 최적해 조합으로 표현할 수 있는 특성이다. 근거 대조 문구: 전체 문제에 대한 최적해가 부분 문제의 최적해의 조합으로 표현	(K 8.2 동적 계획법이란?, PDF p.318)
U23-005	책은 동적 계획법을 적용할 문제에 중복되는 부분 문제와 최적 부분 구조의 두 속성이 필요하다고 설명한다. 근거 대조 문구: 이 두 가지 속성을 만족해야 합니다	(K 8.2 동적 계획법이란?, PDF p.318)
U23-006	기저 조건(base case)은 더 이상의 재귀 없이 해를 구할 수 있는 지점이다. 근거 대조 문구: 더 이상 재귀가 없어도 해를 구할 수 있는 지점	(K 8.2 동적 계획법이란?, PDF p.316)
U23-007	상태(state)를 정할 때는 각 부분 문제를 다른 부분 문제와 다르다고 판단할 수 있는 기준을 찾아야 한다. 근거 대조 문구: 각 부분 문제를 다른 부분 문제와 다르다고 판단할 수 있는 기준	(K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)
U23-008	메모이제이션(memoization)은 하향식 방식에서 부분 문제의 해를 캐시에 넣어 사용하는 기술이다. 근거 대조 문구: 하향식 방식에서 부분 문제의 해를 캐시에 넣어 사용하는 기술	(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U23-009	함수가 호출되면 먼저 현재 상태가 캐시에 있는지 확인하고 이미 저장되어 있으면 그 값을 반환한다. 근거 대조 문구: 먼저 F(n)의 상태가 이미 캐시에 저장 / 이 값을 반환	(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.318)
U23-010	캐시 키는 서로 다른 상태를 충돌 없이 표현해야 하고, 저장값을 다시 참조할 수 있어야 한다. 근거 대조 문구: 유효성 / 서로 다른 상태에 대해 충돌 / 유용성 / 저장된 값을 참조	(K 8.5.7 메모이제이션·캐시 사용하기, PDF p.338)
U23-011	하향식 접근은 재귀 호출이 너무 많으면 호출 스택 메모리를 초과하는 스택 오버플로가 발생할 수 있다. 근거 대조 문구: 너무 많은 재귀 함수 호출 / 정해진 호출 스택 메모리보다 많은 메모리	(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.320)
U23-012	타뷸레이션(tabulation)은 기저 조건 해답부터 시작하여 부분 문제의 해답을 표에 저장하고 재사용하는 방식이다. 근거 대조 문구: 기저 조건 해답부터 시작하여 모든 부분 문제에 대한 해답을 표에 저장한 후 재사용	(K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)
U23-013	타뷸레이션은 앞부분 상태부터 채운 값을 나중 상태에서 재사용한다. 근거 대조 문구: 앞부분 상태부터 채워나가며 이 상태 값을 나중에 재사용	(K 8.6.5 하향식에서 상향식으로 바꾸기, PDF p.367)
U23-014	부분집합의 합 상태에는 현재까지의 합과 현재 처리하는 집합 인덱스가 필요하다. 근거 대조 문구: 현재까지의 부분집합의 합과 현재 처리하고 있는 집합의 인덱스	(K 8.5.9 타뷸레이션, PDF p.343)
U23-015	부분집합의 합 문제는 음수가 아닌 정수 집합 S와 정수 x에 대해 원소 합이 x인 S의 부분집합이 존재하는지를 묻는다. 근거 대조 문구: 음수가 아닌 정수로 이루어진 집합 S와 임의의 정수 x / 합이 x와 같은 부분집합이 존재	(K 8.5 부분집합의 합 문제, PDF p.322)
U23-016	책은 전수 조사, 백트래킹, 메모이제이션, 타뷸레이션의 순서로 접근 방법을 발전시킨다. 근거 대조 문구: 전수 조사, 백트래킹, 메모이제이션, 타뷸레이션	(K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U23-017	전수 조사는 단순한 문제 이해와 동적 계획법의 결과를 비교할 정답을 마련하는 데 도움이 된다. 근거 대조 문구: 단순성 / 정답 비교를 위한 수단	(K 8.5.3 전수 조사, PDF p.325)
U23-018	백트래킹은 유효하지 않은 부분 문제의 경우를 제거하여 알고리즘을 개선한다. 근거 대조 문구: 유효하지 않은 모든 경우를 제거하는 백트래킹	(K 8.5.5 최적화 적용하기 - 백트래킹, PDF p.330)
U23-019	부분집합 합은 실제 구성 원소를 모두 보관하지 않고 sum과 i 두 정수로 중간 상태를 표현할 수 있다. 근거 대조 문구: 중간 단계 상태를 sum과 i 두 개의 정수로 표현	(K 8.5.6 백트래킹을 사용하여 부분집합의 합 문제 풀기, PDF p.333)
U23-020	부분집합 합만 캐시 키로 쓰면 같은 합에 도달한 서로 다른 경우를 구분하지 못하여 잘못된 결과가 생길 수 있다. 근거 대조 문구: 같은 sum 값에 도달할 수 있는 여러 가지 경우의 수 / 잘못된 캐시 사용	(K 8.5.7 메모이제이션·캐시 사용하기, PDF p.338)
U23-021	최장 공통 부분 시퀀스(LCS, Longest Common Subsequence)는 두 데이터 시퀀스에 공통으로 나타나는 가장 긴 부분 시퀀스를 찾는 문제이다. 근거 대조 문구: 두 시퀀스에 공통으로 나타나는 가장 긴 부분 시퀀스	(K 8.6.1 최장 공통 부분 시퀀스 문제, PDF p.355)
U23-022	두 문자열 중 하나라도 빈 문자열이면 LCS 값은 0이다. 근거 대조 문구: 두 개의 문자열 중 하나라도 비어 있는 상태이면 LCS 값은 0	(K 8.6.3 최적 부분 구조 찾기, PDF p.364)
U23-023	두 부분 문자열의 마지막 문자가 같으면 각각 그 문자를 뺀 부분 문자열의 LCS 길이에 1을 더한다. 근거 대조 문구: 마지막 문자가 같으면 / 마지막 한 문자를 제외 / 1을 더한	(K 8.6.3 최적 부분 구조 찾기, PDF p.364)
U23-024	마지막 문자가 다르면 A에서 한 문자를 뺀 경우와 B에서 한 문자를 뺀 경우의 LCS 길이 중 최댓값을 택한다. 근거 대조 문구: 마지막 문자를 제외한 것 / 다음 두 경우의 최댓값	(K 8.6.3 최적 부분 구조 찾기, PDF p.364)
U23-025	LCS 메모이제이션은 두 문자열 길이를 각 차원 크기로 하는 2차원 벡터에 계산 결과를 저장한다. 근거 대조 문구: 결과를 2차원 벡터 / 첫 번째 차원의 크기는 A / 두 번째 차원 크기는 B	(K 8.6.3 최적 부분 구조 찾기, PDF p.364)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U23-026	여행 경로 실습은 각 도시에서 이동 가능한 최대 도시 수가 주어졌을 때 목적지까지 도달하는 방법의 가짓수를 구한다. 근거 대조 문구: 각 도시에서 이동할 수 있는 최대 도시 개수 / 도달할 수 있는 방법의 가짓수	(K 8.5.11 실습 문제 18: 여행 경로, PDF p.349)
U23-027	멜로디 순열 실습은 주어진 멜로디에 음이름 집합의 순열이 나타나는 횟수를 센다. 근거 대조 문구: 주어진 멜로디에 음 집합의 순열이 나타나는 횟수	(K 8.7 실습 문제 21: 멜로디 순열, PDF p.370)
U23-028	멜로디 실습의 유효 순열에서 음이름은 순서와 상관없이 여러 번 반복되어 사용될 수 있다. 근거 대조 문구: 순서에 상관없이 몇 번이든 반복되어 사용	(K 8.7 실습 문제 21: 멜로디 순열, PDF p.370)
U23-029	중복되는 부분 문제와 최적 부분 구조가 있는지 확인한다. 근거 대조 문구: 이 두 가지 속성을 만족	(K 8.2 동적 계획법이란?, PDF p.318)
U23-030	기저 조건과 상태를 정하고 전수 조사·백트래킹으로 문제 구조를 분석한다. 근거 대조 문구: 기저 조건과 상태 / 전수 조사, 백트래킹	(K 8.5.2 기저 조건과 상태 정의하기, PDF p.325)
U23-031	하향식 해법에서 부분 문제의 해답을 캐시에 저장하는 메모이제이션을 적용한다. 근거 대조 문구: 하향식 방식에서 부분 문제의 해를 캐시에	(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)
U23-032	기저 조건에서 시작해 표를 채우는 타뷸레이션으로 구성한다. 근거 대조 문구: 기저 조건 해답부터 시작 / 표에 저장	(K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)
U23-033	부분집합의 합과 인덱스를 0으로 설정하여 시작한다. 근거 대조 문구: 부분집합의 합과 인덱스를 0으로 설정	(K 8.5.9 타뷸레이션, PDF p.344)
U23-034	인덱스 i까지 합 x를 만들 수 있으면 i+1까지도 합 x를 만들 수 있다. 근거 대조 문구: i + 1 사이에서도 x와 같은 부분집합의 합	(K 8.5.9 타뷸레이션, PDF p.344)
U23-035	같은 상태에서 현재 원소를 추가하면 i+1까지 합 x + set[i]도 만들 수 있다. 근거 대조 문구: i + 1 사이에서 x + set[i]와 같은 부분집합의 합	(K 8.5.9 타뷸레이션, PDF p.344)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U23-036	메모이제이션 피보나치 함수 - 원문 발췌 (원문 코드 발췌)
	int Fibonacci(int n)
	{
	if (n < 2)
	return n;
	if (memo[n] != UNKNOWN)
	return memo[n];
	int result = Fibonacci(n - 1) + Fibonacci(n - 2);
	memo[n] = result;
	return result;
	}
	근거 대조 문구: if (memo[n] != UNKNOWN) return memo[n]; / memo[n] = result;
		(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

U23-037	원문은 UNKNOWN = -1, MAX_SIZE = 100인 상수와 memo(MAX_SIZE, UNKNOWN) 벡터를 함수 앞에 정의한다. 근거 대조 문구: const int UNKNOWN = -1; / const int MAX_SIZE = 100; / vector<int> memo(MAX_SIZE, UNKNOWN);	(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)
U23-038	피보나치 수열의 기저 조건과 재귀 관계: F(0) = 0 F(1) = 1 F(n) = F(n-1) + F(n-2); 세 번째 이후 원소는 바로 앞 두 원소의 합이다. 근거 대조 문구: F(0) = 0 / F(1) = 1 / F(n) = F(n-1) + F(n-2)	(K 8.2 동적 계획법이란?, PDF p.316)
U23-039	LCS 테이블을 채우는 원문 로직: i = 0 또는 j = 0: LCS(i, j) = 0 마지막 문자가 같음: LCS(i, j) = LCS(i - 1, j - 1) + 1 그렇지 않음: LCS(i - 1, j)와 LCS(i, j - 1) 중 최댓값; i와 j는 두 부분 문자열의 길이에 해당하는 상태이다. 근거 대조 문구: LCS(i, j) = 0 / LCS(i - 1, j - 1) + 1 / 둘 중에 최댓값	(K 8.6.5 하향식에서 상향식으로 바꾸기, PDF p.367)
U23-040	메모이제이션 근거 대조 문구: 메모이제이션	(K 8.3 메모이제이션: 하향식 접근 방법, PDF p.319)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U23-041	타뷸레이션 근거 대조 문구: 타뷸레이션	(K 8.4 타뷸레이션: 상향식 접근 방법, PDF p.320)
U23-042	이 장은 두 가지 전형적인 동적 계획법 문제를 분석하고 해결 방법을 구현했다. 근거 대조 문구: 두 가지 전형적인 동적 계획법 문제를 분석 / 해결 방법을 구현	(K 8.8 나가며, PDF p.373)
U23-043	상태·기저 조건·재귀 관계로 복잡한 문제를 더 단순한 구성 요소로 분해했다. 근거 대조 문구: 상태, 기저 조건, 재귀 관계 / 단순한 구성 요소로 분해	(K 8.8 나가며, PDF p.373)
U23-044	책은 점진적 접근이 유일한 해법은 아니며 다양한 문제를 지속해서 연습해야 한다고 정리한다. 근거 대조 문구: 유일한 방법은 아닙니다 / 지속적인 연습	(K 8.8 나가며, PDF p.373)
U23-045	중복되는 부분 문제와 최적 부분 구조이다. 근거 대조 문구: 중복되는 부분 문제 / 최적 부분 구조	(K 8.2 동적 계획법이란?, PDF p.318)
U23-046	메모이제이션은 하향식이며, 타뷸레이션은 기저 조건 해답에서 시작하는 상향식이다. 근거 대조 문구: 하향식 방식 / 기저 조건 해답부터 시작	(K 8.3~8.4, PDF p.319-320)
U23-047	같은 합에 도달한 서로 다른 부분집합 구성과 상태가 구분되지 않아 잘못된 캐시 결과를 쓸 수 있다. 근거 대조 문구: 같은 sum 값에 도달할 수 있는 여러 가지 경우의 수	(K 8.5.7 메모이제이션, PDF p.338)
U23-048	각각 마지막 문자를 제외한 문자열의 LCS 길이에 1을 더한다. 근거 대조 문구: 마지막 한 문자를 제외한 문자열 / 1을 더한	(K 8.6.3 최적 부분 구조 찾기, PDF p.364)

24 동적 계획법 II

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U24-001	0-1 배낭 문제와 무한 개수 배낭 문제를 구현하고 상태 공간 축소를 적용한다. 근거 대조 문구: 0-1 배낭 문제와 무한 개수 배낭 문제 / 상태 공간 축소	(K 9장 학습 목표, PDF p.375)
U24-002	가중 그래프의 모든 쌍 최단 경로 문제를 동적 계획법의 최적화 접근으로 해결한다. 근거 대조 문구: 가중 그래프에서 모든 쌍 최단 경로 문제	(K 9장 학습 목표, PDF p.375)
U24-003	P 문제는 다항 시간(polynomial time) 내에 해답을 구할 수 있는 문제이다. 근거 대조 문구: P 문제 / 다항 시간 내에 해답을 구할 수	(K 9.2 P와 NP, PDF p.377)
U24-004	9.2절은 정점에 이미 할당된 색상을 검증하는 문제와 조건에 맞는 색상을 배정할 수 있는지 찾는 문제를 비교한다. 근거 대조 문구: 세 가지 색상 / 인접한 두 정점이 같은 / 지정할 수 있는지	(K 9.2 P와 NP, PDF p.377)
U24-005	원문은 주어진 솔루션의 검증이 쉽다는 점과 실제 솔루션을 만들어내는 어려움을 구분하며 P = NP인가를 미해결 문제로 소개한다. 근거 대조 문구: 솔루션을 검증하는 것은 쉬운 반면 / 만들어내는 작업 / 미해결 문제 / P = NP	(K 9.2 P와 NP, PDF p.378)
U24-006	부분집합 합의 유효성 검증은 원소를 모두 더하여 목표치와 같은지 확인하므로 부분집합 크기에 대해 선형이다. 근거 대조 문구: 원소를 모두 더하여 목표치와 같은지 / 크기에 대해 선형	(K 9.3 부분집합의 합 문제 다시 보기, PDF p.380)
U24-007	원문은 부분집합 합의 동적 계획법을 의사 다항 시간(pseudo-polynomial time)으로 분류한다. 근거 대조 문구: 의사 다항 시간(pseudo-polynomial time)으로 동작	(K 9.3 부분집합의 합 문제 다시 보기, PDF p.381)
U24-008	배낭 문제는 제한 용량을 넘지 않으면서 넣은 물건들의 가격 합을 최대화하는 조합을 찾는 문제이다. 근거 대조 문구: 제한 용량을 넘지 않으면서 / 가격 합이 최대	(K 9.4 배낭 문제, PDF p.381)
U24-009	0-1 배낭에서는 물건을 추가해도 용량을 넘지 않으면 가격을 더하는 선택과, 기존 가격 합을 유지하는 선택을 비교한다. 근거 대조 문구: 무게에 w를 더한 결과가 최대 용량보다 같거나 작을 경우 / 가격 합을 그대로 유지	(K 9.4.1 0-1 배낭 문제, PDF p.383)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U24-010	원문의 0-1 배낭 DP는 이전 행의 같은 무게 값과 현재 물건을 넣는 경우의 가격 합 중 최댓값을 저장한다. 근거 대조 문구: max(DP[i - 1][totalWeight], DP[i - 1][totalWeight - currentWeight] + currentValue)	(K 9.4.2 연습 문제 41: 0-1 배낭 문제, PDF p.386)
U24-011	무한 개수 배낭 문제(unbounded knapsack problem)는 각 물건을 개수 제한 없이 배낭에 담을 수 있는 문제이다. 근거 대조 문구: 각 물건을 개수 제한없이 배낭에 담을 수	(K 9.4.3 무한 개수 배낭 문제, PDF p.386)
U24-012	무한 개수 배낭은 상태 전환 조건이 무게에만 의존하여 1차원 DP 테이블로 표현할 수 있다. 근거 대조 문구: 오직 무게 weight에만 의존 / 1차원 DP 테이블	(K 9.4.4 상태 공간 축소, PDF p.394)
U24-013	상태 공간 축소(state space reduction)는 상태를 표현하는 공간의 크기를 최소화하도록 동적 계획법 알고리즘을 재구성하는 작업이다. 근거 대조 문구: 상태를 표현하기 위해 사용하는 공간의 크기가 최소화하도록 / 재구성	(K 9.4.4 상태 공간 축소, PDF p.389)
U24-014	파스칼의 삼각형은 각 행에 필요한 만큼만 공간을 만들고 행의 좌우 대칭을 활용하여 중복 계산을 줄일 수 있다. 근거 대조 문구: 각각의 행을 딱 필요한 크기로 / 각 행의 절반이 서로 대칭	(K 9.4.4 상태 공간 축소, PDF p.391)
U24-015	파스칼의 삼각형에서 특정 원소만 필요하면 현재 행 계산에 이전 행만 필요하므로 전체 DP 테이블을 저장할 필요가 없다. 근거 대조 문구: 특정 원소만 / 전체를 저장할 필요가 없습니다 / 바로 이전 행	(K 9.4.4 상태 공간 축소, PDF p.392)
U24-016	단일 시작 최단 경로 문제(single-source shortest path problem)는 한 정점에서 나머지 모든 정점까지의 최단 경로를 찾는다. 근거 대조 문구: 하나의 정점에서 시작하여 나머지 모든 정점까지의 최단 경로	(K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)
U24-017	모든 쌍 최단 경로 문제(all-pairs shortest path problem)는 모든 정점 사이의 최단 경로를 계산한다. 근거 대조 문구: 모든 정점 사이의 최단 경로 / 모든 쌍 최단 경로 문제	(K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U24-018	존슨 알고리즘은 벨만-포드의 동적 계획법 접근과 다익스트라의 그리디 접근을 결합한다. 근거 대조 문구: 벨만-포드 / 동적 계획법 / 다익스트라 / 그리디 / 결합	(K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405)
U24-019	최단 경로 메모이제이션은 현재 정점 node와 반복수 depth의 쌍으로 상태를 표현한다. 근거 대조 문구: node → 현재 정점 / depth → 현재 반복수 / node와 depth의 쌍	(K 9.5.2 동적 계획법으로 최단 경로 문제 다루기, PDF p.405-406)
U24-020	플로이드-워셜은 모든 정점을 중간 정점으로 사용하면서 모든 정점 쌍 사이의 거리를 테이블에 저장한다. 근거 대조 문구: 모든 정점을 중간 정점으로 / 모든 정점 쌍 사이의 최단 거리	(K 9.5.5 플로이드-워셜 알고리즘, PDF p.413)
U24-021	현재 start-end 거리보다 start-mid와 mid-end 거리의 합이 짧으면 그 합으로 갱신한다. 근거 대조 문구: distance[start][mid] + distance[mid][end] < distance[start][end]	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)
U24-022	중첩 반복문의 순서는 가장 바깥쪽 mid, 그다음 start, 가장 안쪽 end이다. 근거 대조 문구: 가장 바깥쪽 / mid / 가운데 / start / 가장 안쪽 / end	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)
U24-023	테이블의 distance[i][i]가 0보다 작으면 음수 가중치 사이클이 있는 것으로 판단한다. 근거 대조 문구: 자기 자신으로의 거리가 0보다 작으면 음수 가중치 사이클	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)
U24-024	원문은 밀집 그래프에서 플로이드-워셜을 고려하며, 음수 에지 가중치는 허용하되 음수 사이클에 별도 처리가 필요하다고 설명한다. 근거 대조 문구: 밀집 그래프 / 음수 에지 가중치 / 음수 에지 사이클 / 별도의 처리	(K 9.5.5 플로이드-워셜 알고리즘, PDF p.414)
U24-025	최대 이익 실습은 capacity와 budget의 두 제약을 고려하기 위해 3차원 DP 테이블을 사용하도록 안내한다. 근거 대조 문구: capacity와 budget 두 개의 제약 / DP 테이블은 3차원	(K 9.4.6 실습 문제 22: 최대 이익, PDF p.401)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U24-026	도로 건설 실습은 지반 강도에 따라 비용이 달라지는 지역에서 저렴한 도로 시스템을 설계하는 문제이다. 근거 대조 문구: 최대한 저렴하게 도로 시스템 / 지반 강도 / 비용이 선형으로 증가	(K 9.5.7 실습 문제 23: 도로 건설, PDF p.419)
U24-027	각 정점 거리를 UNKNOWN으로 초기화하고 시작 정점은 0으로 둔다. 근거 대조 문구: UNKNOWN으로 초기화 / start 정점의 거리 값은 0	(K 9.5.1 벨만-포드 알고리즘 다시 보기, PDF p.402)
U24-028	V-1번 반복하며 모든 에지를 검사하고 시작 정점의 거리가 알려진 경우 인접 정점까지의 후보 거리를 비교한다. 근거 대조 문구: 1부터 (V - 1)까지 반복 / 모든 에지를 검사	(K 9.5.1 벨만-포드 알고리즘 다시 보기, PDF p.402)
U24-029	더 짧은 후보 거리로 갱신하며, 마지막에 한 번 더 검사하여 음수 사이클 존재 여부를 확인한다. 근거 대조 문구: 더 작은 값으로 갱신 / 반복을 한 번 더 수행 / 음수 가중치 사이클	(K 9.5.1 벨만-포드 알고리즘 다시 보기, PDF p.402)
U24-030	distance 테이블을 가중치 행렬로 초기화하고 자기 자신까지의 거리는 0으로 설정한다. 근거 대조 문구: 가중치 행렬을 참조 / 자기 자신까지의 거리 값은 0	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.415)
U24-031	mid, start, end의 중첩 반복문에서 더 짧은 거리를 발견할 때 테이블을 갱신한다. 근거 대조 문구: mid / start / end / 더 짧은 거리 / 갱신	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)
U24-032	대각 원소가 음수인지 확인하여 음수 가중치 사이클을 검사하고 distance를 반환한다. 근거 대조 문구: distance[i][i] < 0 / return distance;	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치

U24-033	플로이드-워셜 거리 갱신 - 원문 발췌 (원문 코드 발췌)
	for (int mid = 0; mid < V; mid++) {
	 for (int start = 0; start < V; start++) {
	 for (int end = 0; end < V; end++) {
	 if (distance[start][mid] + distance[mid][end] < distance[start][end]) {
	 distance[start][end] = distance[start][mid] + distance[mid][end];
	 }
	 }
	 }
	}
	근거 대조 문구: for (int mid = 0; mid < V; mid++) / distance[start][end] = distance[start][mid] + distance[mid][end];
		(K 9.5.6 연습 문제 44: 플로이드-워셜 알고리즘 구현하기, PDF p.416)

U24-034	파스칼의 삼각형: 원문 기저 조건·순환식: m = 1, m = n → triangle(n, m) = 1 triangle(n, m) = triangle(n-1, m-1) + triangle(n-1, m); m ≤ n인 두 정수 m, n에 대한 원문 표기. 근거 대조 문구: m = 1, m = n / triangle(n, m) = triangle(n-1, m-1) + triangle(n-1, m)	(K 9.4.4 상태 공간 축소, PDF p.389)
U24-035	플로이드-워셜 알고리즘 복잡도: 시간 O(V³) · 메모리 O(V²); V는 그래프 정점 수이다. 근거 대조 문구: O(V³)의 시간 복잡도와 O(V²)의 메모리 사용량	(K 9.5.4 모든 쌍 최단 경로, PDF p.413)
U24-036	0-1 배낭 근거 대조 문구: 0-1 배낭 문제	(K 9.4.4 상태 공간 축소, PDF p.394)
U24-037	i번째 물건 포함 여부를 결정하기 위해 (i - 1)개 물건의 상태를 고려한다. 근거 대조 문구: (i - 1)개의 물건들만 고려	(K 9.4.4 상태 공간 축소, PDF p.394)
U24-038	무한 개수 배낭 근거 대조 문구: 무한 개수 배낭 문제	(K 9.4.4 상태 공간 축소, PDF p.394)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U24-039	같은 물건을 중복 포함할 수 있고, 상태 전환은 무게에만 의존하므로 1차원 DP로 표현할 수 있다. 근거 대조 문구: 현재 물건을 중복해서 포함 / 오직 무게 weight에만 의존 / 1차원 DP 테이블	(K 9.4.4 상태 공간 축소, PDF p.394)
U24-040	익숙한 문제를 동적 계획법 관점에서 바라보는 것은 적절한 해법을 작성하는 핵심 아이디어를 제공한다. 근거 대조 문구: 익숙한 문제를 동적 계획법 관점 / 핵심 아이디어	(K 9.6 나가며, PDF p.424)
U24-041	배운 기법을 언제 사용할지는 하나로 정해져 있지 않으므로 여러 맥락에 적용해보는 것이 좋다. 근거 대조 문구: 명확하게 정해져 있지 않으며 / 여러 맥락에 다양하게 적용	(K 9.6 나가며, PDF p.424)
U24-042	책은 자료 구조와 여러 기법을 적절하게 사용하는 감각을 얻고 더 깊이 탐구하기를 권한다. 근거 대조 문구: 자료 구조와 여러 기법을 적절하게 / 더 깊이 탐구	(K 9.6 나가며, PDF p.424)
U24-043	M은 부분집합 합의 목표치이고, 원문은 해법을 의사 다항 시간으로 분류한다. 근거 대조 문구: M은 부분집합의 합 목표치 / 의사 다항 시간	(K 9.3 부분집합의 합 문제 다시 보기, PDF p.380-381)
U24-044	상태 전환 조건이 물건 인덱스가 아니라 무게에만 의존하기 때문이다. 근거 대조 문구: 오직 무게 weight에만 의존	(K 9.4.4 상태 공간 축소, PDF p.394)
U24-045	중간 정점 mid, 시작 정점 start, 목표 정점 end 순서이다. 근거 대조 문구: 가장 바깥쪽 / mid / 가운데 / start / 가장 안쪽 / end	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)
U24-046	distance[i][i]가 0보다 작은지 확인한다. 근거 대조 문구: distance[i][i] < 0	(K 9.5.6 플로이드-워셜 알고리즘 구현하기, PDF p.416)

25 실습 문제 풀이

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-001	각 장의 실습 문제를 풀기 위해 수행해야 할 세부 단계를 코드와 함께 확인한다. 근거 대조 문구: 각 장의 실습 문제를 풀기 위해 수행해야 할 세부 단계를 코드와 함께 설명합니다.	(K 부록 실습 문제 풀이, PDF p.426)
U25-002	리스트·트리·해시·분할 정복·그리디·그래프·동적 계획법의 실습 풀이를 다룬다. 근거 대조 문구: A.1 1장 리스트, 스택, 큐 / A.9 9장 동적 계획법 II	(K 부록 A.1~A.9, PDF p.427,441,447,454,463,469,476,494,517)
U25-003	음악 재생 목록을 저장하고 필요한 기능을 제공하기 위해 수정된 형태의 이중 연결 리스트를 구현한다. 근거 대조 문구: 수정된 형태의 이중 연결 리스트를 구현합니다.	(K A.1.1 음악 재생 목록 구현하기, PDF p.427)
U25-004	한 벌의 카드를 만들고 잘 섞은 후 네 명의 플레이어에게 나눠주며, game 구조체에 이 기능을 구현한다. 근거 대조 문구: 한 벌의 카드를 만들고 잘 섞은 후 네 명의 플레이어에게 나눠줍니다.	(K A.1.2 카드 게임 시뮬레이션, PDF p.435)
U25-005	한 라운드 경기에서는 두 플레이어의 카드를 비교하여 서로 같으면 제거하는 함수를 따로 만든다. 근거 대조 문구: 두 플레이어의 카드를 비교하여 서로 같으면 제거하는 함수를 따로 만들겠습니다.	(K A.1.2 카드 게임 시뮬레이션, PDF p.435)
U25-006	Printer 클래스는 요청된 순서대로 프린트하기 위해 std::queue를 사용하고, 최대 인쇄 요청 개수를 템플릿 인자로 받는다. 근거 대조 문구: 요청된 순서대로 프린트를 하기 위해 std::queue를 사용할 것입니다.	(K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)
U25-007	startPrinting()은 대기열이 빌 때까지 jobs.front()를 출력하고 jobs.pop()으로 제거한다. 근거 대조 문구: while (not jobs.empty()) / jobs.front() / jobs.pop();	(K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)
U25-008	파일 시스템 표현에는 N-항 트리를 사용하고, 생성자에서 루트 디렉터리를 만든 뒤 현재 디렉터리로 설정한다. 근거 대조 문구: N-항 트리를 이용하여 / 루트 디렉터리를 만들고, 이를 현재 디렉터리로 설정합니다.	(K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-009	n_ary_node는 디렉터리/파일 정보를 저장하며 name, is_dir, children을 가진다. 근거 대조 문구: 디렉터리/파일 정보를 저장할 노드 구조체를 작성합니다.	(K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)
U25-010	show_path()는 파일이 인자로 전달되면 파일 이름을, 디렉터리가 전달되면 하위 디렉터리와 파일을 모두 출력한다. 근거 대조 문구: 파일 이름을 출력하고 / 하위 디렉터리와 파일을 모두 출력합니다.	(K A.2.1 파일 시스템 자료 구조 만들기, PDF p.444)
U25-011	병합 풀이의 힙 노드는 데이터, 입력 리스트 인덱스, 해당 리스트에서의 데이터 인덱스를 저장한다. 근거 대조 문구: 힙 노드는 데이터와 입력 리스트 인덱스, 그리고 해당 입력 리스트에서의 데이터 인덱스를 저장합니다.	(K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)
U25-012	URLService는 짧은 URL을 키로, 원본 URL을 데이터 값으로 사용하여 맵에 함께 저장한다. 근거 대조 문구: 짧은 URL을 키로 사용하고, 원본 URL을 데이터 값으로 사용할 것입니다.	(K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)
U25-013	lookup()은 짧은 URL을 이용하여 긴 원본 URL을 찾는 룩업 함수이다. 근거 대조 문구: 짧은 URL을 이용하여 긴 원본 URL을 찾을 것입니다. / 룩업 함수를 추가합니다.	(K A.3.1 긴 URL을 짧은 URL로 매핑하기, PDF p.447)
U25-014	이메일 주소 중복 검사 풀이에서는 BloomFilter 클래스를 추가하고 MD5() 함수로 해시 값을 계산한다. 근거 대조 문구: 블룸 필터 클래스를 추가합니다. / MD5() 함수를 사용하여 해시 값을 계산하겠습니다.	(K A.3.2 이메일 주소 중복 검사, PDF p.451-452)
U25-015	이메일 주소를 추가할 때 hash(key)를 호출하고, 해시 값 버퍼를 반복하면서 해당 비트를 true로 설정한다. 근거 대조 문구: 해시 값 버퍼를 0부터 nHashes까지 반복하면서 각 비트를 true로 설정합니다.	(K A.3.2 이메일 주소 중복 검사, PDF p.452)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-016	needs_vaccination()은 목록에 있는 학생이 예방 접종을 받지 않았으면 true, 이미 받았으면 false를 반환하고, 목록에 없으면 true를 반환한다. 근거 대조 문구: 목록에 해당 학생이 없다면 true를 반환	(K A.4.1 예방 접종, PDF p.456)
U25-017	부분 퀵 정렬은 퀵 정렬 알고리즘을 변형하며, 마지막 피벗 위치가 k보다 작으면 오른쪽 부분 벡터도 정렬한다. 근거 대조 문구: 퀵 정렬 알고리즘을 변형하여 / 마지막 피벗 위치가 k보다 작으면 오른쪽 부분 벡터도 정렬	(K A.4.2 부분 정렬, PDF p.458-459)
U25-018	워드카운트의 맵 함수는 각 입력 파일에서 단어를 추출하고, key를 단어로, value를 1로 설정한 쌍을 생성한다. 근거 대조 문구: 여기서 key는 단어이고 value는 1로 설정됩니다.	(K A.4.3 맵리듀스를 이용하여 워드카운트 구현하기, PDF p.462)
U25-019	리듀스 작업에서는 std::accumulate()로 key에 해당하는 단어가 나타나는 총 횟수를 구한다. 근거 대조 문구: std::accumulate() 함수를 사용하여 / 단어가 나타나는 총 횟수를 구할 수 있습니다.	(K A.4.3 맵리듀스를 이용하여 워드카운트 구현하기, PDF p.463)
U25-020	작업 스케줄링은 작업 종료 시간을 기준으로 정렬한 뒤 현재 작업과 시간이 겹치는 작업을 제거한다. 근거 대조 문구: 작업 종료 시간을 기준으로 정렬 / 현재 작업과 시간이 겹치는 작업은 제거	(K A.5.1 작업 스케줄링 문제, PDF p.464-465)
U25-021	웰시-포웰 구현은 정점의 차수 기준으로 내림차순 정렬하고, 인접 정점에 현재 색상이 없으면 해당 색상을 지정한다. 근거 대조 문구: 정점의 차수 기준으로 내림차순 정렬 / 인접한 정점이 color_to_be_assigned 색상을 가지고 있지 않다면	(K A.5.2 웰시-포웰 알고리즘, PDF p.467)
U25-022	이분 그래프 판별 실습은 DFS를 이용하여 주어진 그래프가 이분 그래프인지 확인한다. 근거 대조 문구: DFS를 이용하여 주어진 그래프가 이분 그래프인지 확인	(K A.6.1 이분 그래프 판별하기, PDF p.469)
U25-023	뉴욕 최단 경로 실습은 USA-road-d.NY.gr 파일을 읽어 그래프 객체를 생성하고 다익스트라 알고리즘을 구현한다. 근거 대조 문구: USA-road-d.NY.gr 파일을 읽어서 그래프 객체를 생성 / 다익스트라 알고리즘을 구현합니다.	(K A.6.2 뉴욕에서 최단 경로 찾기, PDF p.472,474)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-024	욕심쟁이 로봇에서 에너지를 최대로 유지하는 것은 이동 시 에너지 손실을 최소로 하는 것과 같으므로, 실제 구현에서는 전력 값의 부호를 바꾼다. 근거 대조 문구: 에너지를 최대로 유지한다는 것은 / 에너지 손실을 최소로 / 전력 값의 부호를 바꿔줘야 합니다.	(K A.7.1 욕심쟁이 로봇, PDF p.478)
U25-025	로봇 풀이의 BellmanFord()는 연습 문제 33과 같으며, 호출할 때 정점 개수를 전체 셀 개수인 N * N으로 지정한다. 근거 대조 문구: 정점의 개수를 전체 셀 개수인 N * N으로 지정해야 합니다.	(K A.7.1 욕심쟁이 로봇, PDF p.479)
U25-026	무작위 그래프 풀이에서는 모든 정점에 들어오거나 나가는 에지가 적어도 하나 존재하는지 인접 행렬의 대각 원소로 확인한다. 근거 대조 문구: 모든 정점에서 들어오거나 나가는 에지가 적어도 하나 존재하는지를 확인	(K A.7.2 무작위 그래프 통계, PDF p.482)
U25-027	BellmanFord()가 반환한 거리 벡터가 비어 있지 않으면 존슨 알고리즘을 계속 수행하여 에지 가중치를 재설정하고 각각의 정점에서 다익스트라 알고리즘을 수행한다. 근거 대조 문구: 반환된 벡터가 비어 있지 않다면 존슨 알고리즘을 계속 수행합니다.	(K A.7.2 무작위 그래프 통계, PDF p.485)
U25-028	미로-순간이동 게임의 미로·방·순간이동장치·점수는 그래프·정점·에지·가중치로 바꾸어 생각하며, 점수를 무한히 낮출 수 있는 상황은 음수 가중치 사이클이 있는 경우이다. 근거 대조 문구: 미로, 방, 순간이동장치, 점수 / 그래프, 정점, 에지, 가중치 / 음수 가중치 사이클	(K A.7.3 미로-순간이동 게임, PDF p.489)
U25-029	다른 구역으로 이동하지 못하는 방을 찾으려면 강한 연결 요소 중에서 다른 연결 요소로 나가는 에지가 없는 것들을 찾고, 코사라주 구현에 필요한 로직을 추가한다. 근거 대조 문구: 다른 연결 요소로 나가는 에지가 없는 것들을 모두 찾아야 합니다.	(K A.7.3 미로-순간이동 게임, PDF p.492)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-030	여행 경로 풀이의 기저 조건은 목적지에 도착했는지를 판단하는 것이며, i번째 도시의 이동 거리 제한이 x이면 (i + 1)번째부터 (i + x)번째 도시 사이로 이동할 수 있다. 근거 대조 문구: 목적지에 도착했는지를 판단 / (i + 1)번째 도시부터 (i + x)번째 도시 사이	(K A.8.1 여행 경로, PDF p.494)
U25-031	여행 경로의 추가 점수 풀이에서는 구간 합 배열(prefix sum array)을 사용하여 안쪽 for 반복문으로 계산하던 구간 합을 상수 시간에 계산한다. 근거 대조 문구: 구간 합 배열(prefix sum array) / 구간 합을 상수 시간에 계산	(K A.8.1 여행 경로, PDF p.499)
U25-032	LCS_Memoization()은 두 문자열의 부분 문자열을 재귀적으로 탐색하며 각 단계에서 memo 테이블에 결과를 저장한다. 근거 대조 문구: 두 문자열의 부분 문자열을 재귀적으로 탐색하고, 각 단계에서 memo 테이블에 결과를 저장합니다.	(K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)
U25-033	LCS_Tabulation()은 문자열 A와 B를 인자로 받고 최장 공통 부분 시퀀스 문자열을 반환한다. 근거 대조 문구: 두 개의 문자열 A와 B를 인자로 받고, 최장 공통 부분 시퀀스 문자열을 반환합니다.	(K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)
U25-034	최장 공통 부분 시퀀스의 길이는 DP[A.size()][B.size()]에서 알 수 있고, 실제 시퀀스는 DP 테이블에 백트래킹을 수행하여 구한다. 근거 대조 문구: DP[A.size()][B.size()] / DP 테이블에 대해 백트래킹을 수행	(K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.507)
U25-035	멜로디 순열 풀이의 상태는 지금까지 고려한 멜로디의 길이와, 현재 음을 추가하거나 추가하지 않은 부분집합의 두 차원으로 표현한다. 근거 대조 문구: 각각의 상태는 2차원으로 표현 / 지금까지 고려한 멜로디의 길이(length)	(K A.8.4 멜로디 순열, PDF p.511)
U25-036	멜로디 순열 풀이에서는 각각의 음을 2의 승수로 간주하고 C++ 비트 단위 연산으로 상태에 해당하는 부분집합을 표현한다. 근거 대조 문구: 각각의 음을 2의 승수로 간주하고 C++ 비트 단위 연산을 이용	(K A.8.4 멜로디 순열, PDF p.512)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-037	최대 이익 실습에서는 선택한 상품 가격 합이 budget을, 수량이 capacity를 넘지 않으면서 이익 합이 최대가 되도록 한다. 근거 대조 문구: 가격 합은 정해진 예산(budget)을 넘지 / 수량은 창고 용량(capacity)을 넘지 / 이익 합이 최대	(K A.9.1 최대 이익, PDF p.518)
U25-038	최대 이익의 상향식 상태는 현재 인덱스, 상품 구매 비용, 상품 개수로 정의되며, 구현은 0-1 배낭 문제 풀이에서 차원이 하나 추가된 형태이다. 근거 대조 문구: 현재 인덱스, 상품 구매 비용, 상품 개수 / 0-1 배낭 문제 풀이에서 차원이 하나 추가된 형태	(K A.9.1 최대 이익, PDF p.519-520)
U25-039	도로 건설에서 모든 집이 단지 입구까지 가장 낮은 비용으로 이동하도록 플로이드-워셜 알고리즘을 사용하고, 경로 재구성을 위한 next 행렬을 둔다. 근거 대조 문구: 플로이드-워셜 알고리즘 / 경로 재구성을 위해 N×N 크기의 정수 행렬 next	(K A.9.2 도로 건설, PDF p.525)
U25-040	플로이드-워셜의 최단 거리가 갱신될 때마다 next[start][end]도 next[start][mid] 값으로 갱신한다. 근거 대조 문구: 최단 거리가 갱신될 때마다 next[start][end] 값도 next[start][mid] 값으로 갱신	(K A.9.2 도로 건설, PDF p.526)
U25-041	각 리스트의 가장 작은 원소를 추출하여 최소 힙을 구성한다. 근거 대조 문구: 각각의 리스트에서 가장 작은 원소를 추출하여 최소 힙을 구성	(K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)
U25-042	힙에서 가장 작은 원소를 꺼내어 출력 벡터에 추가한다. 근거 대조 문구: 힙에서 원소 하나를 꺼내어 출력 벡터에 추가	(K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)
U25-043	꺼낸 원소가 있던 리스트에 다음 원소가 있으면 힙에 넣고, 힙이 완전히 빌 때까지 반복한다. 근거 대조 문구: 다음 원소가 있다면 이를 다시 힙에 넣습니다. 그리고 힙이 완전히 빌 때까지	(K A.2 트리, 힙, 그래프: 병합 풀이 수록 부분, PDF p.445)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-044	입력을 처리하고 그래프를 구성한다(인접 정점 찾기, 가중치 설정하기 등). 근거 대조 문구: 입력 처리하기 / 그래프 구성하기(인접 정점 찾기, 가중치 설정하기 등)	(K A.9.2 도로 건설, PDF p.520)
U25-045	그래프 정점 사이의 최단 거리를 찾는다. 근거 대조 문구: 그래프 정점 사이의 최단 거리 찾기	(K A.9.2 도로 건설, PDF p.521)
U25-046	경로를 재구성한다. 근거 대조 문구: 경로 재구성	(K A.9.2 도로 건설, PDF p.521)
U25-047	지도를 다시 그린다. 근거 대조 문구: 지도 다시 그리기	(K A.9.2 도로 건설, PDF p.521)

U25-048	파일 시스템 노드 — 원문 발췌 (원문 코드 발췌)
	struct n_ary_node
	{
	std::string name;
	bool is_dir;
	std::vector<n_ary_node*> children;
	};
	근거 대조 문구: struct n_ary_node / name / is_dir / children
		(K A.2.1 파일 시스템 자료 구조 만들기, PDF p.441)

U25-049	인쇄 대기열 처리 — 원문 발췌 (원문 코드 발췌)
	void startPrinting()
	{
	while (not jobs.empty())
	{
	std::cout << "인쇄 중: " << jobs.front() << std::endl;
	jobs.pop();
	}
	}
	근거 대조 문구: void startPrinting() / while (not jobs.empty()) / jobs.pop();
		(K A.1.3 사무실 공유 프린터의 인쇄 대기 목록 시뮬레이션, PDF p.439)

번호	문장·원문 코드 / 근거 대조 문구	원문 위치
U25-050	구간 합 배열: sum[i + 1] = sum[i] + distance[i] [L, R] 구간의 원소 합: sum[R+1] - sum[L]; 합을 구할 전체 데이터 개수보다 하나 큰 배열 sum을 선언하고 모든 원소를 0으로 초기화한다. 근거 대조 문구: sum[i + 1] = sum[i] + distance[i] / sum[R+1] - sum[L]	(K A.8.1 여행 경로, PDF p.499)
U25-051	멜로디 음 부분집합 수: $2^{12} = 4096$; 각 음을 2의 승수로 간주하여 비트 단위 연산으로 부분집합을 표현한다. 근거 대조 문구: 하나의 음계에 12개의 음이 있으므로 전체 $2^{12} = 4096$개의 음 부분집합	(K A.8.4 멜로디 순열, PDF p.512)
U25-052	LCS_Memoization() 근거 대조 문구: int LCS_Memoization	(K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)
U25-053	int형 LCS 길이; 2차원 정수 벡터 참조형 memo에 결과를 저장한다. 근거 대조 문구: int LCS_Memoization / vector<vector<int>>& memo / 결과를 저장	(K A.8.2 메모이제이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.501)
U25-054	LCS_Tabulation() 근거 대조 문구: string LCS_Tabulation	(K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)
U25-055	string형 최장 공통 부분 시퀀스 문자열을 반환한다. 근거 대조 문구: 최장 공통 부분 시퀀스 문자열을 반환합니다.	(K A.8.3 타뷸레이션을 이용하여 최장 공통 부분 시퀀스 찾기, PDF p.505)